Universidad Escuela Colombiana de Ingeniería Julio Garavito

Desarrollo Orientado por Objetos

DOPO

Presentado por:

Wilson Steven Mendivelso Sanabria

David Santiago Garzon Rodriguez

Docente:

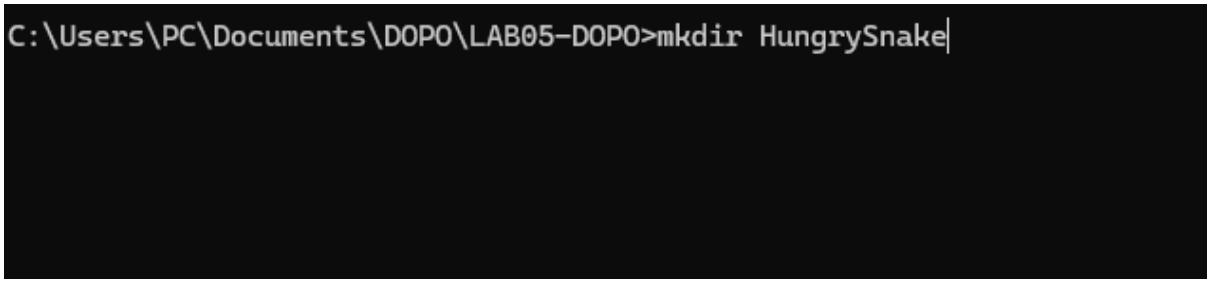
María Irma Diaz Rozo

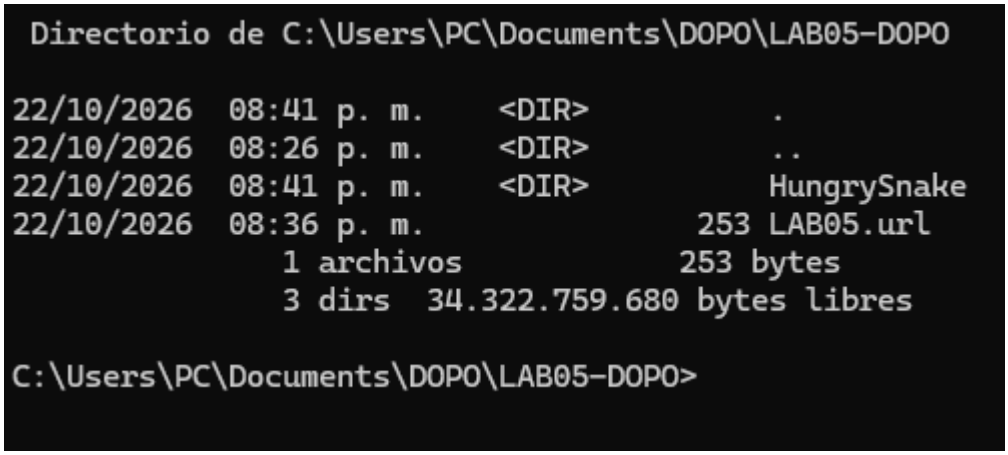
PARTE I. Maqueta

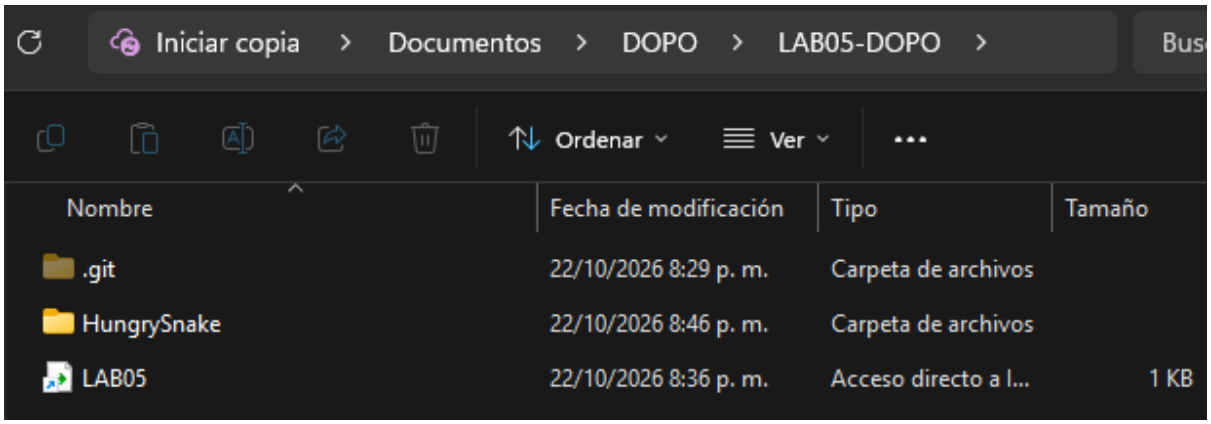
A Directorios

El objetivo de este punto es preparar los directorios para el juego HungrySnake

1. Preparen un directorio llamado HungrySnake con los directorios src, bin y lib; y los subdirectorios para presentación, dominio y pruebas de unidad. Capturen una pantalla con la estructura.







Ahora se crean los subdirectorios de src, bin y lib dentro del directorio HungrySnake:

```
C:\Users\PC\Documents\DOPO\LAB05-DOPO>cd HungrySnake

C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake>mkdir src

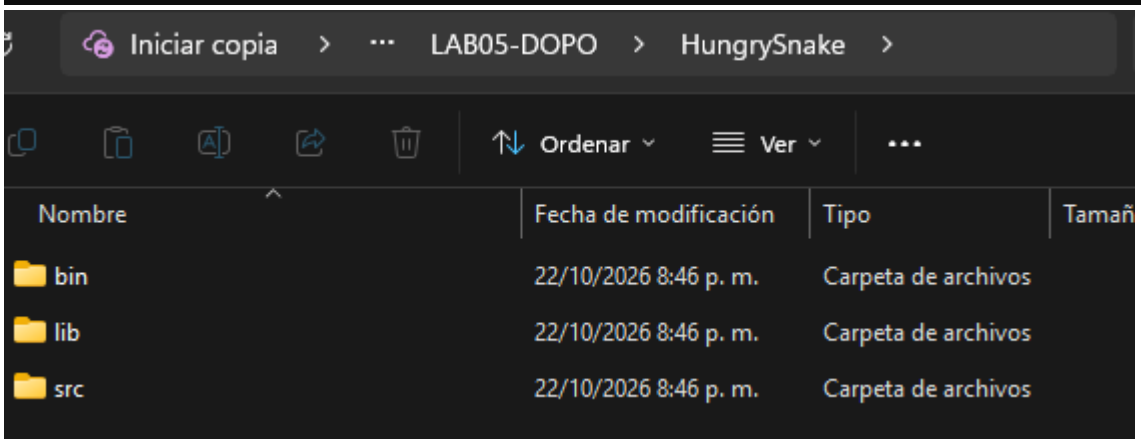
C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake>mkdir bin

C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake>mkdir lib

C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake>dir
 El volumen de la unidad C es Windows-SSD
 El número de serie del volumen es: 6C4B-201D

 Directorio de C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake

22/10/2026  08:46 p. m.    <DIR>          .
22/10/2026  08:41 p. m.    <DIR>          ..
22/10/2026  08:46 p. m.    <DIR>          bin
22/10/2026  08:46 p. m.    <DIR>          lib
22/10/2026  08:46 p. m.    <DIR>          src
               0 archivos              0 bytes
               5 dirs  34.323.517.440 bytes libres
```



Ahora, dentro de los subdirectorios src y bin, se crean los subdirectorios presentation, domain y test.

```
C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake>cd src

C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake\src>mkdir presentation

C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake\src>mkdir domain

C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake\src>mkdir test

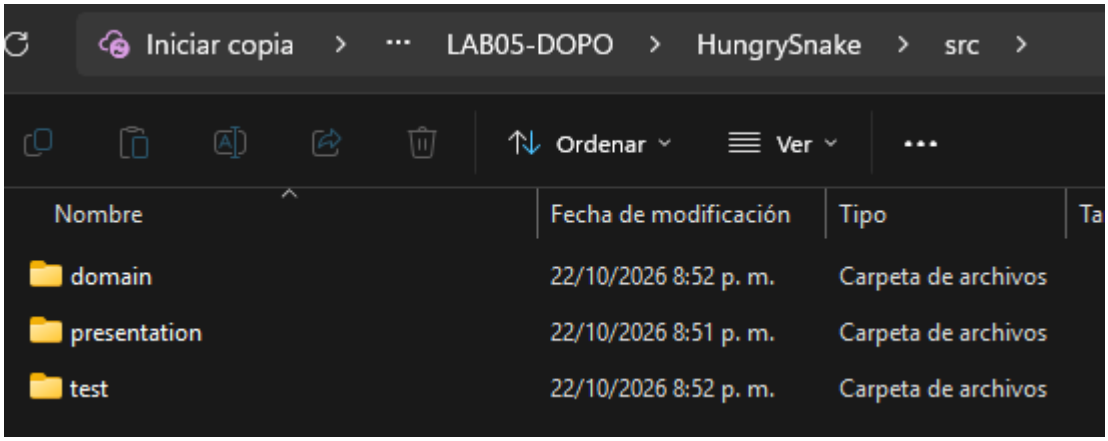
C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake\src>cd..

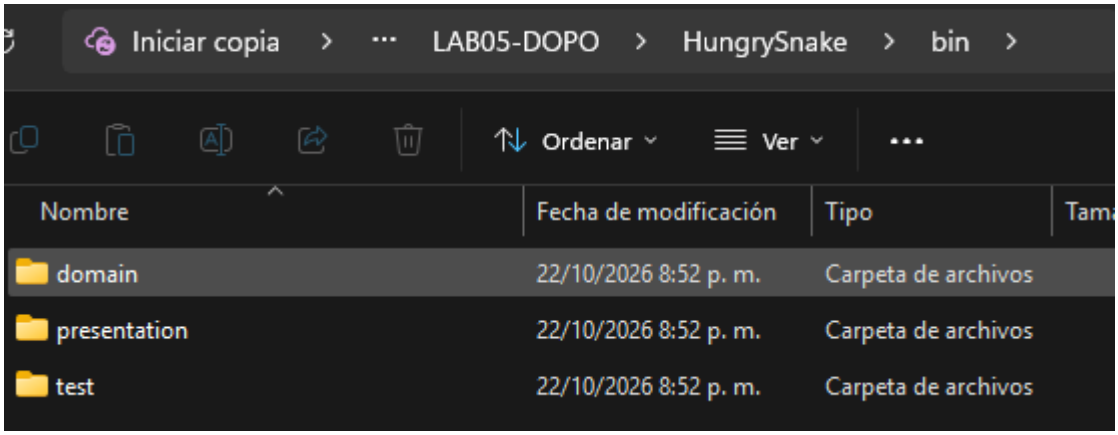
C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake>cd bin

C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake\bin>mkdir presentation

C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake\bin>mkdir domain

C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake\bin>mkdir test
```

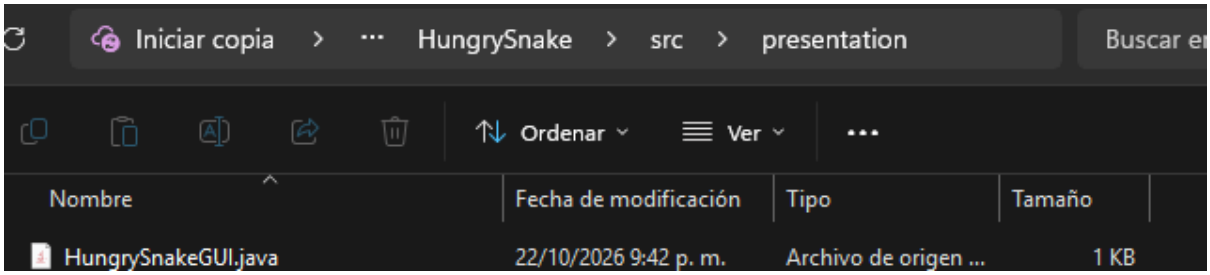




B Ciclo 0: Ventana vacía – Salir

El objetivo es implementar la ventana principal de HungrySnake con un final adecuado desde el icono de cerrar. Utilizar el esquema de prepareView-prepareController.

1. Construyan el primer esquema de la ventana de HungrySnake únicamente con el título “HungrySnake ”. Para esto creen la clase HungrySnakeGUI como un JFrame con su creador (que sólo coloca el título) y el método main que crea un objeto HungrySnakeGUI y lo hace visible. Ejecútenlo. Capturen la pantalla. (Si la ventana principal no es la inicial en su diseño, después deberán mover el main al componente visual correspondiente)




```java
package presentation;
import javax.swing.JFrame;

/**
 * Represents a HungrySnakeGUI
 * HungrySnakeGUI
 */

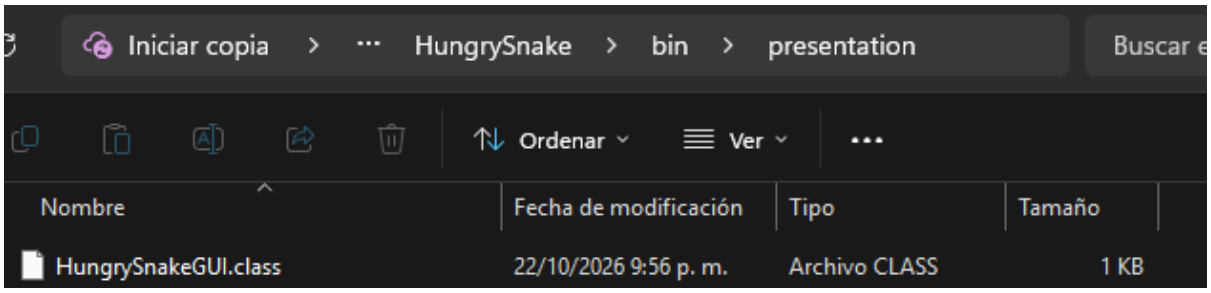
public class HungrySnakeGUI extends JFrame{

    public HungrySnakeGUI(){
        setTitle("HungrySnake");

    }


    /**
     * starts the program
     * @param args
     */
    Run main | Debug main | Run | Debug
    public static void main(String[] args){
        HungrySnakeGUI gui = new HungrySnakeGUI();
        gui.setVisible(true);
    }
}
```

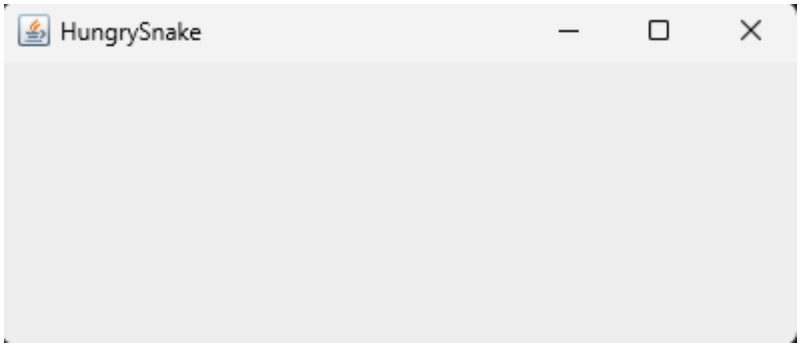
```
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake\src> cd ..
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake> javac -d bin src\presentation\HungrySnakeGUI.j
ava
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake>
```



Para ejecutar:

```
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake> javac -d bin -sourcepath src src/presentation/
HungrySnakeGUI.java
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake> java -cp bin presentation.HungrySnakeGUI
```

Para que se vea el titulo ajustamos el JFrame al ejecutarlo:



2. Modifiquen el tamaño de la ventana para que ocupe un cuarto de la pantalla y ubíquenla en el centro. Para eso inicien la codificación del método prepareView. Capturen esa pantalla.

```
private static final Dimension PREFERRED_DIMENSION = new Dimension (Toolkit.getDefaultToolkit().getScreenSize().width/2, Toolkit.getDefaultToolkit().getScreenSize().height/2);
```

```
public HungrySnakeGUI(){
    setTitle("HungrySnake");
    prepareView();
}

private void prepareView(){
    setSize(PREFERRED_DIMENSION);
    setLocationRelativeTo(null);
```

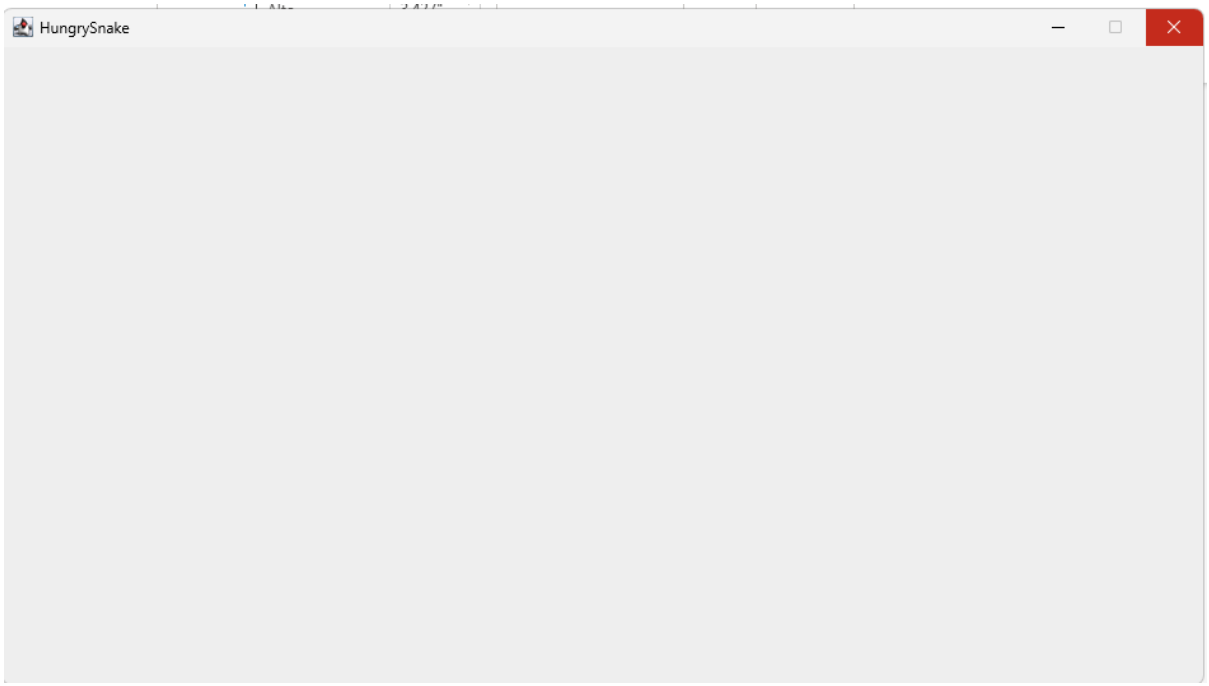
Para que el codigo anterior compile, se debe elminar el archivo del anterior punto y volver a compilar .java con el metodo prepareView() :

```
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake> del bin\presentation\HungrySnakeGUI.class
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake> javac -d bin src\presentation\HungrySnakeGUI.java
OPO\HungrySnake>
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake> java -cp bin presentation.HungrySnakeGUI
```

Luego:



3. Traten de cerrar la ventana. ¿Termina la ejecución? ¿Qué deben hacer en consola para terminar la ejecución?



Al cerrar la ventana en la consola sigue ejecutandose:

```
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake> javac -d bin src\presentation\HungrySnakeGUI.java
OPO\HungrySnake>
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake> java -cp bin presentation.HungrySnakeGUI
```

Por lo cual lo que se hizo en consola para terminar la ejecucion fue hacer la combinacion Control + C:

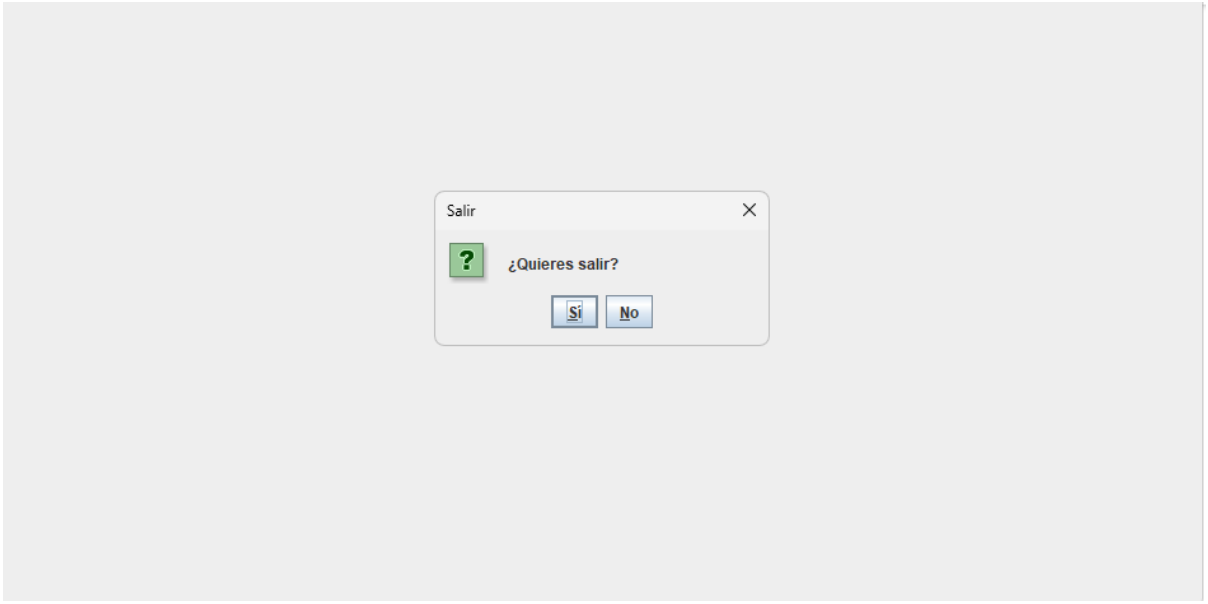
```
HungrySnakeGUI.javaments\DOPO\LAB05-DOPO\HungrySnake>
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake> java -cp bin presentation.HungrySnakeGUI
PS C:\Users\PC\Documents\DOPO\LAB05-DOPO\HungrySnake>
```

4. Estudien en JFrame el método setDefaultCloseOperation. ¿Para qué sirve? ¿Cómo lo usarían si queremos confirmar el cierre de la aplicación? ¿Cómo lo usarían si queremos simplemente cerrar la aplicación?
   -Sirve para definir la operación que pasa cuando el usuario le da al boton de X de la ventana. Normalmente viene con HIDE_ON_CLOSE que hace que se oculte visualmente, pero por dentro la JVM sigue funcionando.
   - Para confirmar el cierre se debe usar setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE) para que el JFrame se cierre cuando se le de a la X. Para luego usar un JOptionPane y preguntarle al usuario si quiere cerrar la ventana seleccionada.
   - Si se quiere solo cerrar, simplemente se usa setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)

5. Preparen el “oyente” correspondiente al icono cerrar que le pida al usuario que confirme su selección. Para eso inicien la codificación del método prepareController y el método asociado a la acción (exit). Ejecuten el programa y cierren el programa. Capturen las pantallas.

```
PS C:\Users\david.garzon-r\Documents\DOPO\LAB05-DOPO\HungrySnake> javac -d bin src\presentation\HungrySnakeGUI.java
PS C:\Users\david.garzon-r\Documents\DOPO\LAB05-DOPO\HungrySnake> java -cp bin presentation.HungrySnakeGUI
PS C:\Users\david.garzon-r\Documents\DOPO\LAB05-DOPO\HungrySnake>
```

-




```java
private void prepareController(){
    setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
    this.addWindowListener(new WindowAdapter(){
        @Override
        public void windowClosing(WindowEvent e){
            exitAction();
        }
    }

    );
}

private void exitAction(){
    int selection = JOptionPane.showConfirmDialog(this, "¿Quieres salir?", "Salir", JOptionPane.YES_NO_OPTION);
    if(selection ==  JOptionPane.YES_OPTION){
        System.exit(0);
    }
}
```

C Ciclo 1: Ventana con menú – Salir

El objetivo es implementar un menú clásico para la aplicación con un final adecuado desde la opción del menú para salir. El menú debe ofrecer mínimo las siguientes opciones **Nuevo, Abrir – Salvar y Salir.** Incluyan los separadores de opciones.

1. Expliquen los componentes visuales necesarios para este menú. ¿Cuáles serían atributos y cuáles podrían ser varibles del método prepareView? Justifique.

Los componentes necesarios son 3, JMenuBar, JMenu y JMenuItem. El JMenuBar es la barra superior de los menus. El JMenu es un boton que se ve en la barra, este sirve normalmente como un contenedor para los JMenuItem, además de mostrar el título de la categoria. Finalmente, los JMenuItem son componentes interactivos, que funcionan estilo botones.

Los atributos serían Nuevo, Abrir, Salvar y Salir, estos son atributos, porque a la hora de presionarlos quiero que hagan una acción especifica y necesito almacenarlos para esto. Por otro lado la barra del menu y el JMenu no es necesario saber si hacen algo con él, porque por defecto ya se espera un funcionamiento, como que el JMenu despliegue esas 4 opciones.

2. Construyan la forma del menú propuesto (prepareView - prepareViewMenu) . Ejecuten. Capturen la pantalla.

```
public class HungrySnakeGUI extends JFrame{
    private static final Dimension PREFERRED_DIMENSION = new Dimension (Toolkit.getDefaultToolkit().getScreenSize().width/2, Toolkit.getDefaultToolkit().getScreenSize().height/2);

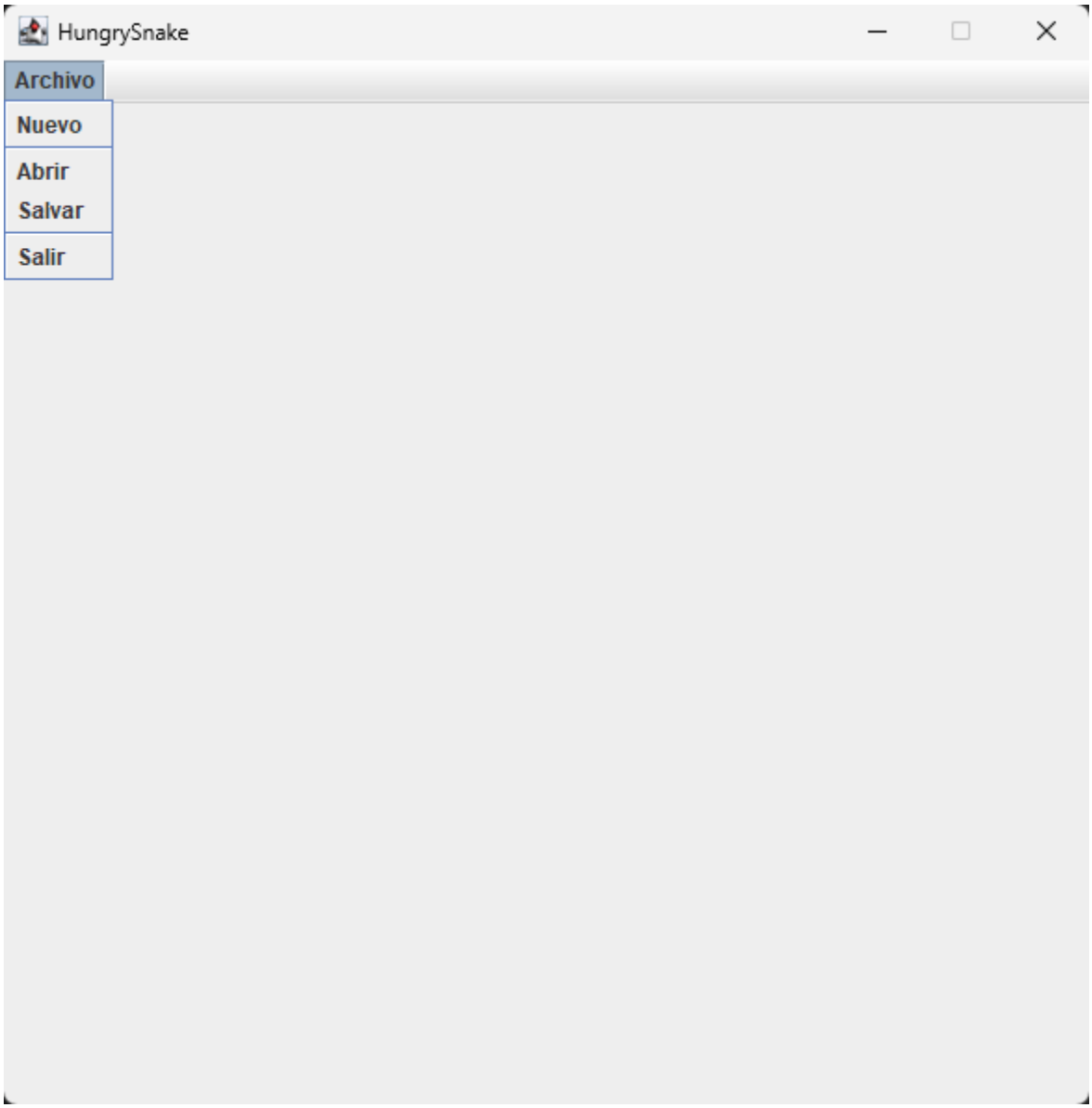
    private JMenuItem nuevo;
    private JMenuItem abrir;
    private JMenuItem salvar;
    private JMenuItem salir;

    public HungrySnakeGUI(){
        setTitle("HungrySnake");
        prepareView();
        prepareController();
    }

    private void prepareView(){
        setSize(PREFERRED_DIMENSION);
        setLocationRelativeTo(null);
        setResizable(false);
        setJMenuBar(prepareViewMenu());
    }
```

```
private JMenuBar prepareViewMenu(){
    nuevo = new JMenuItem("Nuevo");
    abrir = new JMenuItem("Abrir");
    salvar = new JMenuItem("Salvar");
    salir = new JMenuItem("Salir");
    JMenuBar barra = new JMenuBar();
    JMenu archivo = new JMenu("Archivo");
    archivo.add(nuevo);
    archivo.addSeparator();
    archivo.add(abrir);
    archivo.add(salvar);
    archivo.addSeparator();
    archivo.add(salir);

    barra.add(archivo);
    return barra;
}
```



3. Preparen el “oyente” correspondiente al icono cerrar con confirmación (prepareController - prepareControllerMenu). Ejecuten el programa y salgan del programa. Capturen las pantallas.

```java
private void prepareController(){
    setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
    this.addWindowListener(new WindowAdapter(){
        @Override
        public void windowClosing(WindowEvent e){
            exitAction();
        }
    }
    );

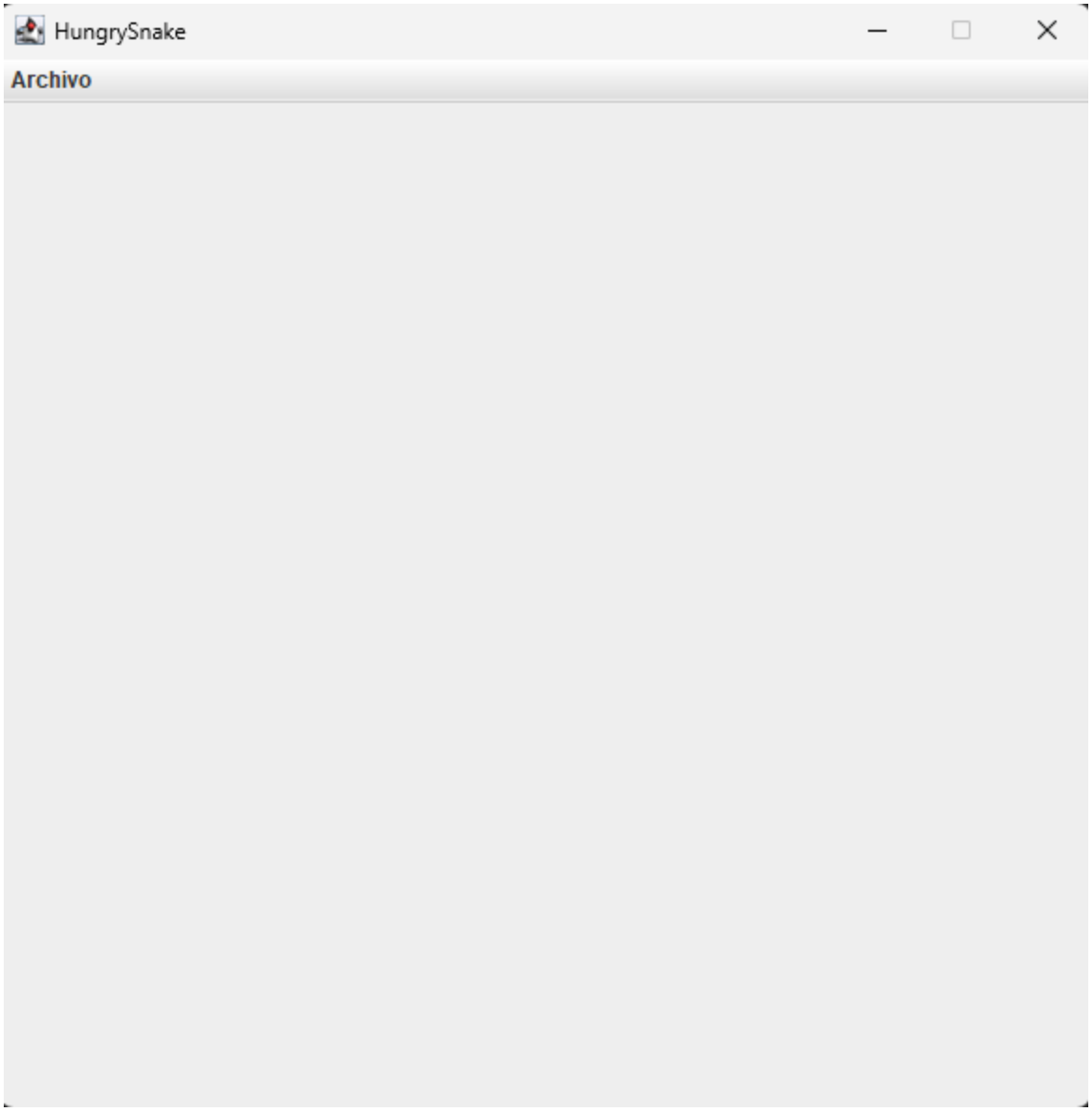
    prepareControllerMenu();

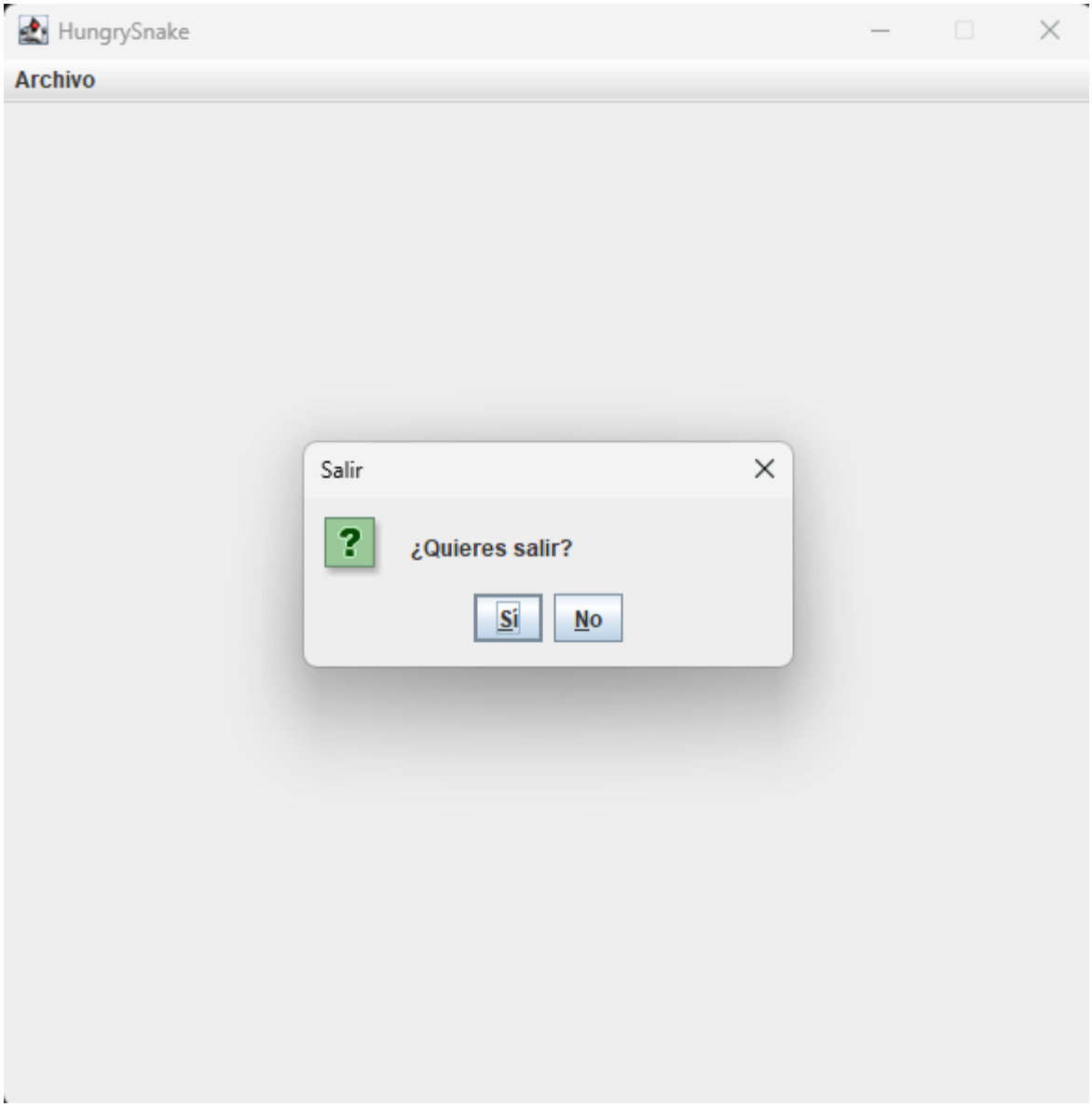
}
```

```java
private void prepareControllerMenu(){
    salir.addActionListener(new ActionListener(){

        public void actionPerformed(ActionEvent e){
            exitAction();
        }
    });

}
```


HungrySnake
Archivo



D Ciclo 2: Salvar y abrir

El objetivo es preparar la interfaz para las funciones de persistencia.

1. Detallen el componente JFileChooser especialmente los métodos : JFileChooser, showOpenDialog, showSaveDialog, getSelectedFile.

El componente JFileChooser es un componente que permite escoger archivos del computador, haciendo de función de explorador de archivos.
-JFileChooser se encarga de crearlo.

-showOpenDialog es una ventana que permite abrir un archivo ya existente.
-showSaveDialog es para guardar un estado
-getSelectedFile es para que después de que el usuario acepta, se obtiene el archivo elegido.

2. Implementen parcialmente los elementos necesarios para salvar y abrir. Al seleccionar los archivos indique que las funcionalidades están en construcción detallando la acción y el nombre del archivo seleccionado.

```
private void prepareControllerMenu(){
    salir.addActionListener(new ActionListener(){

        public void actionPerformed(ActionEvent e){
            exitAction();
        }
    });

    salvar.addActionListener(new ActionListener(){

        public void actionPerformed(ActionEvent e){
            saveAction();
        }
    });

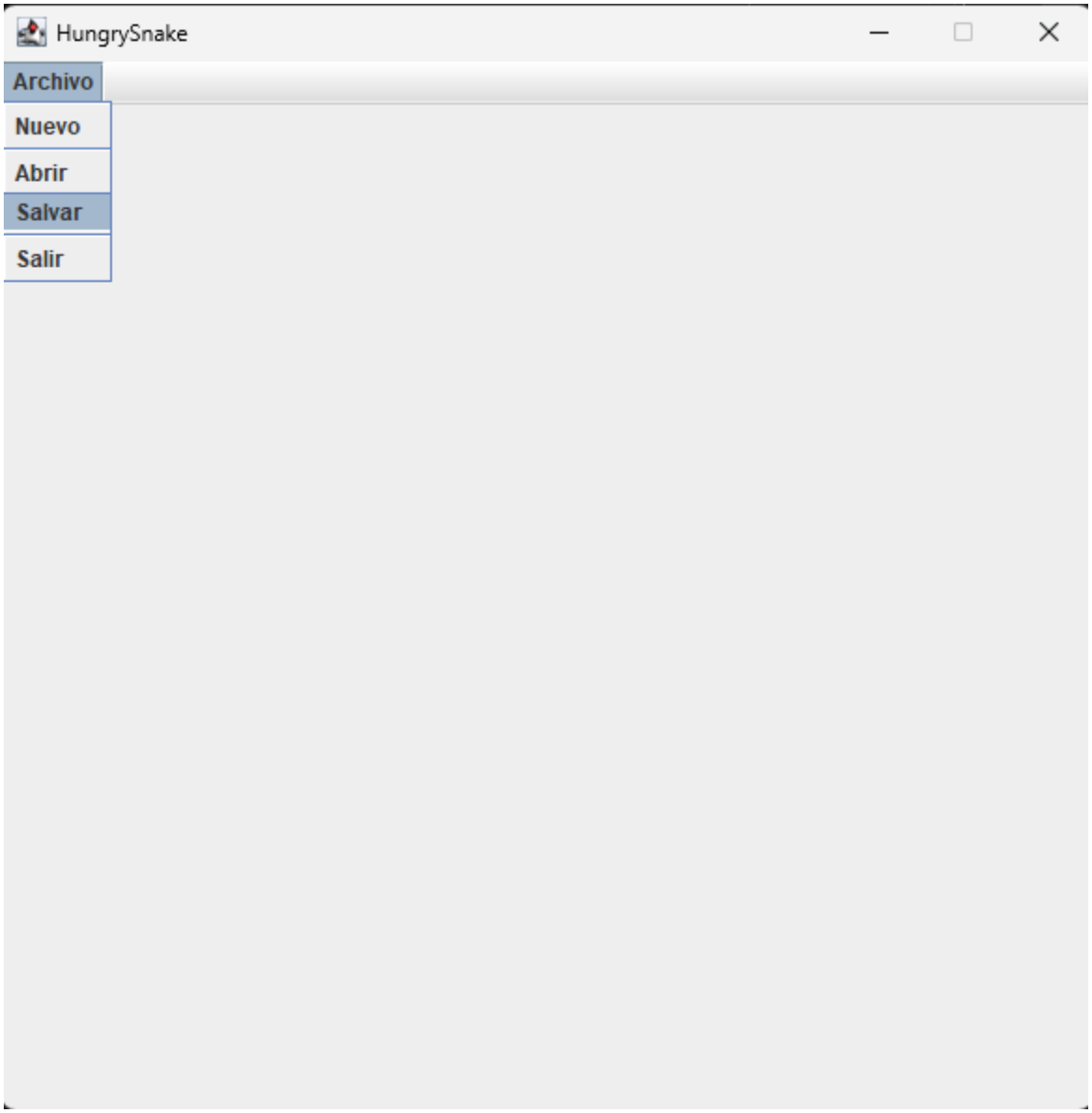
    abrir.addActionListener(new ActionListener(){

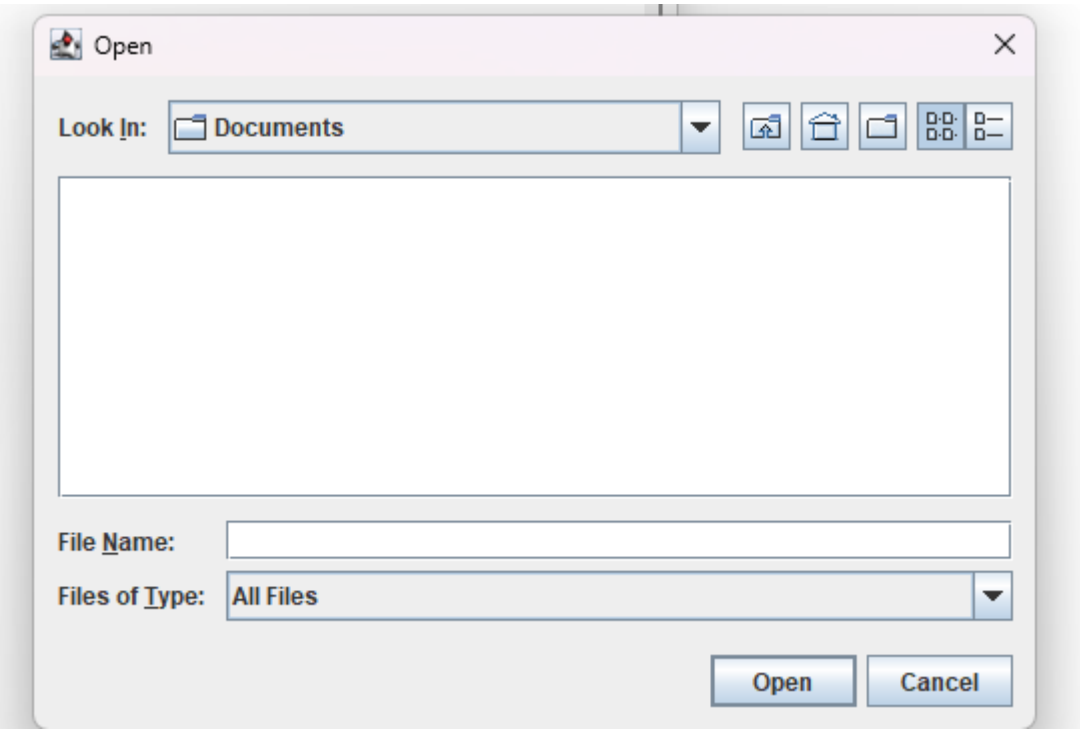
        public void actionPerformed(ActionEvent e){
            openAction();
        }
    });
}
```

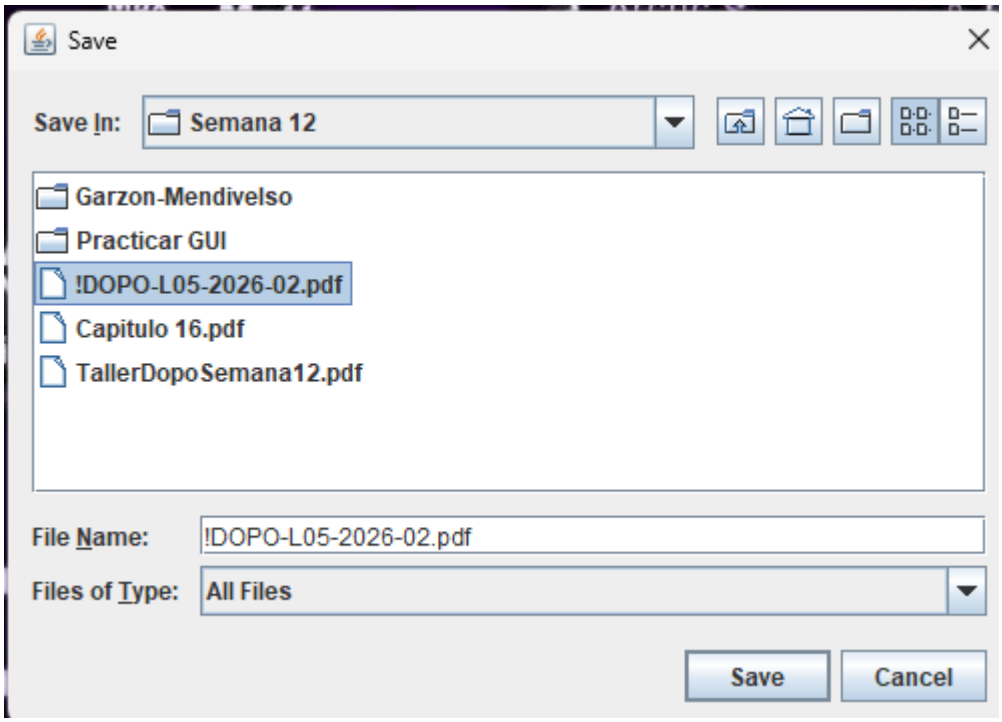
```
private void openAction(){
    JFileChooser chooser = new JFileChooser();
    chooser.showOpenDialog(this);
    File archivo = chooser.getSelectedFile();
    if(archivo == null){
        return;
    }
    JOptionPane.showMessageDialog(this, "Acción en construcción:\n" +"Acción abrir\n"+"Nombre del archivo: "+ archivo.getName(), "Abrir", JOptionPane.INFORMATION_MESSAGE
}

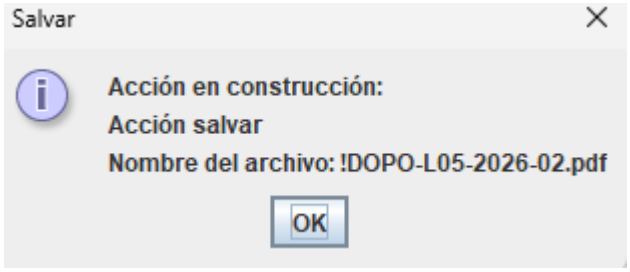
private void saveAction(){
    JFileChooser chooser = new JFileChooser();
    chooser.showSaveDialog(this);
    File archivo = chooser.getSelectedFile();
    if(archivo == null){
        return;
    }
    JOptionPane.showMessageDialog(this, "Acción en construcción:\n" +"Acción salvar\n"+"Nombre del archivo: "+ archivo.getName(), "Salvar", JOptionPane.INFORMATION_MESSA
}
```

3. Ejecuten las dos opciones y capturen las pantallas más significativas.
   Para salvar:

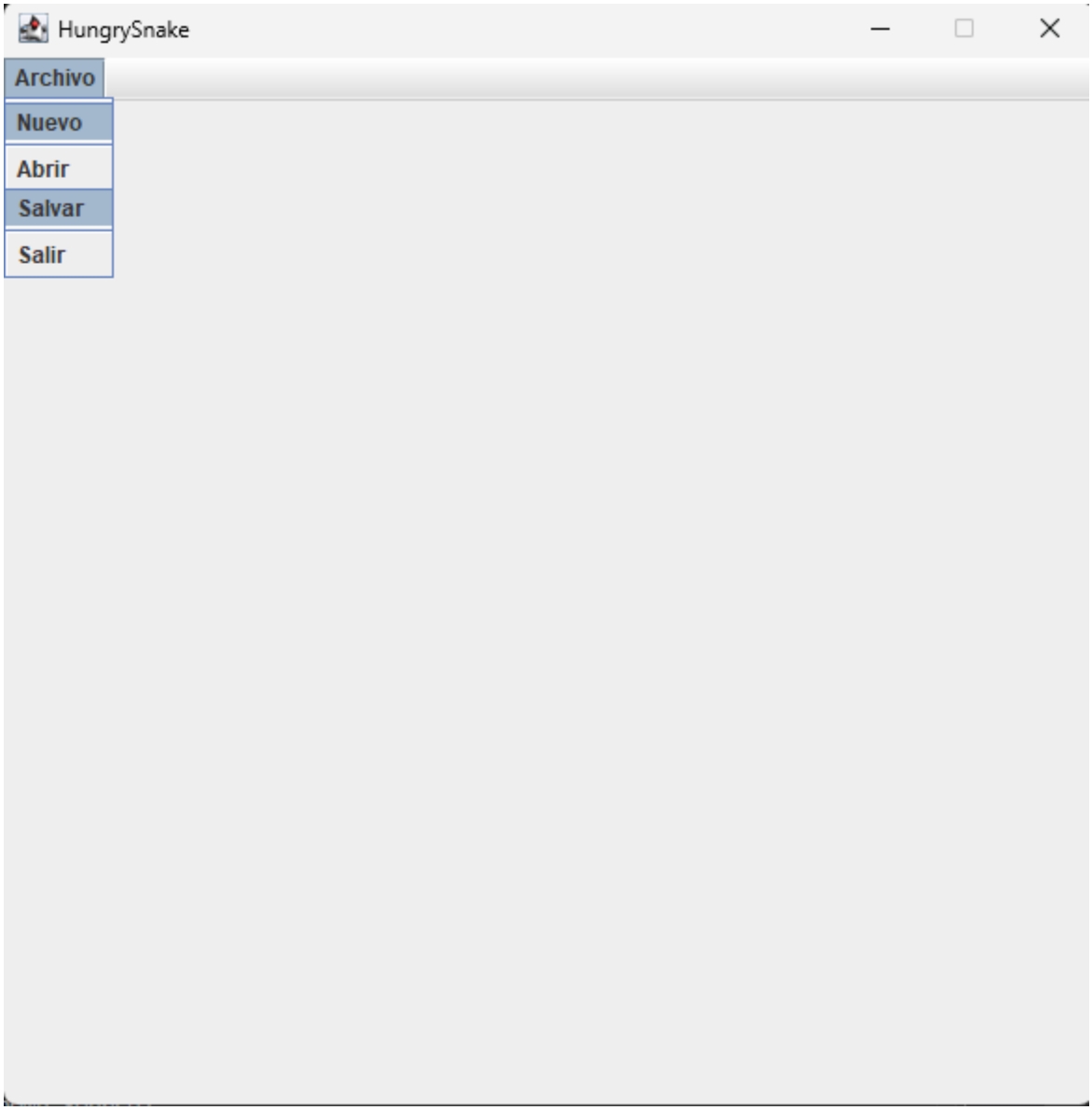

HungrySnake
Archivo
Nuevo
Abrir
Salvar
Salir

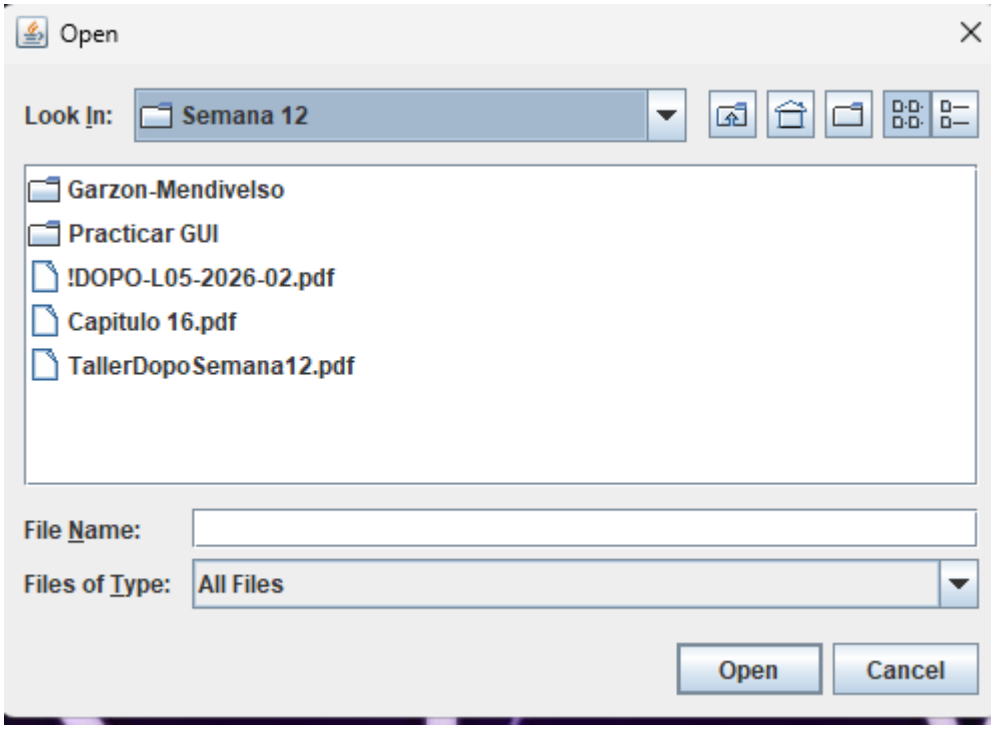

Open
Look In:
Documents
File Name:
Files of Type:
All Files
Open
Cancel


Save
Save In:
Semana 12
Garzon-Mendivelso
Practicar GUI
!DOPO-L05-2026-02.pdf
Capitulo 16.pdf
TallerDopoSemana12.pdf
File Name:
!DOPO-L05-2026-02.pdf
Files of Type:
All Files
Save
Cancel


Salvar
Acción en construcción:
Acción salvar
Nombre del archivo: !DOPO-L05-2026-02.pdf
OK

Para abrir:


HungrySnake
Archivo
Nuevo
Abrir
Salvar
Salir



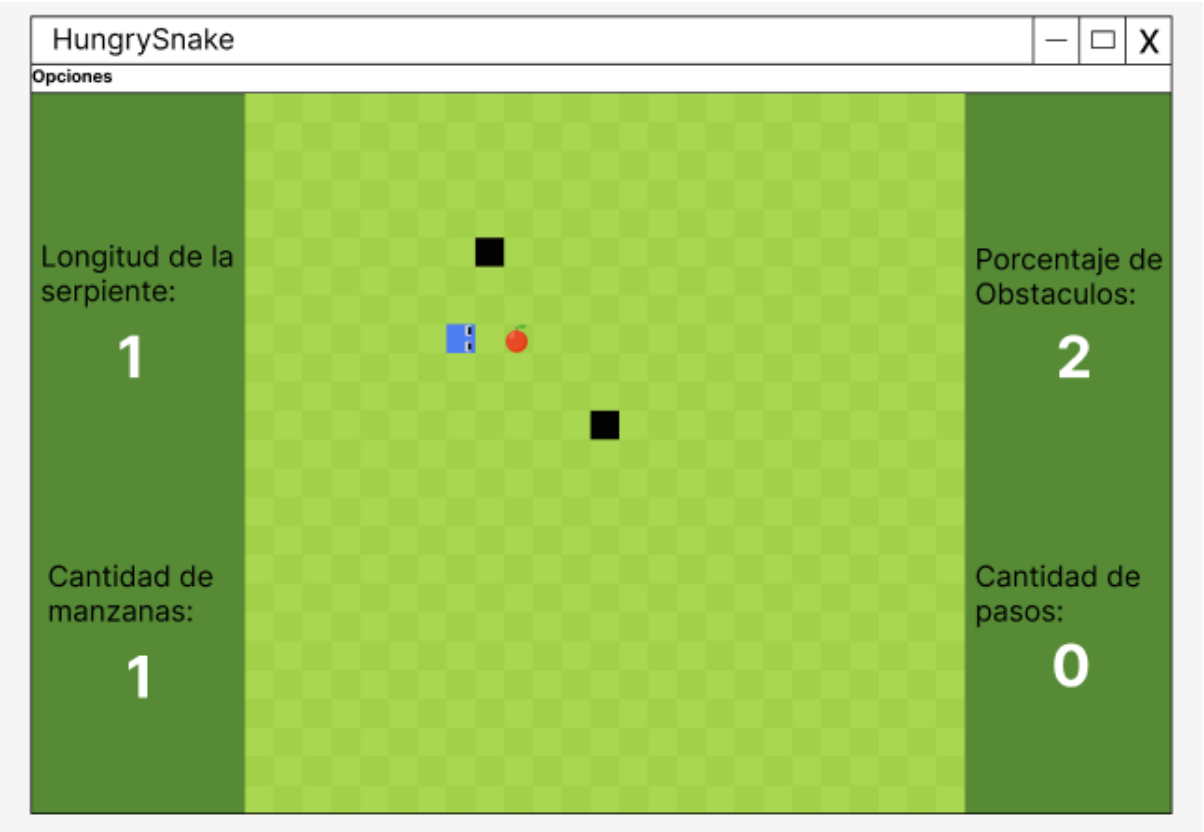


E Ciclo 3: Forma de la ventana principal:

El objetivo es codificar el diseño de la ventana principal (todos los elementos de primer nivel):

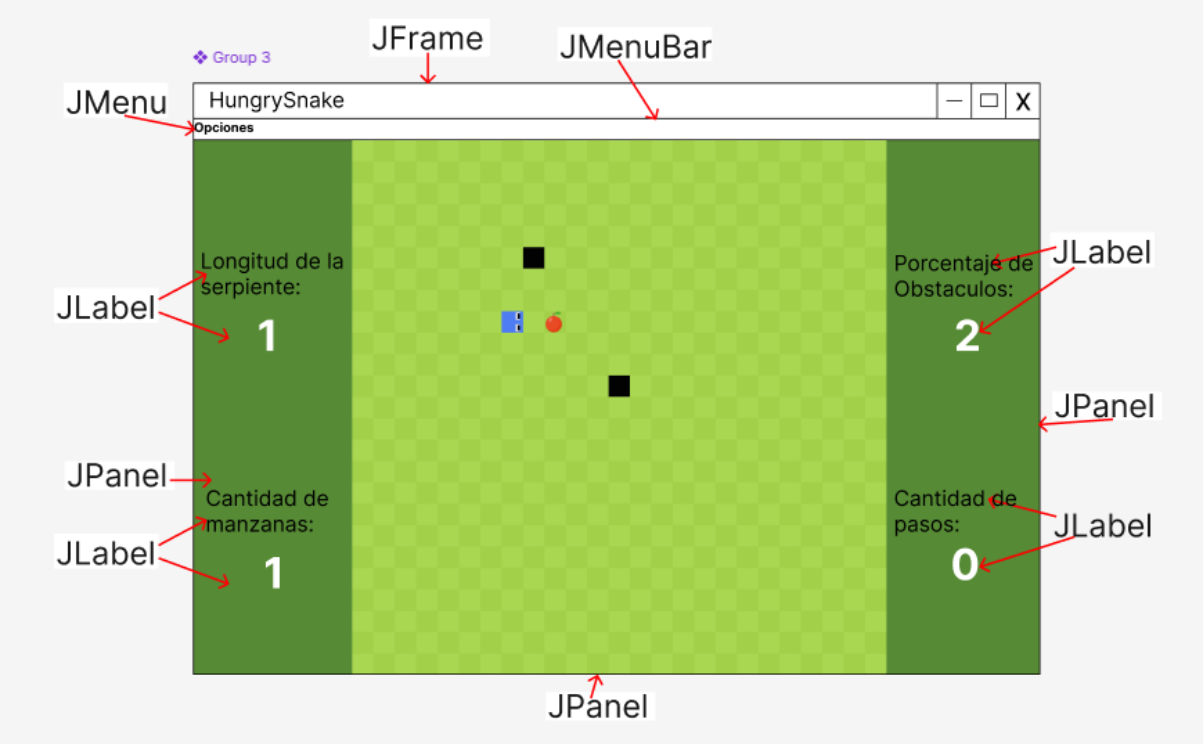
1. Presenten el bosquejo de interfaz con todos los componentes necesarios. (Incluyan la imagen).

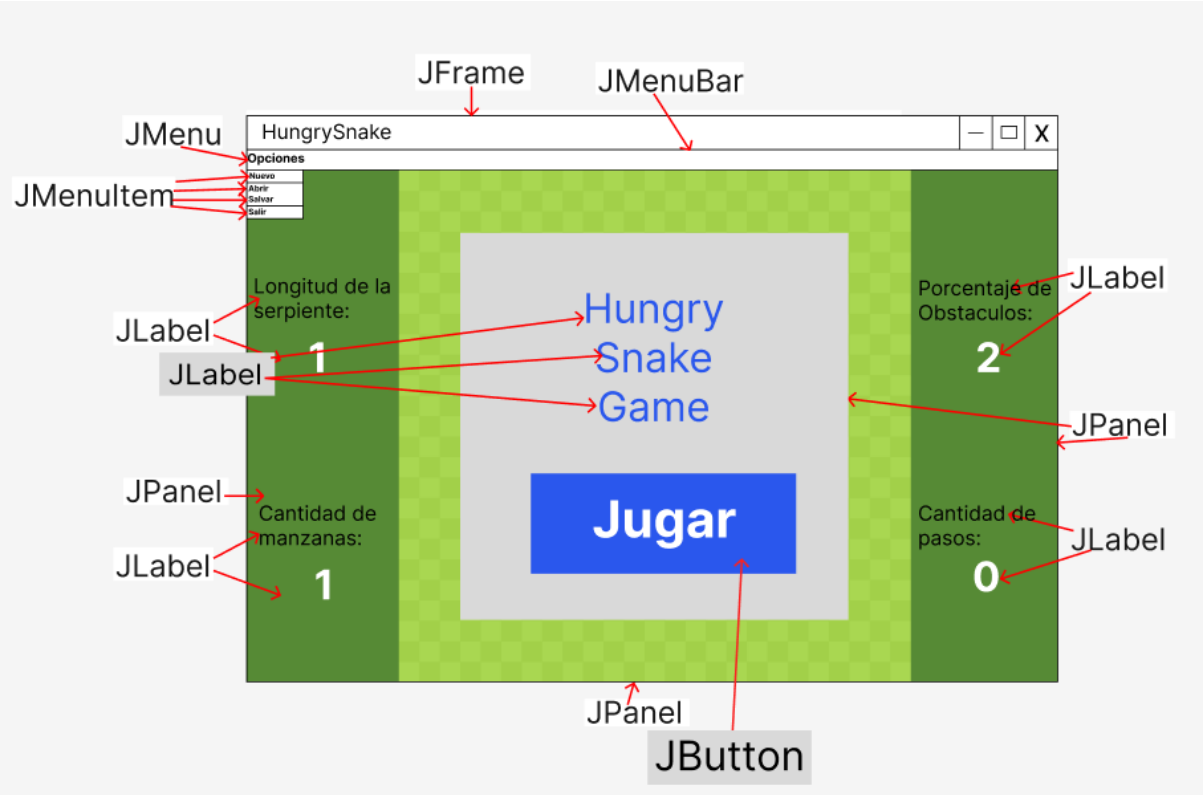
La interfaz principal sería de la siguiente manera:





Especificando los componentes:





Siendo que todas las casillas del tablero se entienden que serán JPanel, pero no se pone por evitar la saturación de cosas en pantalla.

2. Continúen con la implementación definiendo los atributos necesarios y extendiendo el método **prepareView**. Para la zona del tablero definan un método

prepareViewBoard y un método **refresh** que actualiza la vista del tablero considerando, por ahora, el tablero inicial por omisión. Este método lo vamos a implementar realmente en otros ciclos.

Lo que se hizo para este apartado, fue primero extender prepareView, haciendo que prepareViewBoard le mande el JPanel que sea toda la vista:

```
private void prepareView(){
    setSize(PREFERRED_DIMENSION);
    setLocationRelativeTo(null);
    setResizable(false);
    setJMenuBar(prepareViewMenu());
    add(prepareViewBoard());
}
```

Por otro lado, se creó prepareViewBoard, pero para evitar las grandes cantidades de código se dividió en varios pedazos, comprendiendo que, en el bosquejo se ve hay tres zonas principales, la zona de la izquierda, la zona central y la de la derecha, donde las zonas de la izquierda y derecha se encargan es de simplemente llevar información de la partida.

```
private JPanel prepareViewBoard(){
    JPanel panel = new JPanel();
    panel.setLayout(new BorderLayout());
    panel.add(prepareInformationLeftArea(), BorderLayout.WEST);
    panel.add(preparePlayArea(), BorderLayout.CENTER);
    panel.add(prepareInformationRightArea(), BorderLayout.EAST);
    return panel;
}
```

Y se define un BorderLayout para definir de una manera mucho más fácil las zonas de división del tablero

```java
private JPanel prepareInformationLeftArea(){
    JPanel panel = new JPanel();
    panel.setLayout(new GridLayout(7,1));
    panel.setPreferredSize(new Dimension((PREFERRED_DIMENSION.width -  PREFERRED_DIMENSION.height)/2,0));
    panel.setBackground(new Color(86,138,53));

    length = new JLabel("1", JLabel.CENTER);
    length.setFont(new Font("Arial", Font.BOLD, 32));
    length.setForeground(Color.WHITE);

    JLabel snakeLength = new JLabel("Tamaño serpiente:", JLabel.CENTER);
    snakeLength.setFont(new Font("Arial", Font.BOLD, PREFERRED_DIMENSION.width/45));

    JLabel fruitsInformation = new JLabel("Cantidad de frutas:", JLabel.CENTER);
    fruitsInformation.setFont(new Font("Arial", Font.BOLD, PREFERRED_DIMENSION.width/45));

    fruits = new JLabel(String.valueOf(fruitsNumber), JLabel.CENTER);
    fruits.setFont(new Font("Arial", Font.BOLD, 32));
    fruits.setForeground(Color.WHITE);

    for(int i = 0; i < 7; i++){
        if(i==1){
            panel.add(snakeLength);
        }
        else if (i == 2){
            panel.add(length);
        }
        else if(i==4){
            panel.add(fruitsInformation);
        }
        else if (i == 5){
            panel.add(fruits);
        }
        else{
            panel.add(new JLabel());
        }
    }

    return panel;
}
```

```java
private JPanel prepareInformationRightArea(){
    JPanel panel = new JPanel();
    panel.setLayout(new GridLayout(7,1));
    panel.setPreferredSize(new Dimension((PREFERRED_DIMENSION.width -  PREFERRED_DIMENSION.height)/2,0));

    panel.setBackground(new Color(86,138,53));

    obstacles = new JLabel(String.valueOf(obstaclesNumber)+"%", JLabel.CENTER);
    obstacles.setFont(new Font("Arial", Font.BOLD, 32));
    obstacles.setForeground(Color.WHITE);
    steps = new JLabel(String.valueOf(stepsNumber), JLabel.CENTER);
    steps.setFont(new Font("Arial", Font.BOLD, 32));
    steps.setForeground(Color.WHITE);
    JLabel wallsInformation = new JLabel("Porcentaje de Obstaculos:", JLabel.CENTER);
    wallsInformation.setFont(new Font("Arial", Font.BOLD, PREFERRED_DIMENSION.width/50));
    JLabel stepsInformation = new JLabel("Cantidad de pasos:", JLabel.CENTER);
    stepsInformation.setFont(new Font("Arial", Font.BOLD, PREFERRED_DIMENSION.width/50));


    for(int i = 0; i < 7; i++){
        if(i==1){
            panel.add(wallsInformation);
        }
        else if (i == 2){
            panel.add(obstacles);
        }
        else if(i==4){
            panel.add(stepsInformation);
        }
        else if (i == 5){
            panel.add(steps);
        }
        else{
            panel.add(new JLabel());
        }
    }
    return panel;
}
```

Siendo que en ambos se arma un GridLayout para mantener separados los objetos unos de otros y permitir mayor orden.

Con respecto a la zona central:

```java
private JPanel preparePlayArea(){
    game = new JPanel();
    game.setBackground(new Color(161, 130,98));
    game.setPreferredSize(new Dimension(PREFERRED_DIMENSION.width -  PREFERRED_DIMENSION.height,0));
    game.setLayout(new BorderLayout());

    JLabel panel1 = new JLabel();
    panel1.setPreferredSize(new Dimension(0,(PREFERRED_DIMENSION.width -  PREFERRED_DIMENSION.height)/8));
    game.add(panel1, BorderLayout.NORTH);

    JLabel panel2 = new JLabel();
    panel2.setPreferredSize(new Dimension(0,(PREFERRED_DIMENSION.width -  PREFERRED_DIMENSION.height)/8));
    game.add(panel2, BorderLayout.SOUTH);

    JLabel panel3 = new JLabel();
    panel3.setPreferredSize(new Dimension((PREFERRED_DIMENSION.width -  PREFERRED_DIMENSION.height)/8,0));
    game.add(panel3, BorderLayout.WEST);

    JLabel panel4 = new JLabel();
    panel4.setPreferredSize(new Dimension((PREFERRED_DIMENSION.width -  PREFERRED_DIMENSION.height)/8,0));
    game.add(panel4, BorderLayout.EAST);

    game.add(prepareStartArea(), BorderLayout.CENTER);
    return game;
}
```

Se busca definir la sensación de una pantalla de inicio donde se pondrá el prepareStartArea, para que de este modo se le de una sensación al usuario de interacción e inicio de juego.

```
private JPanel prepareStartArea(){
    JPanel gameStart = new JPanel();
    gameStart.setLayout(new GridLayout(7,3));
    gameStart.setBackground(new Color(217,217,217));



    JLabel hungryWord = new JLabel("Hungry", JLabel.CENTER);
    hungryWord.setFont(new Font("Arial", Font.BOLD, PREFERRED_DIMENSION.width/30));
    hungryWord.setForeground(snakeColor);

    JLabel snakeWord = new JLabel("Snake", JLabel.CENTER);
    snakeWord.setFont(new Font("Arial", Font.BOLD, PREFERRED_DIMENSION.width/30));
    snakeWord.setForeground(snakeColor);

    JLabel gameWord = new JLabel("Game", JLabel.CENTER);
    gameWord.setFont(new Font("Arial", Font.BOLD, PREFERRED_DIMENSION.width/30));
    gameWord.setForeground(snakeColor);

    jugar = new JButton("Jugar");
    jugar.setFont(new Font("Arial", Font.BOLD, PREFERRED_DIMENSION.width/40));

    for(int i = 0; i < 7; i++){
        for(int j = 0; j < 3; j++){
            if (i == 1 && j == 1){
              gameStart.add(hungryWord);
            }
            else if (i ==2 && j == 1){
              gameStart.add(snakeWord);
            }
            else if(i == 3 && j == 1){
                gameStart.add(gameWord);
            }
            else if (i == 5 && j == 1){
              gameStart.add(jugar);
            }
            else{
              gameStart.add(new JLabel());
            }
        }
    }

    return gameStart;
}
```

Siendo que en esta zona, simplemente está definido el titulo del juego y el botón de jugar para iniciar el juego.

Por otro lado, es necesario cambiar un poco el prepareControllerMenu para hacer que nuevo tenga la posibilidad de iniciar el juego:

```java
    private void prepareControllerMenu(){
        salir.addActionListener(new ActionListener(){

            public void actionPerformed(ActionEvent e){
                exitAction();
            }
        });

        salvar.addActionListener(new ActionListener(){

            public void actionPerformed(ActionEvent e){
                saveAction();
            }
        });

        abrir.addActionListener(new ActionListener(){

            public void actionPerformed(ActionEvent e){
                openAction();
            }
        });
        nuevo.addActionListener( new ActionListener(){
            public void actionPerformed(ActionEvent e){
                startGameAction();
            }
        });
    }
```

Iniciar el juego consiste en, pedir la cantidad de frutas y de muros:

```java
private void startGameAction(){
    String newFruitsNumber = JOptionPane.showInputDialog(this, "Dame la cantidad de frutas que va a tener: ");
    if(newFruitsNumber != null){
        try{
            fruitsNumber = Integer.parseInt(newFruitsNumber);
        }
        catch(NumberFormatException e){
            JOptionPane.showMessageDialog(this, "El texto no es un número válido", "No es un número válido", JOptionPane.ERROR_MESSAGE);
        }
    }
    else{
        fruitsNumber = 1;
    }
    String newWallsNumber = JOptionPane.showInputDialog(this, "Dame el porcentaje de muros que va a tener: ");
    if(newWallsNumber != null){
        try{
            obstaclesNumber = Double.parseDouble(newWallsNumber);
        }
        catch(NumberFormatException e){
            JOptionPane.showMessageDialog(this, "El texto no es un número válido", "No es un número válido", JOptionPane.ERROR_MESSAGE);
        }
    }
    else{
        obstaclesNumber = 2.0;
    }

    game.removeAll();
    prepareGame();
    refresh();
    game.revalidate();
    game.repaint();
}
```

Para luego poder crear el tablero a través de prepareGame, además de que se define refresh, se pone un inicio cualquiera para probar el dibujado de los componentes y demás:

```java
private void prepareGame(){
    game.setLayout(new GridLayout(rows, columns));
    board = new JPanel[rows][columns];

    for(int i = 0; i < rows; i++){
        for(int j = 0; j < columns; j++){
            board[i][j] = new JPanel();
            if((i+j)%2 == 0){
                board[i][j].setBackground(firstFloorColor);
            }
            else{
                board[i][j].setBackground(secondFloorColor);
            }
            board[i][j].setLayout(new BorderLayout());
            game.add(board[i][j]);
        }
    }
}

private void refresh(){
    int[] snakePosition = new int[]{3,3};
    int[] obstacle1Position = new int[]{12,2};
    int[] obstacle2Position = new int[]{5,5};
    int[] fruitPosition = new int[]{10,8};

    paintPanelComponents(c: 'H',snakePosition[0], snakePosition[1] );
    paintPanelComponents(c: 'O',obstacle1Position[0], obstacle1Position[1] );
    paintPanelComponents(c: 'O',obstacle2Position[0], obstacle2Position[1] );
    paintPanelComponents(c: 'F',fruitPosition[0], fruitPosition[1] );

    obstacles.setText(String.valueOf(obstaclesNumber)+"%");
    steps.setText(String.valueOf(stepsNumber));
    fruits.setText(String.valueOf(fruitsNumber));
}
```

Por otro lado, fue necesario crear ese nuevo metodo de paintPanelComponents, porque no se fue dicho hasta ahora, pero fue necesario añadir y distribuir de mejor manera los atributos, puesto que en determinado punto se dijo que la zona central funcionaría con un GridLayout que tenga todas las casillas del tablero y demás, pero es necesario pintar los componentes poco a poco, eso sí, la mejor forma que hayamos para lograrlo, es crear un JPanel temporal y este lo añadimos a ese JPanel, así, podemos manejar el cambio de colores, sin tener que crear un nuevo JPanel directamente en el GridLayout, puesto que, si tenemos un atributo que guarda la referencia en memoria de eso JPanel, solo añadiendo un componente podemos estarle cambiando el color.

Quedando así:

```java
public class HungrySnakeGUI extends JFrame{
    //JFrame size
    private static final Dimension PREFERRED_DIMENSION = new Dimension (Toolkit.getDefaultToolkit().getScreenSize().width/2, Toolkit.getDefaultToolkit().getScreenSize().height/2);
    //For JPanelArea there's less 16px for width and there's less 64px for the height

    //Menu items
    private JMenuItem nuevo;
    private JMenuItem abrir;
    private JMenuItem salvar;
    private JMenuItem salir;


    //Information items
    private JLabel length;
    private JLabel obstacles;
    private JLabel steps;
    private JLabel fruits;

    //Game
    private JPanel game;
    private JButton jugar;
    private JPanel[][] board;

    //Game attributes
    private int rows = 25;
    private int columns = 25;
    private int stepsNumber = 0;
    private int fruitsNumber = 1;
    private double obstaclesNumber = 2;
    private char lastDirection = 'N';

    //Game colors
    private Color snakeColor = Color.BLUE;
    private Color obstaclesColor = Color.BLACK;
    private Color fruitsColor = Color.RED;
    private Color firstFloorColor = new Color(162,208,73);
    private Color secondFloorColor = new Color(169,215,81);
```

Por otro lado el método que de paintPanelComponents quedó como:

```java
private void paintPanelComponents(char c, int row, int column){
    JPanel panel = new JPanel();
    //Wall
    if(c == 'O'){
        panel.setBackground(wallsColor);
    }
    //Fruit
    else if(c == 'F'){
        panel = new JPanel(){
            @Override
            protected void paintComponent (Graphics g){
                super.paintComponent(g);
                g.setColor(fruitsColor);
                g.fillOval(getWidth()/10,getHeight()/10,4*getWidth()/5,4*getHeight()/5);
                g.setColor(new Color(161, 130,98));
                g.fillRect(9*getWidth()/20,-2*getHeight()/20,4*getWidth()/25,4*getHeight()/10);
            }
        };
        if((row+column)%2 == 0){
            panel.setBackground(new Color(162,208,73));
         }
        else{
            panel.setBackground(new Color(169,215,81));
            }
    }
    //SnakeBody
    else if(c == 'S'){
        panel.setBackground(snakeColor);
    }
```

```
//SnakeHead
else if(c == 'H'){
    panel = new JPanel(){

        protected void paintComponent (Graphics g){
            super.paintComponent(g);
            g.setColor(snakeColor);
            g.fillRect(0,0,getWidth(),getHeight());
            if(lastDirection == 'E'){
                g.setColor(Color.WHITE);
                g.fillRect(12*getWidth()/20,4*getHeight()/20,4*getWidth()/25,3*getHeight()/10);
                g.fillRect(12*getWidth()/20,12*getHeight()/20,4*getWidth()/25,3*getHeight()/10);
                g.setColor(Color.BLACK);
                g.fillRect(13*getWidth()/20,5*getHeight()/20,3*getWidth()/25,2*getHeight()/10);
                g.fillRect(13*getWidth()/20,13*getHeight()/20,3*getWidth()/25,2*getHeight()/10);

            }
            else if(lastDirection == 'W'){
                g.setColor(Color.WHITE);
                g.fillRect(5*getWidth()/20,4*getHeight()/20,4*getWidth()/25,3*getHeight()/10);
                g.fillRect(5*getWidth()/20,12*getHeight()/20,4*getWidth()/25,3*getHeight()/10);
                g.setColor(Color.BLACK);
                g.fillRect(5*getWidth()/20,5*getHeight()/20,3*getWidth()/25,2*getHeight()/10);
                g.fillRect(5*getWidth()/20,13*getHeight()/20,3*getWidth()/25,2*getHeight()/10);
            }
            else if(lastDirection == 'S'){
                g.setColor(Color.WHITE);
                g.fillRect(4*getHeight()/20,12*getWidth()/20,3*getHeight()/10,4*getWidth()/25);
                g.fillRect(12*getHeight()/20,12*getWidth()/20,3*getHeight()/10,4*getWidth()/25);
                g.setColor(Color.BLACK);
                g.fillRect(5*getHeight()/20,13*getWidth()/20,2*getHeight()/10,3*getWidth()/25);
                g.fillRect(13*getHeight()/20,13*getWidth()/20,2*getHeight()/10,3*getWidth()/25);
            }
            else if(lastDirection == 'N'){
                g.setColor(Color.WHITE);
                g.fillRect(4*getHeight()/20,4*getWidth()/20,3*getHeight()/10,4*getWidth()/25);
                g.fillRect(12*getHeight()/20,4*getWidth()/20,3*getHeight()/10,4*getWidth()/25);
                g.setColor(Color.BLACK);
                g.fillRect(5*getHeight()/20,4*getWidth()/20,2*getHeight()/10,3*getWidth()/25);
                g.fillRect(13*getHeight()/20,4*getWidth()/20,2*getHeight()/10,3*getWidth()/25);
            }
        }
    };
```

```
            }
        };
    }
    else{
        if((row + column)%2 == 0){
            panel.setBackground(firstFloorColor);
        }
        else{
            panel.setBackground(secondFloorColor);
        }

    }
    board[row][column].add(panel);
}
```

Eso sí, la acción más importante a definir fue startGameAction:

```java
private void startGameAction(){
    String newFruitsNumber = JOptionPane.showInputDialog(this, "Dame la cantidad de frutas que va a tener: ");
    if(newFruitsNumber != null){
        try{
            fruitsNumber = Integer.parseInt(newFruitsNumber);
        }
        catch(NumberFormatException e){
            JOptionPane.showMessageDialog(this, "El texto no es un número válido", "No es un número válido", JOptionPane.ERROR_MESSAGE);
        }
    }
    else{
        fruitsNumber = 1;
    }
    String newWallsNumber = JOptionPane.showInputDialog(this, "Dame el porcentaje de muros que va a tener: ");
    if(newWallsNumber != null){
        try{
            obstaclesNumber = Double.parseDouble(newWallsNumber);
        }
        catch(NumberFormatException e){
            JOptionPane.showMessageDialog(this, "El texto no es un número válido", "No es un número válido", JOptionPane.ERROR_MESSAGE);
        }
    }
    else{
        obstaclesNumber = 2.0;
    }

    game.removeAll();
    prepareGame();
    refresh();
    game.revalidate();
    game.repaint();
}
```

3. Ejecuten y capturen la pantalla.



En esta pantalla se ve el inicio que digo yo, de jugar en la parte central y en los laterales información del juego, al darle click a Jugar:



Pregunta la cantidad de frutas a tener, entonces digito 50:

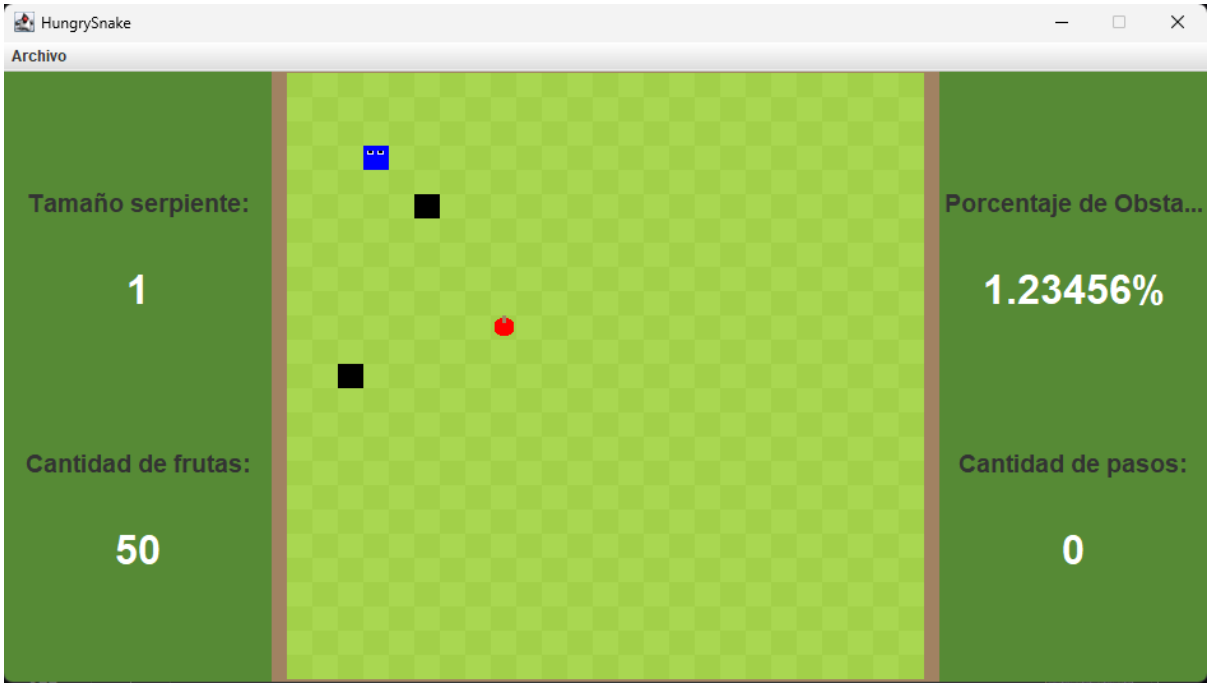


Y le doy OK, luego:



Digito un número cualquiera:





Sin embargo, se ve animada la versión estandar de tener 2 muros y 1 fruta, puesto que de momento no hay un dominio activo, además se comprueba que los bordes reaccionan a los inputs del usuario.

F Ciclo 4: Cambiar colores

1. Expliquen los elementos (vista – controlador) necesarios para implementar este caso de uso.

   Para este ciclo, usaremos una nueva opción que aparezca el menú, en la cual al selecionarla, se abrirá un mini cuadro donde pregunten a que elemento quieren cambiar el color (fruta, serpiente y obstáculo). Para ello:
   -se agrega el nuevo atributo JMenuItem llamado colores.

```
//Menu Items
private JMenuItem nuevo;
private JMenuItem abrir;
private JMenuItem salvar;
private JMenuItem salir;
private JMenuItem colores;

//Information items
```

   -Luego en el método prepareViewMenu se crea un JMenu llamado color, en este se agrega el atributo colores y se agrega en el JMenuBar (barra)

```
private JMenuBar prepareViewMenu(){
    nuevo = new JMenuItem("Nuevo");
    abrir = new JMenuItem("Abrir");
    salvar = new JMenuItem("Salvar");
    salir = new JMenuItem("Salir");
    JMenuBar barra = new JMenuBar();
    JMenu archivo = new JMenu("Archivo");
    JMenu color = new JMenu("Colores");
    archivo.add(nuevo);
    archivo.addSeparator();
    archivo.add(abrir);
    archivo.add(salvar);
    archivo.addSeparator();
    archivo.add(salir);

    color.add(colores);

    barra.add(archivo);
    barra.add(color);
    return barra;
}
```

Ahora, en el controlador, al momento de adicionar el nuevo JMenuItem, se le añade a ese nuevo atributo un oyente(ActionListener) y por esto creando un nuevo metodo llamado changeColorAction():

```
private void prepareControllerMenu(){
    salir.addActionListener(new ActionListener(){

        public void actionPerformed(ActionEvent e){
            exitAction();
        }
    });

    salvar.addActionListener(new ActionListener(){

        public void actionPerformed(ActionEvent e){
            saveAction();
        }
    });

    abrir.addActionListener(new ActionListener(){

        public void actionPerformed(ActionEvent e){
            openAction();
        }
    });
    nuevo.addActionListener( new ActionListener(){
        public void actionPerformed(ActionEvent e){
            startGameAction();
        }
    });
    colores.addActionListener( new ActionListener(){
        public void actionPerformed(ActionEvent e){
            changeColorAction();
        }
    })
}
```

2. Detalle el comportamiento de JColorChooser especialmente el método estático showDialog.

- el componente JColorChooser es un componente que permite que el usuario escoja un color que se proporcionan en un panel, el método estático showDialog() despliega una ventana emergente para que el usuario escoja el color, este método se encarga de crear posicionar y destruir esta ventana cuando se haga determinada acción

3. Implementen los componentes necesarios para cambiar los colores requeridos.

   Para ello se implementa el metodo changeColorAction():
   En el cual usamos el metodo estatico showDialog del componente JColorChooser,

También usamos un JOptionPane para que el usuario pueda escojer a que le cambiara el color, y usamos otro JOptionPane para decir si el color que quieren poner ya esta siendo usado por otra cosa

```
private void changeColorAction(){
    String[] opciones = {"Frutas","Serpiente","Obstaculos","Piso 1", "Piso 2"};
            int opcion = JOptionPane.showOptionDialog( this, "¿A que le quieres cambiar el color?", "Cambiar color", JOptionPane.DEFAULT_OPTION, JOptionPane.QUESTION_MESSAGE, null, opciones, "Caja");
            if(opcion != -1){
                Color color1 = JColorChooser.showDialog( this, "Selecciona un color", Color.ORANGE);
                allColors = new ArrayList<>();
                allColors.add(snakeColor);
                allColors.add(obstaclesColor);
                allColors.add(fruitsColor);
                allColors.add(firstFloorColor);
                allColors.add(secondFloorColor);
                boolean isIn = false;

                if(allColors.contains(color1)){
                    JOptionPane.showMessageDialog(this, "Este color ya está en uso", "Color", JOptionPane.ERROR_MESSAGE);
                    isIn = true;
                }


                if(!isIn && color1 != null){

                    if (opcion == 0){
                        fruitsColor = color1;
                        fruits.setForeground(fruitsColor);
                    }
                    else if(opcion == 1){
                        snakeColor = color1;
                        length.setForeground(snakeColor);
                    }
                    else if(opcion == 2) {
                        obstaclesColor = color1;
                        obstacles.setForeground(obstaclesColor);
                    }
                    else if(opcion == 3){
                        firstFloorColor = color1;

                    }else{
                        secondFloorColor = color1;

                    }
                    refresh();
                }

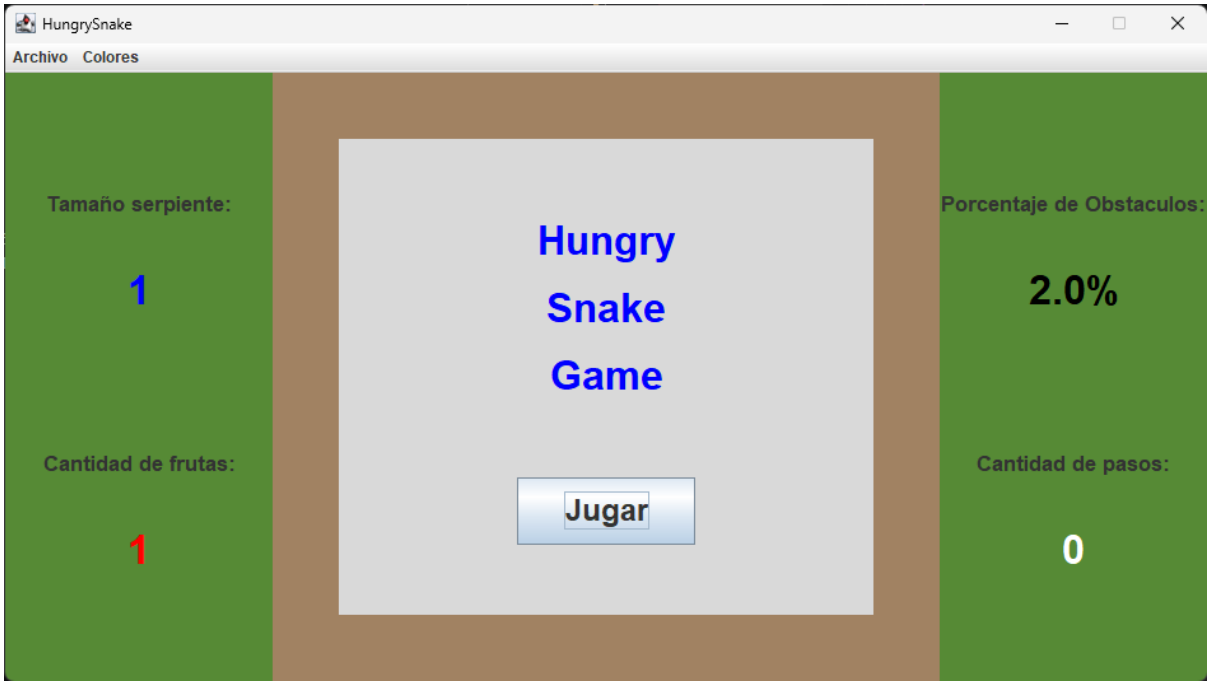
    }
```

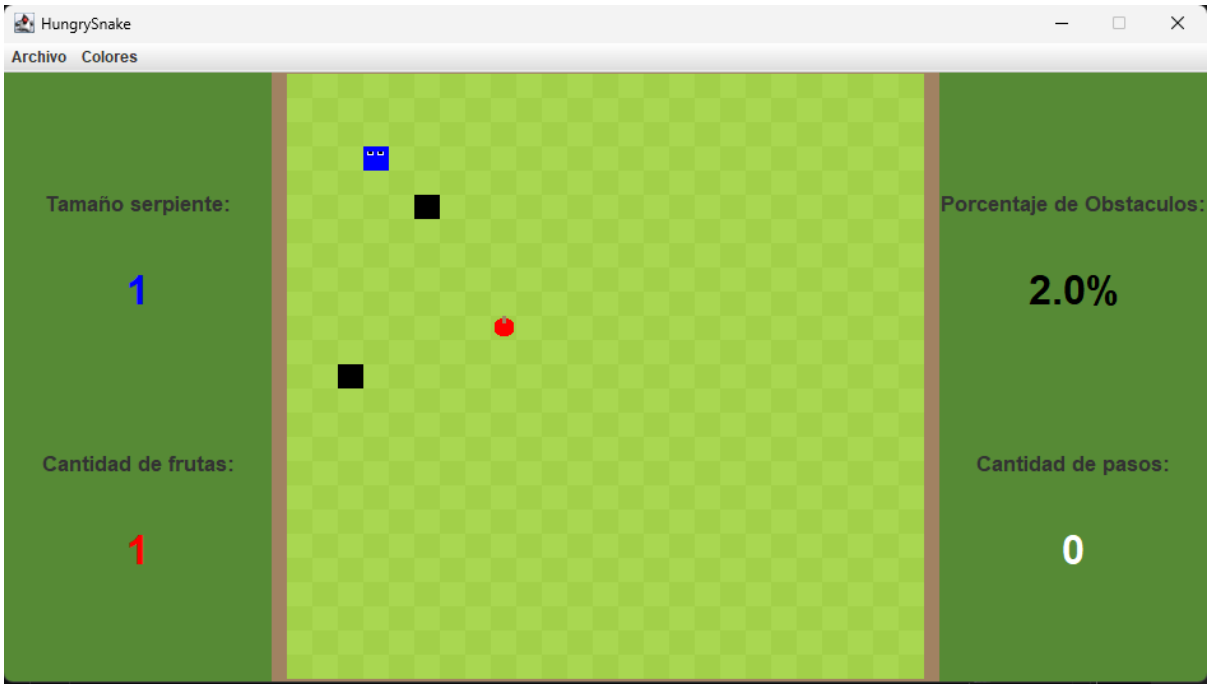
En los atributos se agrega un arreglo donde estan todos los colores de cada cosa:

```
//Game colors
private Color snakeColor = Color.BLUE;
private Color obstaclesColor = Color.BLACK;
private Color fruitsColor = Color.RED;
private Color firstFloorColor = new Color(162,208,73);
private Color secondFloorColor = new Color(169,215,81);
private ArrayList<Color> allColors;
```

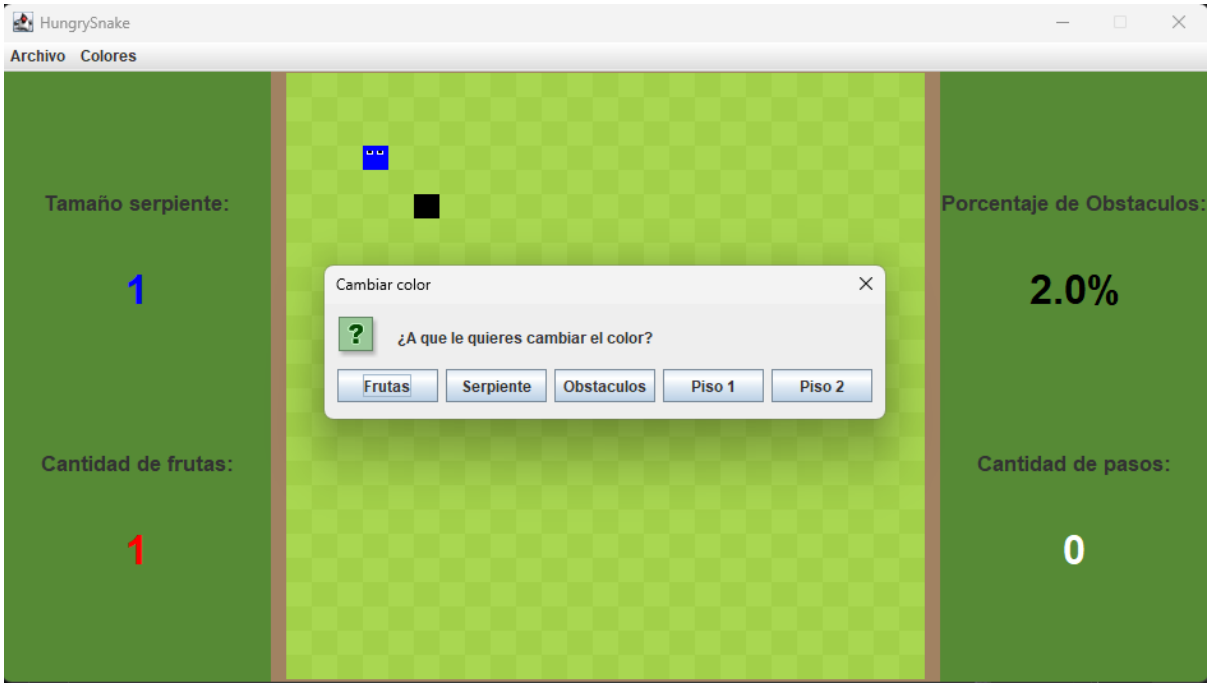
4. Ejecuten el caso de uso y capture las pantallas más significativas.

-se inicia:

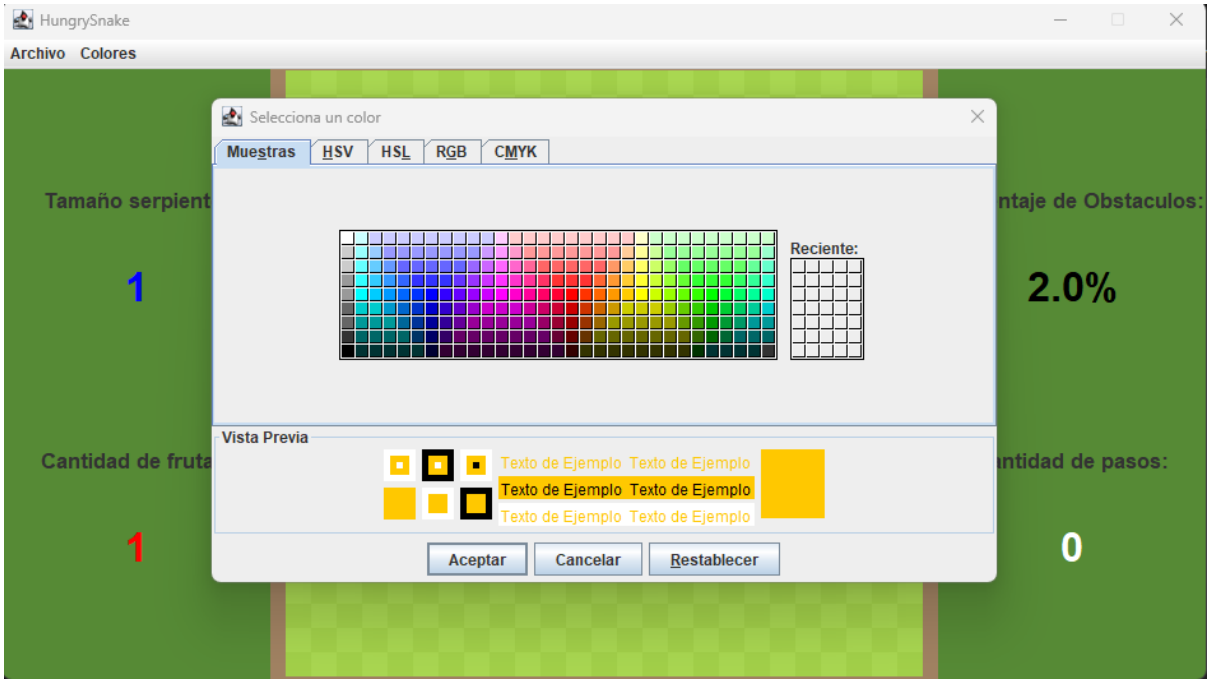

HungrySnake
Archivo Colores
Tamaño serpiente:
1
Cantidad de frutas:
1
Hungry
Snake
Game
Jugar
Porcentaje de Obstaculos:
2.0%
Cantidad de pasos:
0

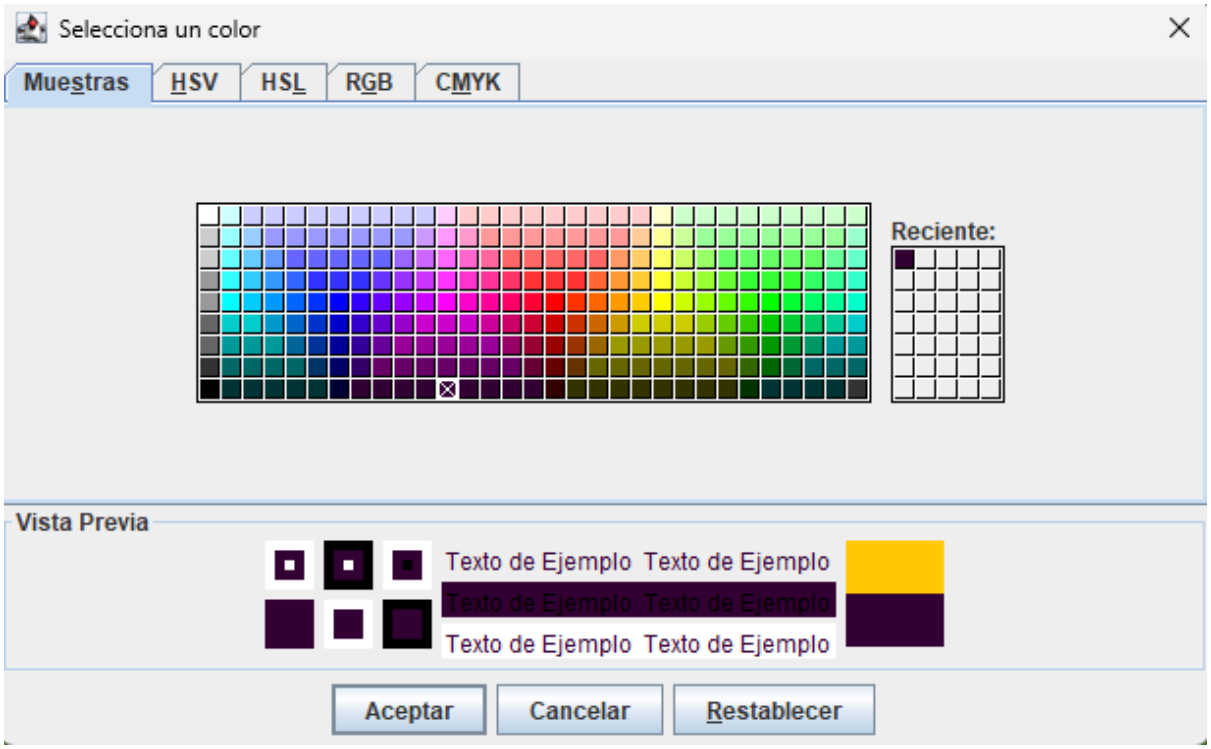

HungrySnake
Archivo Colores
Tamaño serpiente:
1
Cantidad de frutas:
1
Porcentaje de Obstaculos:
2.0%
Cantidad de pasos:
0





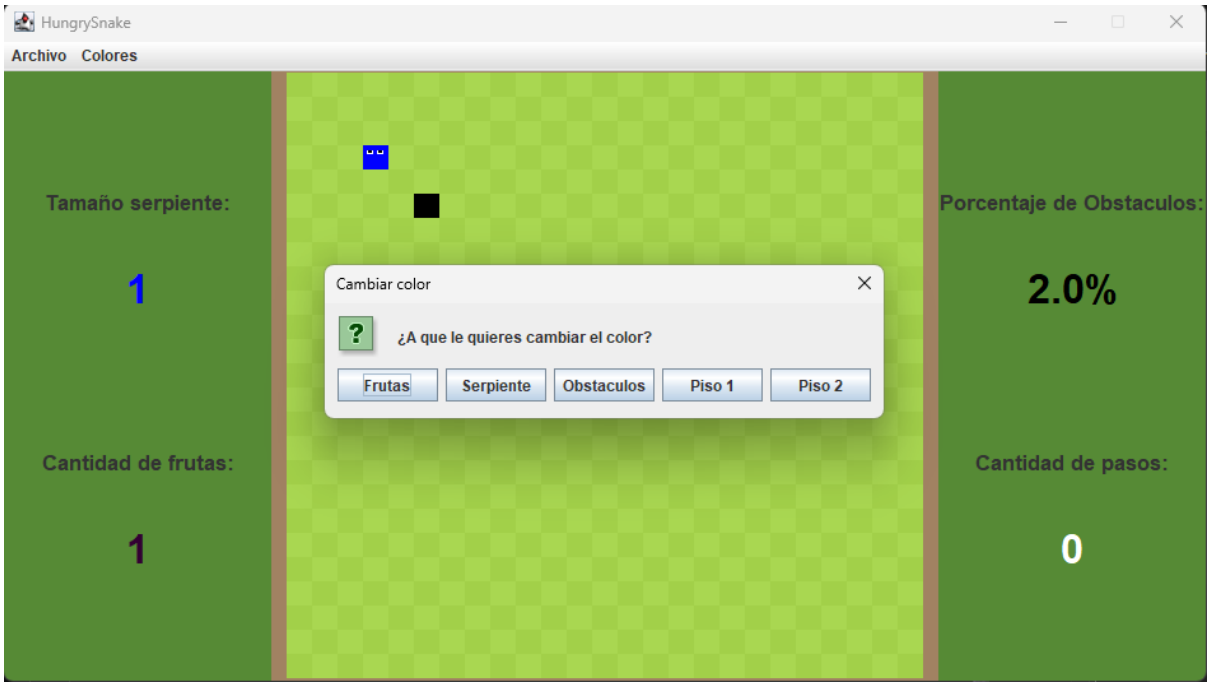
-Se le da la opcion de Frutas:


HungrySnake
Archivo Colores
Selecciona un color
Muestras HSV HSL RGB CMYK
Reciente:
Vista Previa
Texto de Ejemplo Texto de Ejemplo
Texto de Ejemplo Texto de Ejemplo
Texto de Ejemplo Texto de Ejemplo
Aceptar Cancelar Restablecer
Tamaño serpient
ntaje de Obstaculos:
1
2.0%
Cantidad de fruta
ntidad de pasos:
1
0

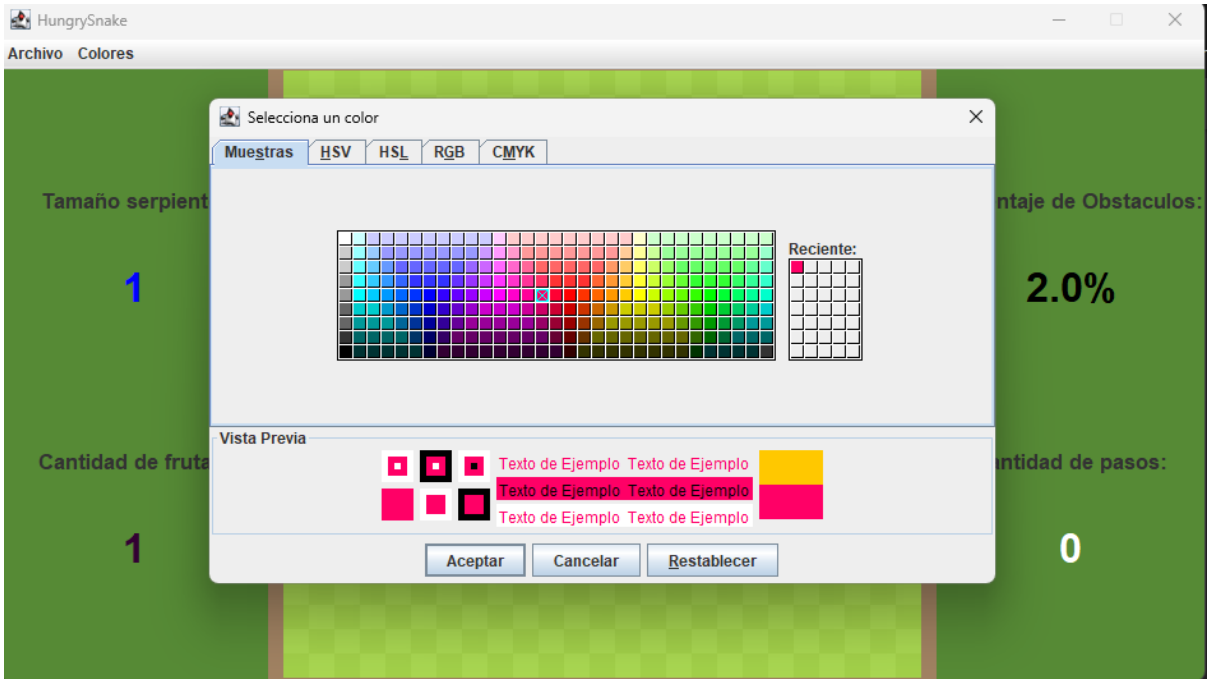

Selecciona un color
Muestras HSV HSL RGB CMYK
Reciente:
Vista Previa
Texto de Ejemplo Texto de Ejemplo
Texto de Ejemplo Texto de Ejemplo
Texto de Ejemplo Texto de Ejemplo
Aceptar Cancelar Restablecer

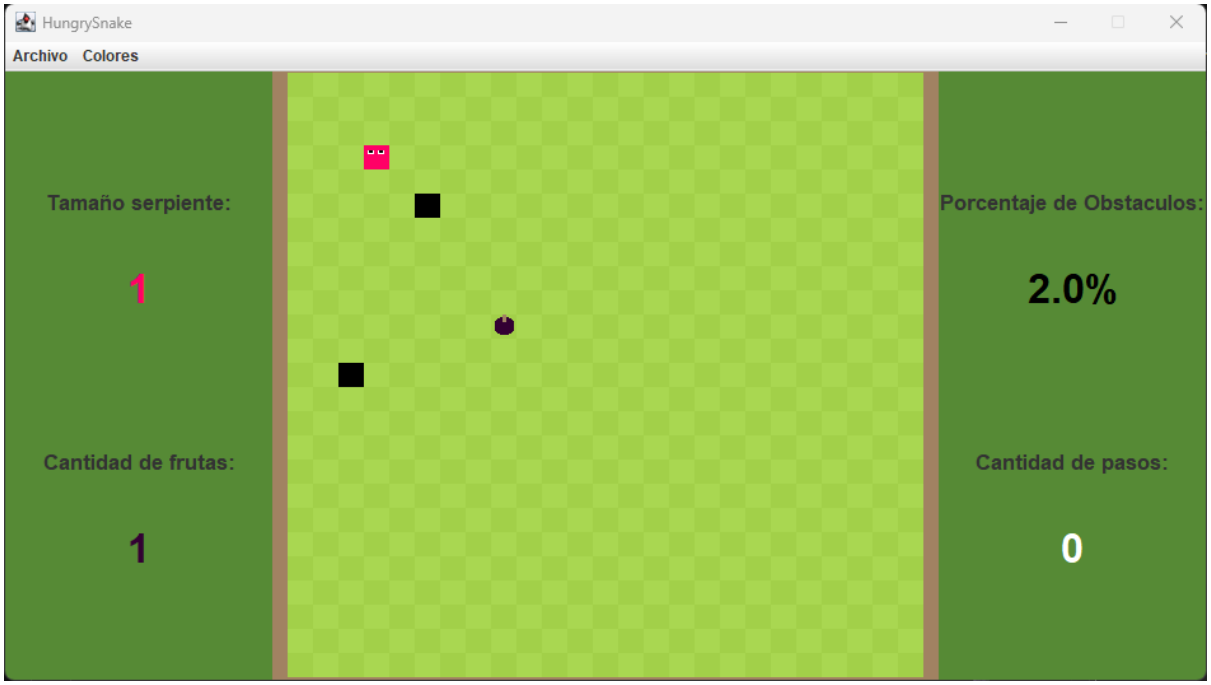


Se le vuelve a oprimir a Colores y cambiar color:

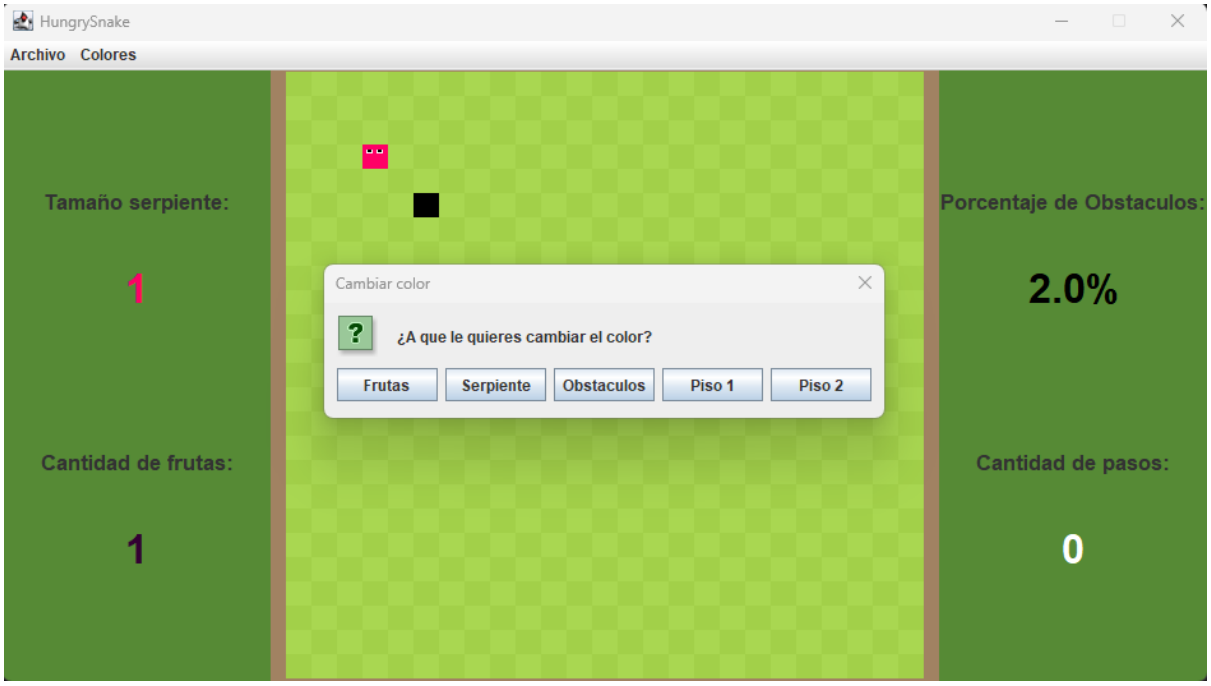


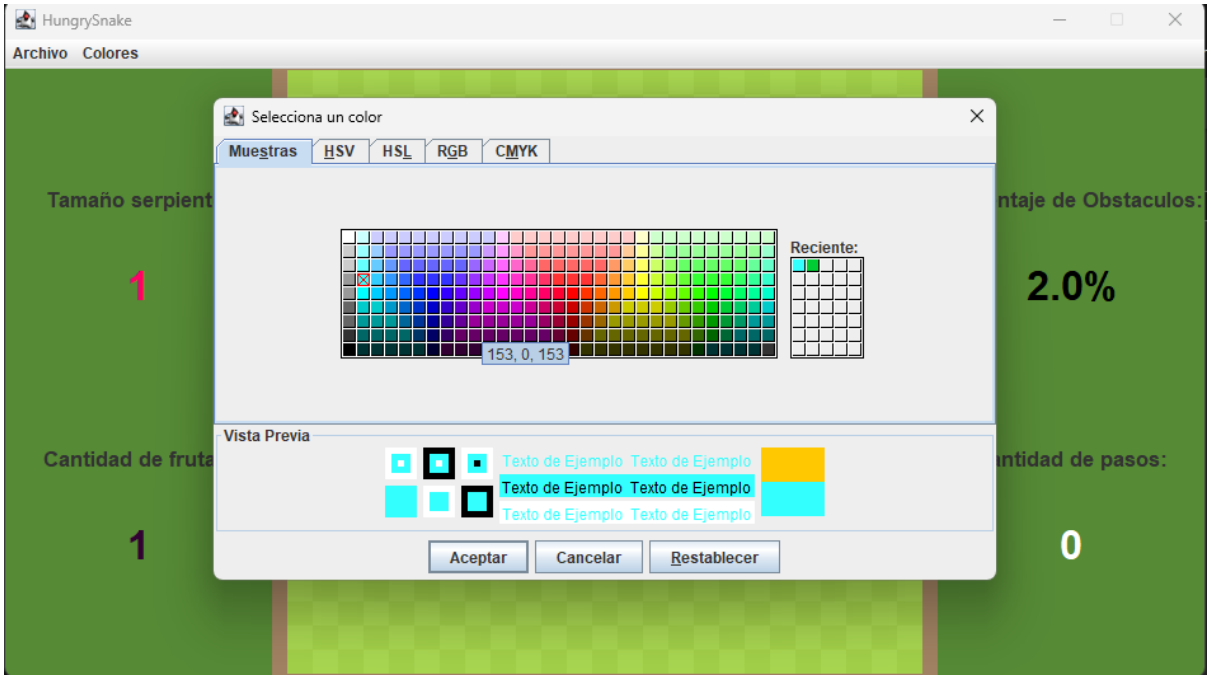
y se seleciona Serpiente:





Luego se vuelve a hacer y se seleciona Obstaculos:

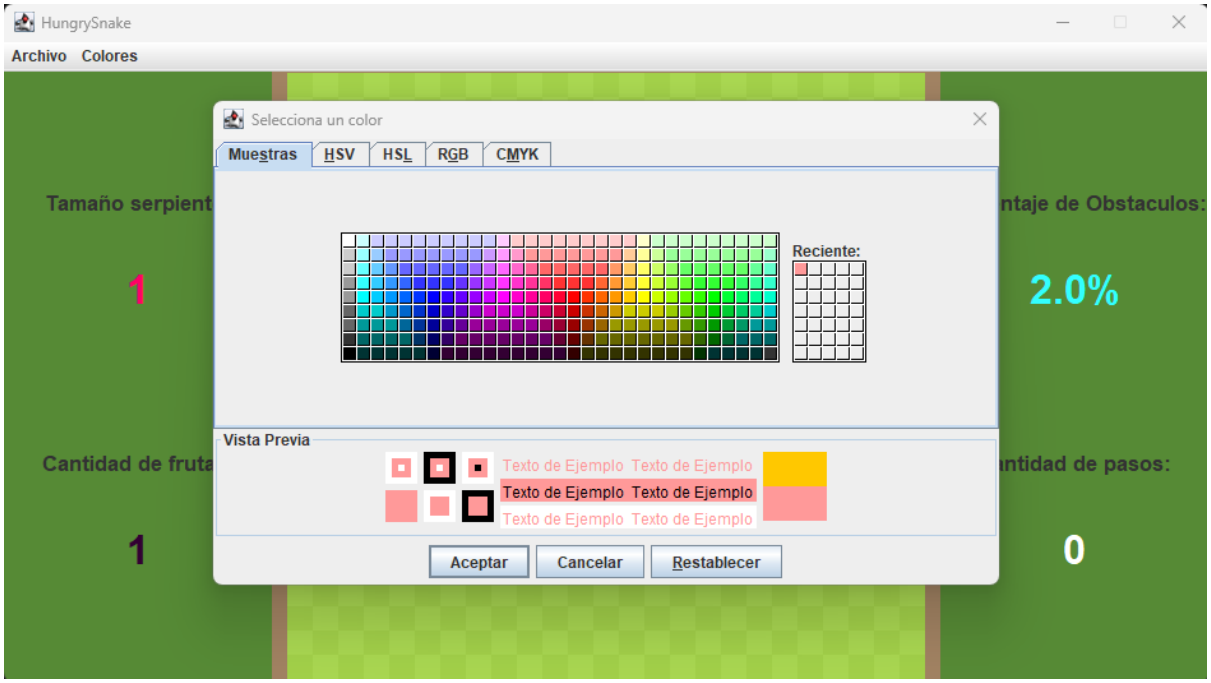

HungrySnake
Archivo Colores
Tamaño serpiente:
1
Cantidad de frutas:
1
Porcentaje de Obstaculos:
2.0%
Cantidad de pasos:
0
Cambiar color
¿A que le quieres cambiar el color?
Frutas
Serpiente
Obstaculos
Piso 1
Piso 2

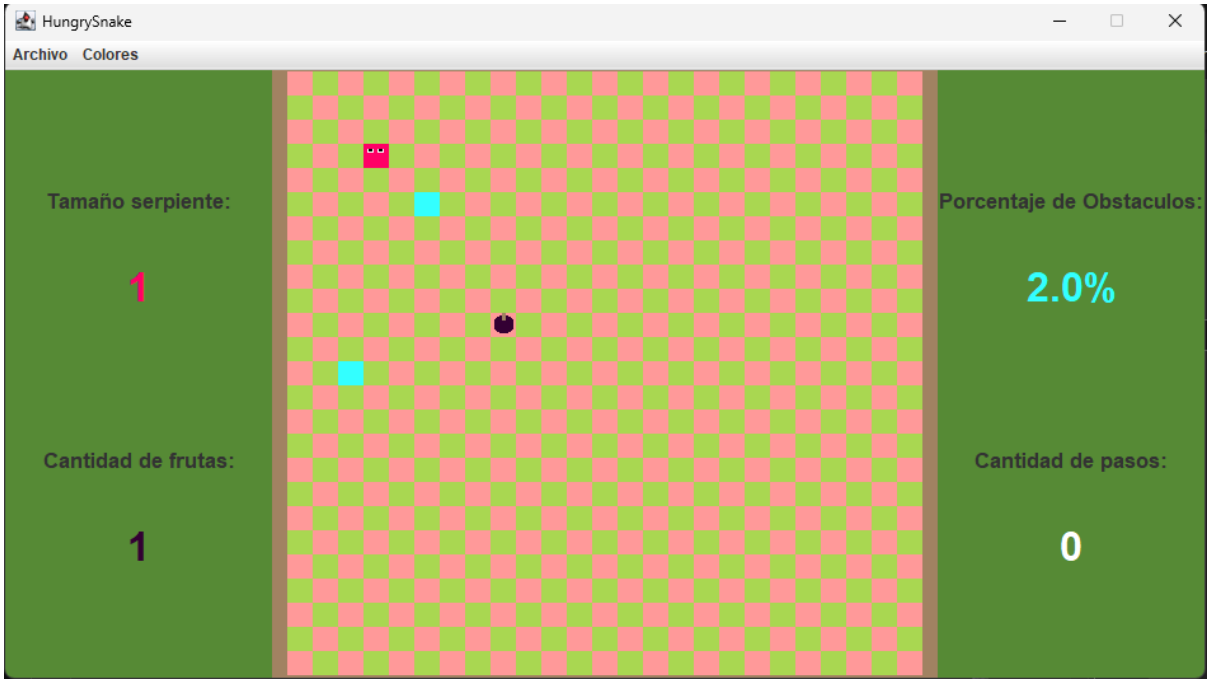

HungrySnake
Archivo Colores
Selecciona un color
Muestras
HSV
HSL
RGB
CMYK
Reciente:
153, 0, 153
Vista Previa
Texto de Ejemplo Texto de Ejemplo
Texto de Ejemplo Texto de Ejemplo
Texto de Ejemplo Texto de Ejemplo
Aceptar
Cancelar
Restablecer
1
2.0%
0

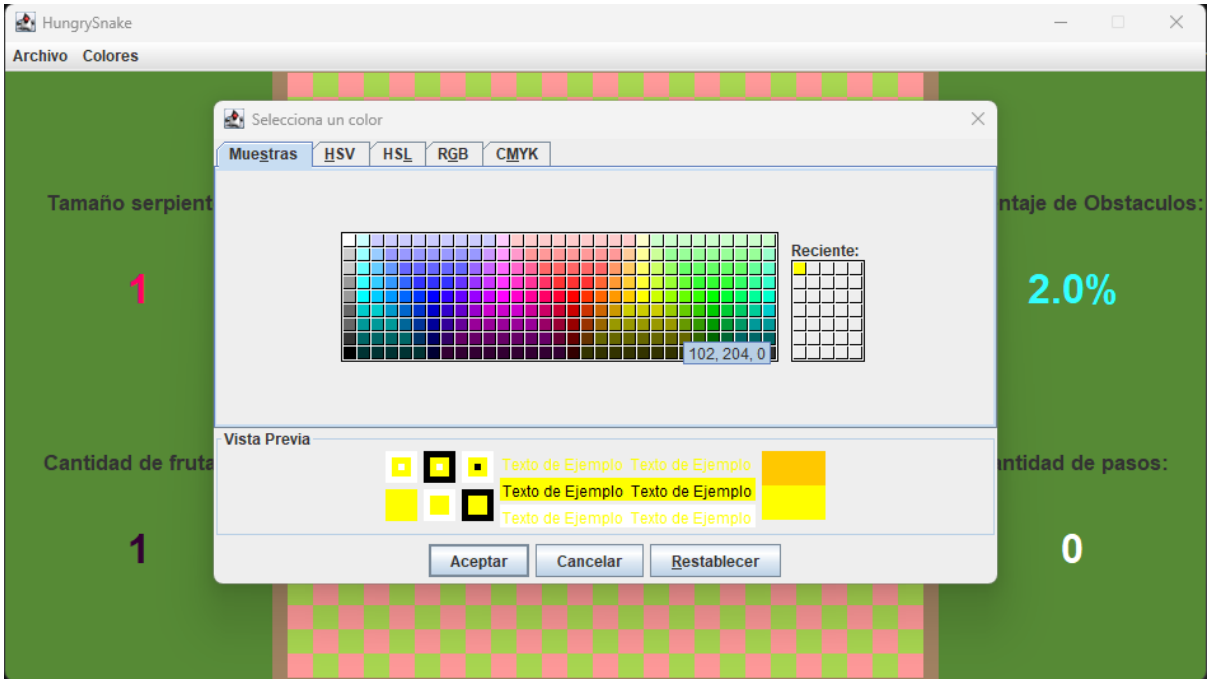


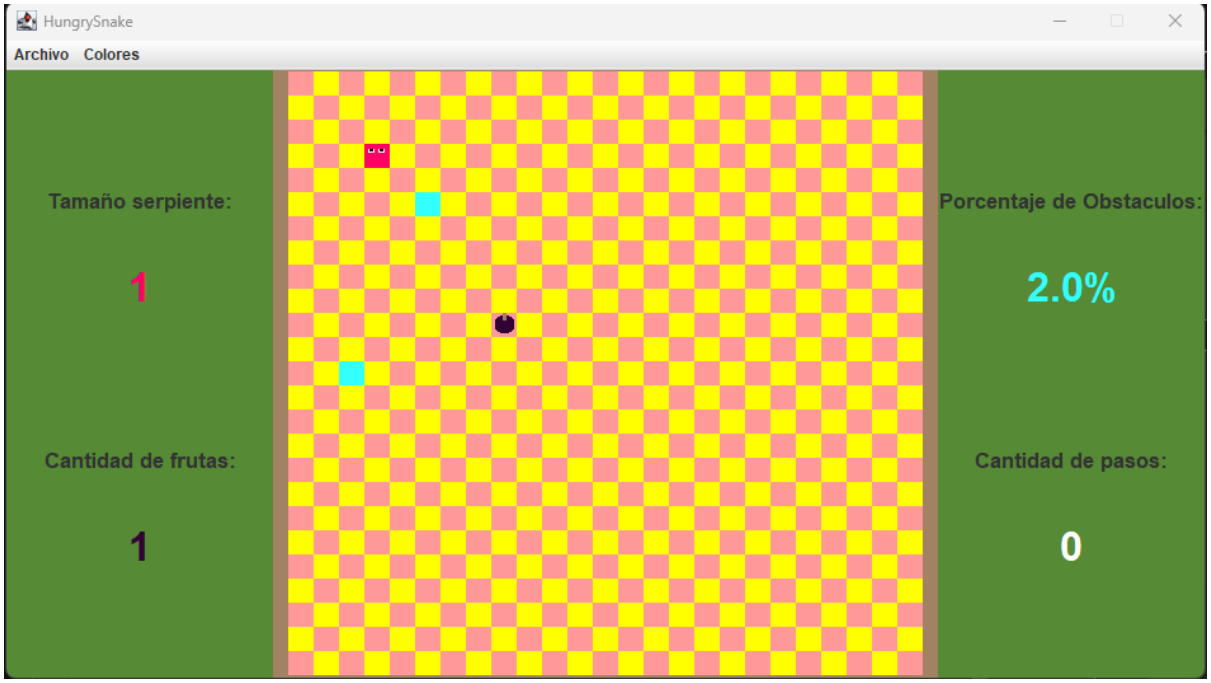
Luego a cada piso:




HungrySnake
Archivo Colores
Selecciona un color
Muestras HSV HSL RGB CMYK
Reciente:
Vista Previa
Texto de Ejemplo Texto de Ejemplo
Texto de Ejemplo Texto de Ejemplo
Texto de Ejemplo Texto de Ejemplo
Aceptar Cancelar Restablecer
Tamaño serpient
1
Cantidad de fruta
1
ntaje de Obstaculos:
2.0%
ntidad de pasos:
0


HungrySnake
Archivo Colores
Tamaño serpiente:
1
Cantidad de frutas:
1
Porcentaje de Obstaculos:
2.0%
Cantidad de pasos:
0



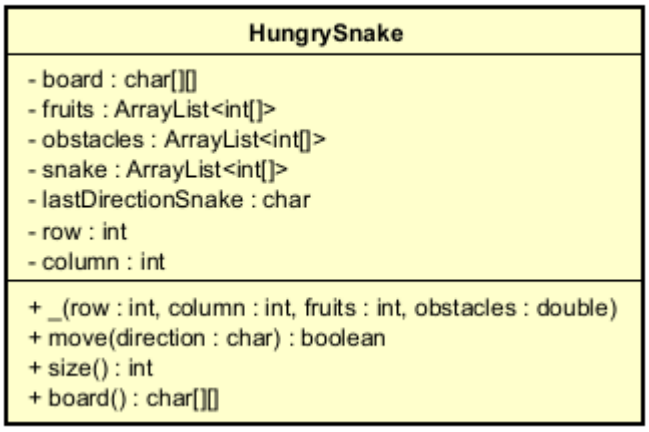


Parte II. Funcionalidades básicas

A Ciclo 5: Módelo HungrySnake.

El objetivo es implementar la capa de dominio para HungrySnake .

Para el diseño en astah:



1. Construya los métodos básicos del juego

Para HungrySnake se definen los siguientes atributos:

```java
/**
 * Represents a HungrySnake
 */
public class HungrySnake{
    private char[][] board;
    private ArrayList<int[]> fruits;
    private ArrayList<int[]> walls;
    private ArrayList<int[]> snake;
    private char lastDirectionSnake;
    private int row;
    private int column;
```

Se crearon 4 métodos:

- HungrySnake(int row, int column, int fruits, double obstacles): Este se encarga de cumplir los requisitos de:
    - Crear tablero
    - Poner obstaculos y frutas
    - Ubicar la serpiente

```java
/**
 * Create a HungrySnake
 * @param row are the game's rows
 * @param column are the game's columns
 * @param fruits are the fruits that game will have
 * @param obstacles are the percentage of obstacles
 * @throws HungrySnakeException
 */
public HungrySnake(int row, int column, int fruits, double obstacles) throws HungrySnakeException{
    if(row<=0 || column <=0 || fruits<0 || obstacles<0){
        throw new HungrySnakeException(HungrySnakeException.NEGATIVE_NUMBERS);
    }
    if((row*column)<= (fruits + ((row*column) * obstacles/100))){
        throw new HungrySnakeException(HungrySnakeException.CANT_PUT_A_SNAKE);
    }
    board = new char[row][column];
    this.fruits = new ArrayList<>();
    walls = new ArrayList<>();
    snake = new ArrayList<>();
    this.row = row;
    this.column = column;


    for(int i = 0; i<fruits; i++){
        int a = (int)(Math.random()*row);
        int b = (int)(Math.random()*column);
        while(board[a][b] !='\u0000'){
            a = (int)(Math.random()*row);
            b = (int)(Math.random()*column);
        }
        board[a][b]='F';
        this.fruits.add(new int[]{a,b});
    }
```

```
    int numObs = (int)((row * column) * obstacles/100);
    for(int i = 0; i<numObs; i++){
        int a = (int)(Math.random()*row);
        int b = (int)(Math.random()*column);
        while(board[a][b] !='\u0000'){
            a = (int)(Math.random()*row);
            b = (int)(Math.random()*column);
        }
        board[a][b]='O';
        walls.add(new int[]{a,b});
    }


    snake = new ArrayList<>();
    boolean found = false;
    for(int i = 0; i<row; i++){
        if(found == true){
            break;
        }
        for(int j = 0; j<column; j++){
            if(board[i][j] == '\u0000'){
                snake.add(new int[]{i,j});
                board[i][j] = 'H';
                lastDirectionSnake = 'E';
                found = true;
                break;

            }
        }
    }

}
```

- move(char direction): La relevancia de este es encargarse de mover la serpiente: Este se encarga de solucionar el requisito de:
    - Mover la serpiente

```java
/**
 * returns if the snake moved successfully
 * @param direction is the direction where the snake will move
 * @return if the snake didn't crash with an obstacle
 */
public boolean move(char direction) throws HungrySnakeException {
    boolean isOk = true;
    int posYHead = snake.get(0)[0];
    int posXHead = snake.get(0)[1];

    if (direction == 'N') {
        if (lastDirectionSnake != 'S' || size() <= 1) {
            int newY = posYHead - 1;
            int newX = posXHead;

            if (newY < 0) {
                throw new HungrySnakeException(HungrySnakeException.EXIT_EDGE);
            }

            if (board[newY][newX] == 'F') {
                board[posYHead][posXHead] = 'S';
                board[newY][newX] = 'H';
                snake.add(0, new int[]{newY, newX});
                lastDirectionSnake = 'N';
                if(size()+walls.size()<row*column){
                    int a = (int)(Math.random()*row);
                    int b = (int)(Math.random()*column);
                    while(board[a][b] !='\u0000'){
                        a = (int)(Math.random()*row);
                        b = (int)(Math.random()*column);
                    }
                    board[a][b]='F';
                    fruits.remove(new int[]{newY, newX});
                    fruits.add(new int[]{a,b});
                }
```

```
            } else if (board[newY][newX] == '\u0000') {
                int[] tempTail = snake.remove(size() - 1);
                board[tempTail[0]][tempTail[1]] = '\u0000';

                board[newY][newX] = 'H';
                snake.add(0, new int[]{newY, newX});
                lastDirectionSnake = 'N';
            } else {
                isOk = false;
            }
        } else {
            throw new HungrySnakeException(HungrySnakeException.LAST_DIRECTION);
        }
    } else if (direction == 'S') {
        if (lastDirectionSnake != 'N' || size() <= 1) {
            int newY = posYHead + 1;
            int newX = posXHead;

            if (newY >= this.row) {
                throw new HungrySnakeException(HungrySnakeException.EXIT_EDGE);
            }

            if (board[newY][newX] == 'F') {
                board[posYHead][posXHead] = 'S';
                board[newY][newX] = 'H';
                snake.add(0, new int[]{newY, newX});
                lastDirectionSnake = 'S';
                if(size()+walls.size()<row*column){
                    int a = (int)(Math.random()*row);
                    int b = (int)(Math.random()*column);
                    while(board[a][b] !='\u0000'){
                        a = (int)(Math.random()*row);
                        b = (int)(Math.random()*column);
                    }
                    board[a][b]='F';
                    fruits.remove(new int[]{newY, newX});
                    fruits.add(new int[]{a,b});
                }

            } else if (board[newY][newX] == '\u0000') {
                int[] tempTail = snake.remove(size() - 1);
                board[tempTail[0]][tempTail[1]] = '\u0000';
```

```
                board[newY][newX] = 'H';
                snake.add(0, new int[]{newY, newX});
                lastDirectionSnake = 'S';
            } else {
                isOk = false;
            }
        } else {
            throw new HungrySnakeException(HungrySnakeException.LAST_DIRECTION);
        }
    } else if (direction == 'E') {
        if (lastDirectionSnake != 'W' || size() <= 1) {
            int newY = posYHead;
            int newX = posXHead + 1;

            if (newX >= this.column) {
                throw new HungrySnakeException(HungrySnakeException.EXIT_EDGE);
            }

            if (board[newY][newX] == 'F') {
                board[posYHead][posXHead] = 'S';
                board[newY][newX] = 'H';
                snake.add(0, new int[]{newY, newX});
                lastDirectionSnake = 'E';
                if(size()+walls.size()<row*column){
                    int a = (int)(Math.random()*row);
                    int b = (int)(Math.random()*column);
                    while(board[a][b] !='\u0000'){
                        a = (int)(Math.random()*row);
                        b = (int)(Math.random()*column);
                    }
                    board[a][b]='F';
                    fruits.remove(new int[]{newY, newX});
                    fruits.add(new int[]{a,b});
                }
            } else if (board[newY][newX] == '\u0000') {
                int[] tempTail = snake.remove(size() - 1);
                board[tempTail[0]][tempTail[1]] = '\u0000';

                board[newY][newX] = 'H';
                snake.add(0, new int[]{newY, newX});
                lastDirectionSnake = 'E';
```

```
            } else {
                isOk = false;
            }
        } else {
            throw new HungrySnakeException(HungrySnakeException.LAST_DIRECTION);
        }
    } else if (direction == 'W') {
        if (lastDirectionSnake != 'E' || size() <= 1) {
            int newY = posYHead;
            int newX = posXHead - 1;

            if (newX < 0) {
                throw new HungrySnakeException(HungrySnakeException.EXIT_EDGE);
            }

            if (board[newY][newX] == 'F') {
                board[posYHead][posXHead] = 'S';
                board[newY][newX] = 'H';
                snake.add(0, new int[]{newY, newX});
                lastDirectionSnake = 'W';
                if(size()+walls.size()<row*column){
                    int a = (int)(Math.random()*row);
                    int b = (int)(Math.random()*column);
                    while(board[a][b] !='\u0000'){
                        a = (int)(Math.random()*row);
                        b = (int)(Math.random()*column);
                    }
                    board[a][b]='F';
                    fruits.remove(new int[]{newY, newX});
                    fruits.add(new int[]{a,b});
                }
            } else if (board[newY][newX] == '\u0000') {
                int[] tempTail = snake.remove(size() - 1);
                board[tempTail[0]][tempTail[1]] = '\u0000';

                board[newY][newX] = 'H';
                snake.add(0, new int[]{newY, newX});
                lastDirectionSnake = 'W';
            } else {
                isOk = false;
            }
        } else {
            throw new HungrySnakeException(HungrySnakeException.LAST_DIRECTION);
        }
    }
    return isOk;
```

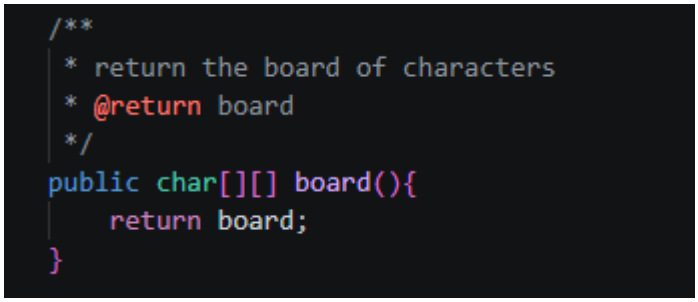
Se crea el método snakeSize(): El cual se encarga de devolver el tamaño de la serpiente:

Se encarga de solucionar el requisito de:

- Consultar tamaño de la serpiente

```
/**
 * return the size of the arraylist snake
 * @return snake's size
 */
public int size(){
    return snake.size();
}
```

Por último se encuentra el método de board() que se encarga de devolver todo el board:



```java
/**
 * return the board of characters
 * @return board
 */
public char[][] board(){
    return board;
}
```

2. Ejecuten las pruebas y capturen el resultado.

Se crearon las siguientes pruebas:

```
st > HungrySnakeTest.java > HungrySnakeTest > shouldNotCreateBoardWithNegativeNumbers()
package test;
import domain.*;

import static org.junit.Assert.*;
import org.junit.After;
import org.junit.Before;
import org.junit.Test;

public class HungrySnakeTest {

    /**
     * Shouldn't create a board with negative numbers
     */

    @Test
    public void shouldNotCreateBoardWithNegativeNumbers(){
        try{
            HungrySnake game = new HungrySnake(-1,-1,-1,-1);
            fail(message: "Didn't throw an exception");
        }
        catch(HungrySnakeException e){
            assertEquals(e.getMessage(), HungrySnakeException.NEGATIVE_NUMBERS);
        }
    }
```

```
    /**
     * Shouldn't create a board where the snake can't enter
     */
    @Test
    public void shouldNotCreateABoardWhereSnakeCanNotEnter(){
      try{
            HungrySnake game = new HungrySnake(row: 10,column: 10,fruits: 50,obstacles: 50);
            fail(message: "Didn't throw an exception");
        }
        catch(HungrySnakeException e){
            assertEquals(e.getMessage(), HungrySnakeException.CANT_PUT_A_SNAKE);
        }
    }
    /**
     * Should create a board where just there's a space for the snake
     */
    @Test
    public void shouldCreateABoardWhereThereIsAtLeastOnePositionForSnake(){
        try{
            HungrySnake game = new HungrySnake(row: 10,column: 10,fruits: 49,obstacles: 50);
            char[][] board = game.board();
            assertEquals(expected: 10,board[0].length);
            assertEquals(expected: 10,board.length);


        }
        catch(HungrySnakeException e){
            fail(message: "Did throw an exception");
        }
    }
```

```java
/**
 * Should add correct number of fruits to the board.
 */
@Test
public void shouldAddCorrectNumberOfFruitsToTheBoard(){
    try{
        HungrySnake game = new HungrySnake(row: 10,column: 10,fruits: 50,obstacles: 0);
        char[][] board = game.board();
        int fruitsNumber = 0;
        for(int i = 0; i < 10; i++){
            for(int j = 0; j < 10; j++){
                if (board[i][j] == 'F'){
                    fruitsNumber++;
                }
            }
        }
        assertEquals(expected: 50,fruitsNumber);


    }
    catch(HungrySnakeException e){
        fail(message: "Did throw an exception");
    }
}
```

```java
/**
 * Should add correct percentage of obstacles to the board.
 */
@Test
public void shouldAddCorrectNumberOfObstaclesToTheBoard(){
    try{
        HungrySnake game = new HungrySnake(row: 10,column: 10,fruits: 0,obstacles: 50);
        char[][] board = game.board();
        int obstaclesNumber = 0;
        for(int i = 0; i < 10; i++){
            for(int j = 0; j < 10; j++){
                if (board[i][j] == 'O'){
                    obstaclesNumber++;
                }
            }
        }
        assertEquals(expected: 50,obstaclesNumber);


    }
    catch(HungrySnakeException e){
        fail(message: "Did throw an exception");
    }
}
```

```java
 */
@Test public void shouldSnakeMoveCorrectly(){
    try{
        HungrySnake game = new HungrySnake(row: 10,column: 10,fruits: 0,obstacles: 0);
        char[][] board = game.board();
        int[] snakePosition = null;

        for(int i = 0; i < 10; i++){
            if(snakePosition != null){
                break;
            }
            for(int j = 0; j < 10; j++){
                if (board[i][j] == 'H'){
                    snakePosition = new int[]{i,j};
                }
            }
        }
        int yPos = snakePosition[0];
        int xPos = snakePosition[1];
        if(snakePosition[0] != 0){
            game.move(direction: 'N');
            board = game.board();
            assertEquals(expected: 'H', board[snakePosition[0]-1][snakePosition[1]]);
        }
        else if(snakePosition[1] != 0){
            game.move(direction: 'W');
            board = game.board();
            assertEquals(expected: 'H', board[snakePosition[0]][snakePosition[1]-1]);
        }
        else if(snakePosition[0] != 9){
            game.move(direction: 'S');
            board = game.board();
            assertEquals(expected: 'H', board[snakePosition[0]+1][snakePosition[1]]);
        }else if(snakePosition[1] != 9){
            game.move(direction: 'E');
            board = game.board();
            assertEquals(expected: 'H', board[snakePosition[0]-1][snakePosition[1]]);
        }


    }
    catch(HungrySnakeException e){
        fail(message: "Did throw an exception");
    }
}
```

```java
@Test
public void shouldJustBeOneHeadIfSnakeMoves(){
    try{
        HungrySnake game = new HungrySnake(row: 10,column: 10,fruits: 0,obstacles: 0);
        char[][] board = game.board();
        int[] snakePosition = null;

        for(int i = 0; i < 10; i++){
            if(snakePosition != null){
                break;
            }
            for(int j = 0; j < 10; j++){
                if (board[i][j] == 'H'){
                    snakePosition = new int[]{i,j};
                }
            }
        }
        int yPos = snakePosition[0];
        int xPos = snakePosition[1];
        if(snakePosition[0] != 0){
            game.move(direction: 'N');
        }
        else if(snakePosition[1] != 0){
            game.move(direction: 'W');
        }
        else if(snakePosition[0] != 9){
            game.move(direction: 'S');
        }else if(snakePosition[1] != 9){
            game.move(direction: 'E');
        }
        board = game.board();
        int headsNumber = 0;
        for(int i = 0; i<10; i++){
            for(int j = 0; j <10; j++){
                if (board[i][j] == 'H'){
                    headsNumber++;
                }
            }
        }
        assertEquals(expected: 1, headsNumber);

    }
    catch(HungrySnakeException e){
        fail(message: "Did throw an exception");
    }
}
```

```java
/**
 * Snake should not exit the edge.
 */
@Test
public void shouldNotExitTheEdge(){
    try{
        HungrySnake game = new HungrySnake(row: 5,column: 5,fruits: 0,obstacles: 0);
        for(int i = 0; i < 6; i++){
            game.move(direction: 'E');
        }
    }
    catch(HungrySnakeException e){
        assertEquals(HungrySnakeException.EXIT_EDGE,e.getMessage());
    }

}
```

```java
/**
 * Should fruit be replaced and snake must grow.
 */
@Test
public void shouldFruitBeReplacedAndSnakeGrows(){
    try{
        HungrySnake game = new HungrySnake(row: 5,column: 1,fruits: 1,obstacles: 0);
        char[][] board = game.board();
        int snakePosY = 0;
        int fruitPosY = 0;
        for(int i = 0; i<5; i++){
            if(board[i][0] == 'F'){
                fruitPosY = i;
            }
            else if(board[i][0] == 'H'){
                snakePosY = i;
            }
        }
        if(snakePosY<fruitPosY){
            for(int i =0; i < fruitPosY-snakePosY; i++){
                game.move(direction: 'S');
            }
        }
        else{
            for(int i =0; i < snakePosY-fruitPosY; i++){
                game.move(direction: 'N');
            }
        }
        board = game.board();
        int fruitFound = 0;

        for(int i = 0; i<5; i++){
            if(board[i][0] == 'F'){
                fruitFound++;
            }
        }
        assertEquals(expected: 2, game.size());
        assertEquals(expected: 1, fruitFound);
    }
    catch(HungrySnakeException e){
        fail(message: "Did throw an exception");
    }
}
```

```java
/**
 * Should not add more fruits if there's no space
 */
@Test
public void shouldNotAddMoreFruitsIfThereIsNoSpace(){
    try{
        HungrySnake game = new HungrySnake(row: 2,column: 1,fruits: 1,obstacles: 0);
        char[][] board = game.board();
        int snakePosY = 0;
        int fruitPosY = 0;
        for(int i = 0; i<2; i++){
            if(board[i][0] == 'F'){
                fruitPosY = i;
            }
            else if(board[i][0] == 'H'){
                snakePosY = i;
            }
        }
        if(snakePosY<fruitPosY){
            for(int i =0; i < fruitPosY-snakePosY; i++){
                game.move(direction: 'S');
            }
        }
        else{
            for(int i =0; i < snakePosY-fruitPosY; i++){
                game.move(direction: 'N');
            }
        }
        board = game.board();
        int fruitFound = 0;

        for(int i = 0; i<2; i++){
            if(board[i][0] == 'F'){
                fruitFound++;
            }
        }
        assertEquals(expected: 2, game.size());
        assertEquals(expected: 0, fruitFound);
    }
    catch(HungrySnakeException e){
        fail(message: "Did throw an exception");
    }
}
```

```java
/**
 * Shouldn't be ok the move is snake collides with an obstacle
 */
public void shouldNotBeOkIfSnakeCollidesWithAnObstacle(){
    try{
        HungrySnake game = new HungrySnake(row: 2,column: 1,fruits: 1,obstacles: 0);
        char[][] board = game.board();
        int snakePosY = 0;
        int obstaclePosY = 0;
        for(int i = 0; i<2; i++){
            if(board[i][0] == 'O'){
                obstaclePosY = i;
            }
            else if(board[i][0] == 'H'){
                snakePosY = i;
            }
        }
        if(snakePosY<obstaclePosY){
            for(int i =0; i < obstaclePosY-snakePosY; i++){
                assertFalse(game.move(direction: 'S'));
            }
        }
        else{
            for(int i =0; i < snakePosY-obstaclePosY; i++){
                assertFalse(game.move(direction: 'N'));
            }
        }

    }
    catch(HungrySnakeException e){
        fail(message: "Did throw an exception");
    }
}
```

```java
/**
 * If snake moves the last space where it was must be void.
 */
@Test
public void shouldSpaceSnakeWasBeNull(){

    try{
        HungrySnake game = new HungrySnake(row: 10,column: 10,fruits: 0,obstacles: 0);
        char[][] board = game.board();
        int[] snakePosition = null;

        for(int i = 0; i < 10; i++){
            if(snakePosition != null){
                break;
            }
            for(int j = 0; j < 10; j++){
                if (board[i][j] == 'H'){
                    snakePosition = new int[]{i,j};
                }
            }
        }
        int yPos = snakePosition[0];
        int xPos = snakePosition[1];
        if(snakePosition[0] != 0){
            game.move(direction: 'N');
        }
        else if(snakePosition[1] != 0){
            game.move(direction: 'W');
        }
        else if(snakePosition[0] != 9){
            game.move(direction: 'S');
        }else if(snakePosition[1] != 9){
            game.move(direction: 'E');
        }
        board = game.board();
        assertTrue('\u0000'==game.board()[yPos][xPos]);

    }
    catch(HungrySnakeException e){
        fail(message: "Did throw an exception");
    }
}
```

```java
/**
 * Snake shouldn't move back if it has at least two of length
 */
@Test
public void shouldNotMoveBackIfItHasAtLeastTwoOfLength(){
    try{
        HungrySnake game = new HungrySnake(row: 5,column: 1,fruits: 1,obstacles: 0);
        char[][] board = game.board();
        int snakePosY = 0;
        int fruitPosY = 0;
        for(int i = 0; i<5; i++){
            if(board[i][0] == 'F'){
                fruitPosY = i;
            }
            else if(board[i][0] == 'H'){
                snakePosY = i;
            }
        }
        if(snakePosY<fruitPosY){
            for(int i =0; i < fruitPosY-snakePosY; i++){
                game.move(direction: 'S');
                game.move(direction: 'N');
            }
        }
        else{
            for(int i =0; i < snakePosY-fruitPosY; i++){
                game.move(direction: 'N');
                game.move(direction: 'S');

            }
        }

         fail(message: "Did throw an exception");
    }
    catch(HungrySnakeException e){
        assertEquals(HungrySnakeException.LAST_DIRECTION, e.getMessage());
    }
}
```

Para ejecutar las pruebas se usan los siguientes comandos:

Primero se compila el dominio:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 12\Garzon-Mendivelso\HungrySnake>javac -d bin src\domain\*.java
```

Luego se compilan las pruebas:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 12\Garzon-Mendivelso\HungrySnake>javac -cp "bin;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" -d bin src\test\
*.java
```

Al correr las pruebas:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 12\Garzon-Mendivelso\HungrySnake>java -cp "bin;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.runner.
JUnitCore test.HungrySnakeTest
JUnit version 4.13.2
............
Time: 0,029

OK (12 tests)
```

B Ciclo 6: Jugar

El objetivo es implementar el juego

1. Expliquen los elementos necesarios para implementar este caso de uso.
   Es necesario primero tener un atributo de tipo HungrySnake para lograr usar toda la lógica del dominio .Posteriormente para adaptarnos es necesario modificar algunas cosas que ya existían antes, especialmente el refresh.
   - Se importa el dominio y se agrega el nuevo atributo:

```java
package presentation;
import domain.*;

import javax.swing.*;
import java.awt.event.*;
import java.io.File;
import java.awt.*;
import java.util.ArrayList;

/**
 * Represents a HungrySnakeGUI
 * HungrySnakeGUI
 */

public class HungrySnakeGUI extends JFrame{
    //JFrame size
    private static final Dimension PREFERRED_DIMENSION = new Dimension (Toolkit.getDefaultToolkit().getScreenSize().width/2, Toolkit.getDefaultToolkit().getScreenSize().height/2);
    //For JPanelArea there's less 16px for width and there's less 64px for the height

    //Model
    private HungrySnake hungrySnake;
```

   - Luego en refresh, se fusiona el tablero de el dominio para que pinte los componentes correspondientes:

```java
private void refresh(){
    char[][] gameBoard = hungrySnake.board();

    for(int i = 0; i < gameBoard.length; i++){
        for(int j = 0;j< gameBoard[0].length; j++){
            board[i][j].removeAll();
            paintPanelComponents(gameBoard[i][j], i,j);
        }
    }


    obstacles.setText(String.valueOf(obstaclesNumber)+"%");
    steps.setText(String.valueOf(stepsNumber));
    fruits.setText(String.valueOf(fruitsNumber));
    length.setText(String.valueOf(hungrySnake.size()));

    game.revalidate();
    game.repaint();
}
```

2. Implementen los componentes necesarios para jugar. ¿Cuántos oyentes necesitan? ¿Por qué?
   En el método prepareControlleBoard() se necesitan 2 oyentes en total (además del que ya estaba en la barra del menú).
   1. Un ActionListener para el botón jugar: Es necesario para detectar el clic en la pantalla de bienvenida e invocar el método startGameAction(), el cual solicita los parámetros iniciales, instancia el modelo HungrySnake y reemplaza la pantalla de inicio por la cuadrícula del tablero.
   2. Un único KeyListener (mediante la clase anónima KeyAdapter) registrado sobre el panel game: Solo se necesita un oyente de teclado para manejar las cuatro direcciones de movimiento porque, dentro del método keyPressed(KeyEvent e), el objeto KeyEvent nos permite identificar exactamente qué tecla se presionó mediante e.getKeyCode(). Evaluando las constantes VK_RIGHT, VK_LEFT, VK_UP y VK_DOWN, un solo oyente qu mande la acción a moveAction('E'), moveAction('W'), moveAction('N') o moveAction('S') respectivamente, sin necesidad de crear un oyente distinto por cada flecha.

```
private void prepareControllerBoard(){
    jugar.addActionListener( new ActionListener(){
        public void actionPerformed(ActionEvent e){
            startGameAction();
        }
    });
    game.addKeyListener(new KeyAdapter(){
        @Override
        public void keyPressed(KeyEvent e){
            if(e.getKeyCode() == KeyEvent.VK_RIGHT){
                moveAction(direction: 'E');
            }
            else if (e.getKeyCode() == KeyEvent.VK_LEFT){
                moveAction(direction: 'W');
            }
            else if (e.getKeyCode() == KeyEvent.VK_UP){
                moveAction(direction: 'N');
            }
            else if (e.getKeyCode() == KeyEvent.VK_DOWN){
                moveAction(direction: 'S');
            }
        }

    });
}
```

En el método moveAction(char direction) es donde se une el dominio con la presentación, invocando hungrySnake.move(direction) y actualizando el tablero con refresh():

```
private void moveAction(char direction){
    try {
        if(!hungrySnake.move(direction)){
            ended = true;
            lastDirection = direction;
            decreaseSnakeColor();
            refresh();
            JOptionPane.showMessageDialog(this, "¡Se acabó el juego!", "Fin", JOptionPane.INFORMATION_MESSAGE);
            game.setFocusable(false);
            JOptionPane.showMessageDialog(this, "Estadísticas de la partida:"+"\n"+"Tamaño máximo de la serpiente: "+String.valueOf(hungrySnake.size())+"\n"+"Pasos dados: "+ String.valueOf(stepsNumber)+"\n"+"Frut
            ended = true;
        }
        else{
            lastDirection = direction;
            refresh();
            stepsNumber++;
            if(wonHungrySnake()){
                JOptionPane.showMessageDialog(this, "¡Ganaste! ¡Felicidades!", "Fin", JOptionPane.INFORMATION_MESSAGE);
                game.setFocusable(false);
                JOptionPane.showMessageDialog(this, "Estadísticas de la partida:"+"\n"+"Tamaño máximo de la serpiente: "+String.valueOf(hungrySnake.size())+"\n"+"Pasos dados: "+ String.valueOf(stepsNumber)+"\n"+"
                ended = true;
            }
        }

    } catch (HungrySnakeException e) {
        if(e.getMessage().equals(HungrySnakeException.EXIT_EDGE)){
            lastDirection = direction;
            ended = true;
            decreaseSnakeColor();
            refresh();
            JOptionPane.showMessageDialog(this, HungrySnakeException.EXIT_EDGE +"\n"+"¡Se acabó el juego!", "Fin", JOptionPane.INFORMATION_MESSAGE);
            game.setFocusable(false);
            JOptionPane.showMessageDialog(this, "Estadísticas de la partida:"+"\n"+"Tamaño máximo de la serpiente: "+String.valueOf(hungrySnake.size())+"\n"+"Pasos dados: "+ String.valueOf(stepsNumber)+"\n"+"Frut
        }
        else{
            JOptionPane.showMessageDialog(this, "Ha ocurrido un error: " + e.getMessage(), "Error",JOptionPane.ERROR_MESSAGE );
        }

    }
}
```

```
if(!hungrySnake.move(direction)){
    ended = true;
    lastDirection = direction;
```

Adicionalmente, se implementaron las siguientes mejoras en la jugabilidad:

Cambio de color y ojos al perder: Cuando la serpiente se estrella o sale de los límites, se invoca el método decreaseSnakeColor() para aclarar su color y se activa la bandera ended = true para reflejar el estado final en la cabeza de la serpiente:

```
private void decreaseSnakeColor(){
    colorBeforeDeath = snakeColor;
    int red = snakeColor.getRed();
    int green = snakeColor.getGreen();
    int blue = snakeColor.getBlue();

    snakeColor = new Color((int)(red+(255-red)*0.5), (int)(green+(255-green)*0.5), (int)(blue+(255-blue)*0.5));
}
```

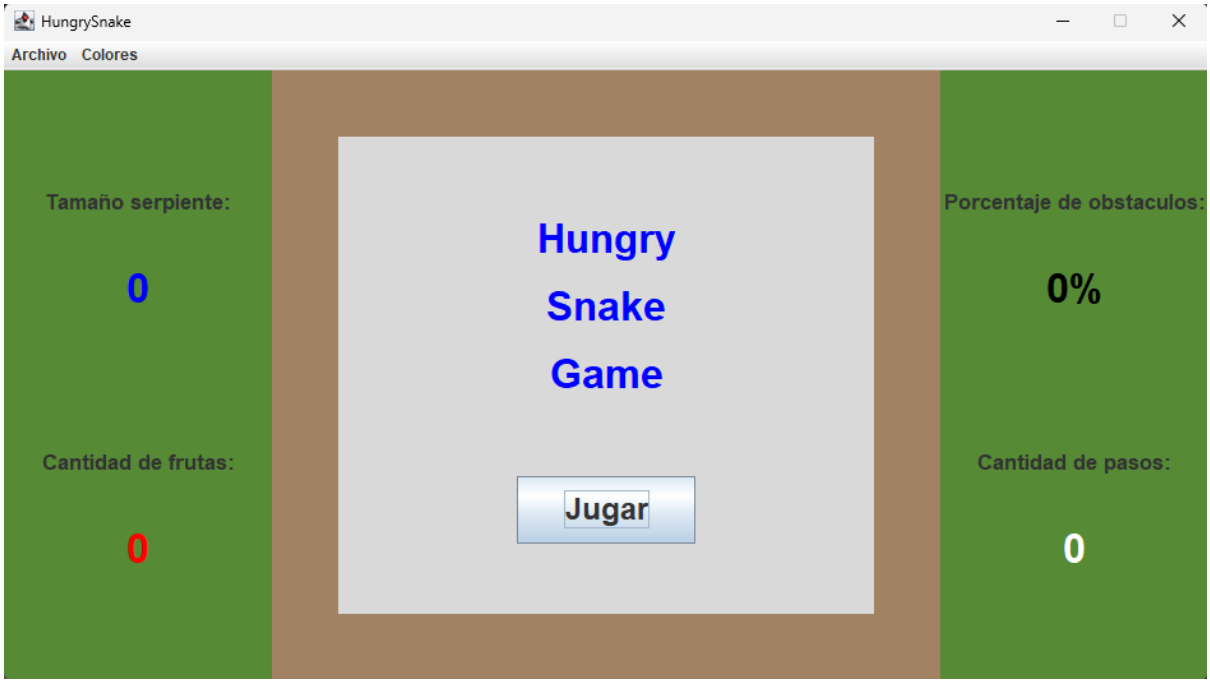
- Estadísticas de fin de partida: Al terminar el juego (ya sea por perder o al capturar la excepción HungrySnakeException.WIN), se despliega un JOptionPane con las estadísticas alcanzadas: tamaño máximo de la serpiente, pasos dados, cantidad de frutas y porcentaje de obstáculos.
- Nueva partida desde el menú: Se conectó el JMenuItem nuevo con startGameAction() para permitir configurar e iniciar una partida nueva en cualquier momento, restaurando el contador de pasos y el color original de la serpiente (colorBeforeDeath):

-
```
private void startGameAction(){
    stepsNumber = 0;
    String newFruitsNumber = JOptionPane.showInputDialog(this, "Dame la cantidad de frutas que va a tener: ");
    if(newFruitsNumber != null){
        try{
            fruitsNumber = Integer.parseInt(newFruitsNumber);
        }
        catch(NumberFormatException e){
            JOptionPane.showMessageDialog(this, "El texto no es un número válido." + "\n"+"Se pondrá por defecto 1", "No es un número válido", JOptionPane.ERROR_MESSAGE);
        }
    }
    else{
        fruitsNumber = 1;
    }
    String newWallsNumber = JOptionPane.showInputDialog(this, "Dame el porcentaje de obstáculos que va a tener: ");
    if(newWallsNumber != null){
        try{
            obstaclesNumber = Double.parseDouble(newWallsNumber);
        }
        catch(NumberFormatException e){
            JOptionPane.showMessageDialog(this, "El texto no es un número válido."+ "\n"+"Se pondrá por defecto 2%", "No es un número válido", JOptionPane.ERROR_MESSAGE);
        }
    }
    else{
        obstaclesNumber = 2.0;
    }
        try{
          hungrySnake = new HungrySnake(rows, columns, fruitsNumber, obstaclesNumber);
          snakeColor = colorBeforeDeath;
          length.setForeground(snakeColor);
          ended = false;
         }
        catch(HungrySnakeException e){
            JOptionPane.showMessageDialog(this, "Ha ocurrido un error: " + e.getMessage(), "Error",JOptionPane.ERROR_MESSAGE );
            startGameAction();
        };
    lastDirection = 'E';

    game.setFocusable(true);
    game.requestFocusInWindow();
    game.removeAll();
    prepareGame();
    refresh();
}
```

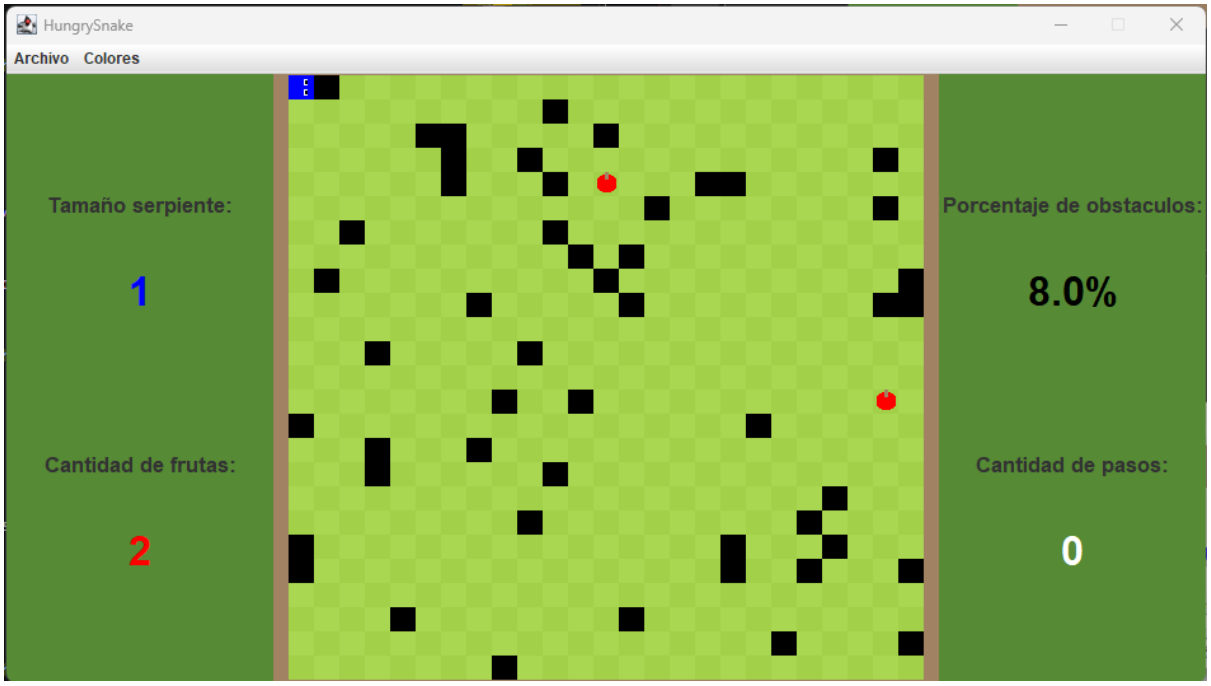
3. Ejecuten el caso de uso y capture las pantallas más significativas.

Se inicia el juego:

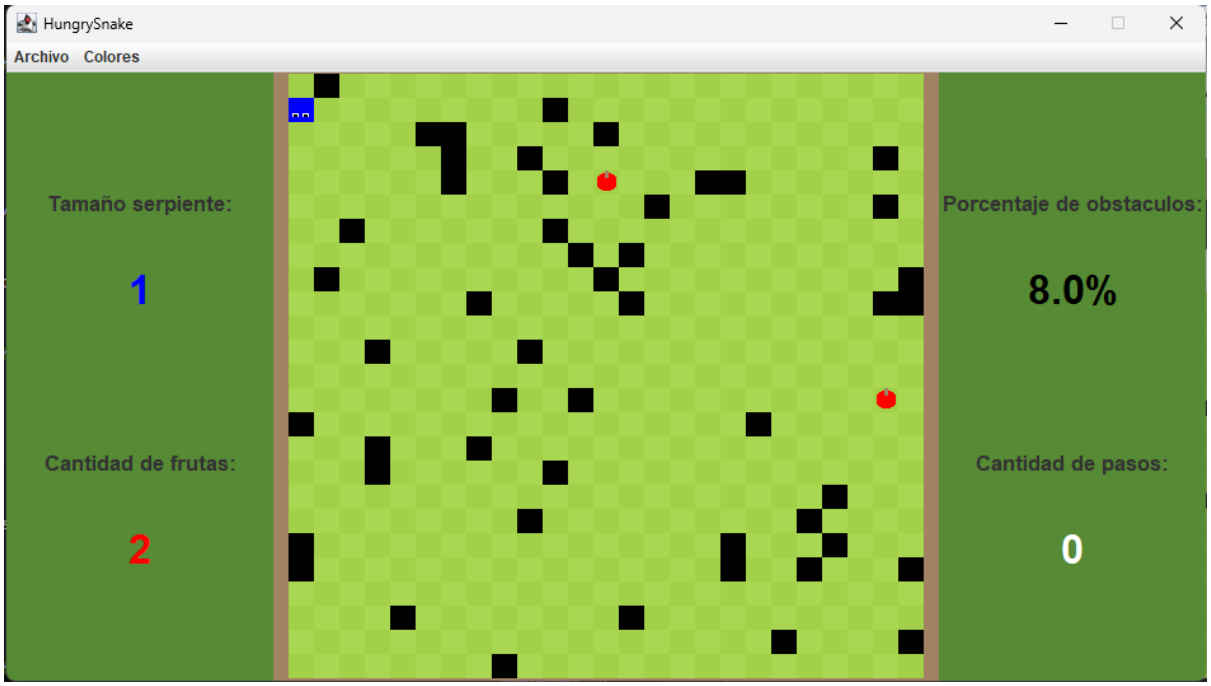



HungrySnake
Archivo Colores
Tamaño serpiente:
0
Cantidad de frutas:
0
Hungry
Entrada
Dame la cantidad de frutas que va a tener:
2
Aceptar
Cancelar
Jugar
Porcentaje de obstaculos:
0%
Cantidad de pasos:
0

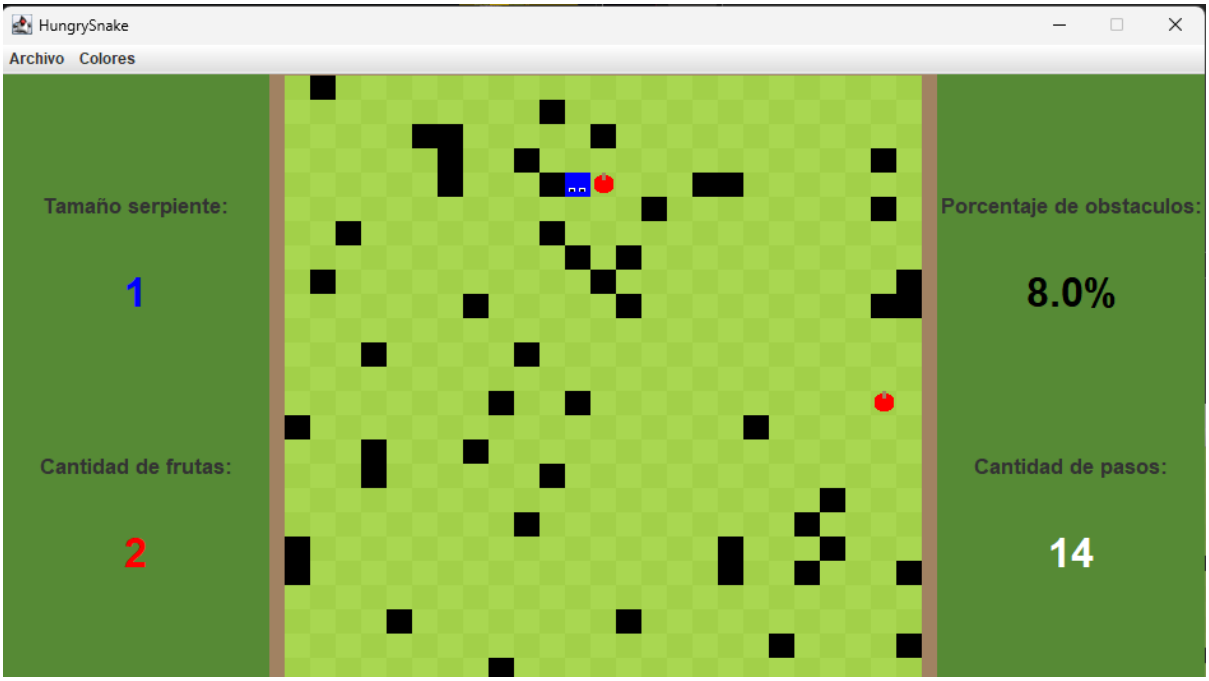

HungrySnake
Archivo Colores
Tamaño serpiente:
0
Cantidad de frutas:
0
Hungry
Entrada
Dame el porcentaje de obstáculos que va a tener:
8
Aceptar
Cancelar
Jugar
Porcentaje de obstaculos:
0%
Cantidad de pasos:
0



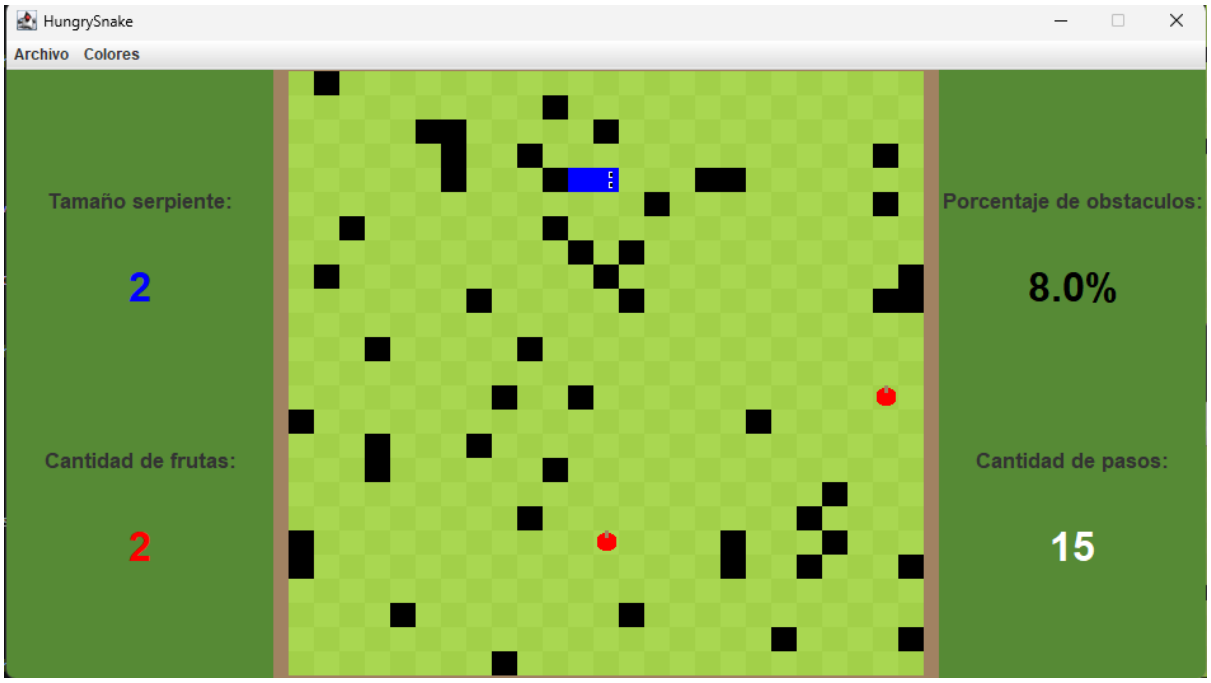
-Al darle la flecha hacia abajo:



- Luego, al ir hasta la fruta mas cerca:



Y comerla:

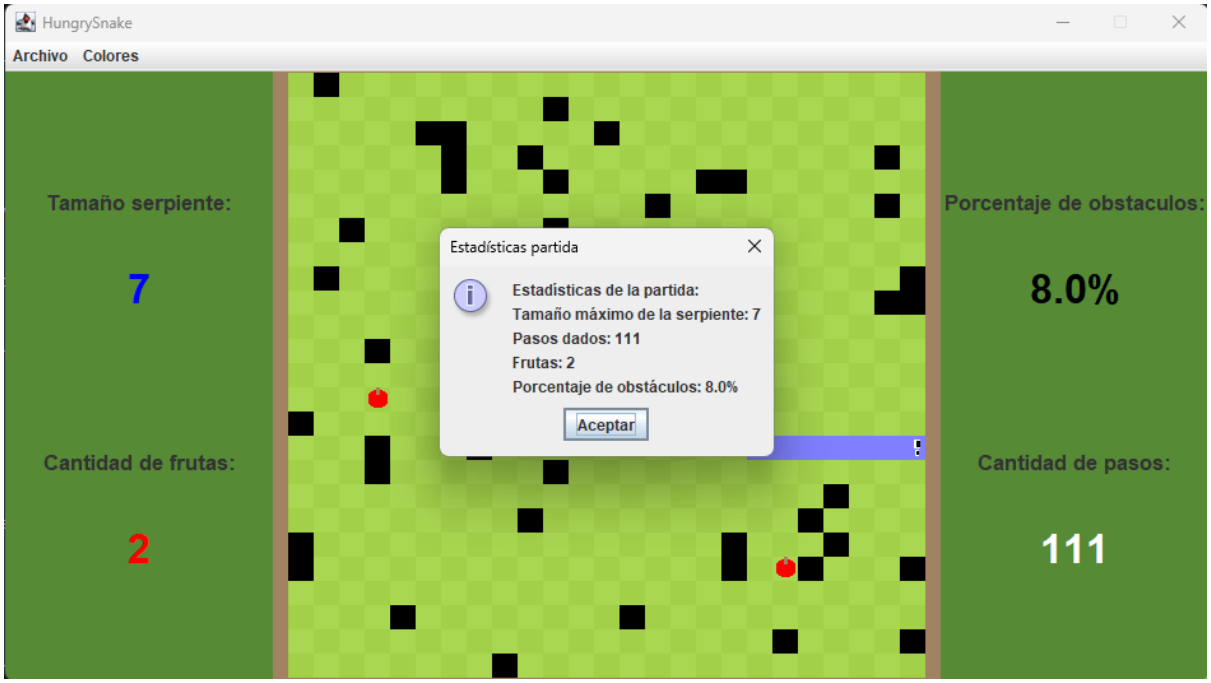


Haciendo eso varias veces:

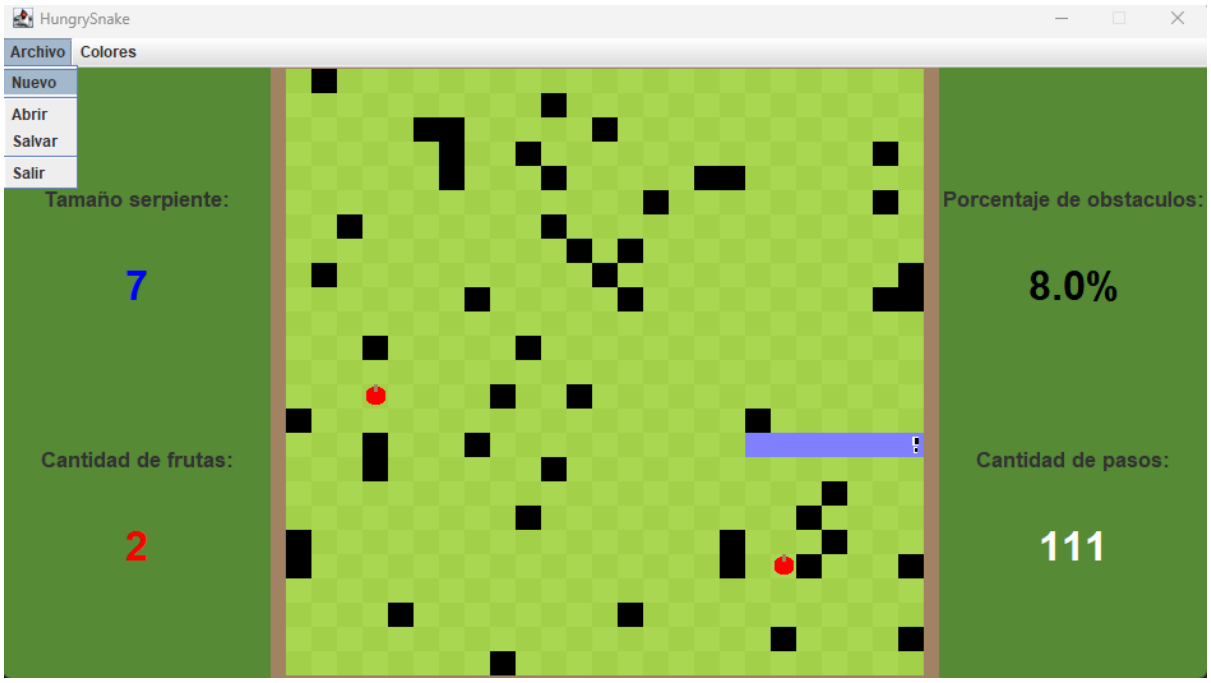


Siempre se mantienen las 2 frutas en el tablero, tambien se ve como se agranda la serpiente al comer fruta, luego, mencionado lo que se implementó:





Ahora, al darle a lo mencionado de "Nuevo":

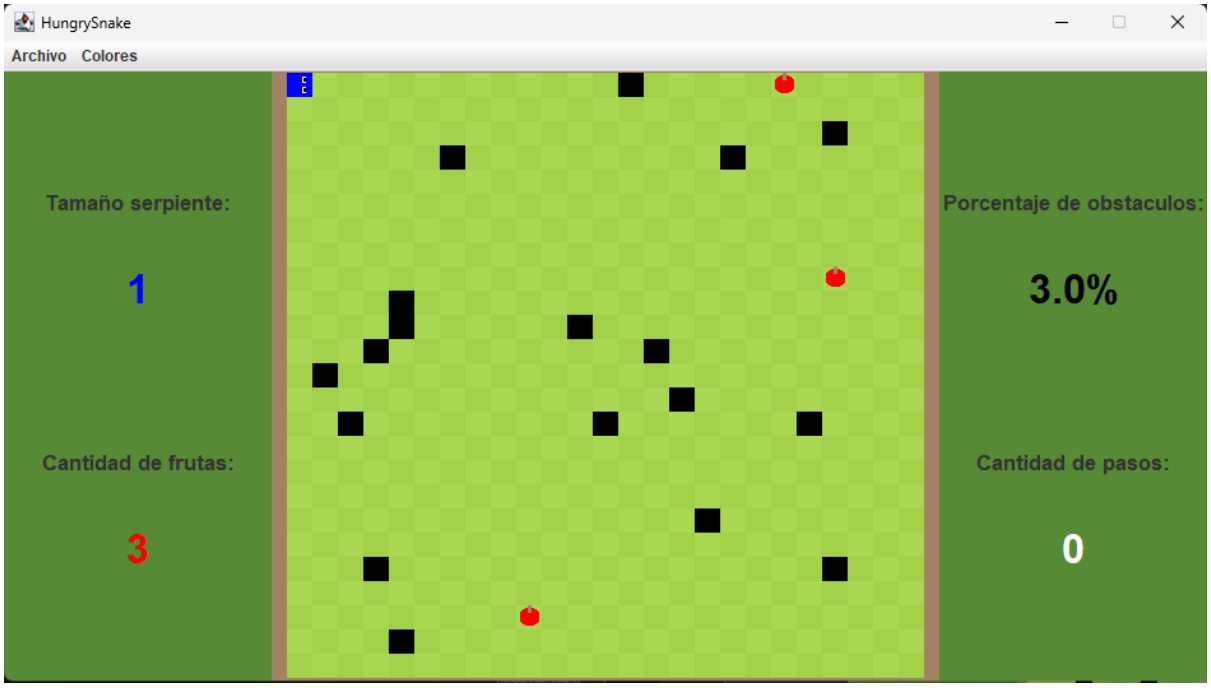


Nos vuelven a aparecer las ventanas (JOpcionPane) para asignar nuevamente la cantidad de frutas y el porcentaje de obstaculos:





Finalmente inicia el nuevo juego con las nuevas caracteristicas dadas:



Parte III. BONO: Funcionalidades complementarias

A Ciclo 7: Reiniciar

El objetivo es implementar este caso de uso.

1. Expliquen los elementos a usar para implementar este caso de uso.

Se agrega un nuevo JMenuItem llamado "Reiniciar" para que pertenezca dentro de el mismo JMenuBar que se ha venido trabajando

```
//Menu items
private JMenuItem nuevo;
private JMenuItem abrir;
private JMenuItem salvar;
private JMenuItem salir;
private JMenuItem colores;
private JMenuItem reiniciar;
```

Luego se agrega en JMenuBar dentro de preapreViewMenu() para que se coloque junto a la otra opcion de Nuevo:

```
private JMenuBar prepareViewMenu(){
    nuevo = new JMenuItem("Nuevo");
    abrir = new JMenuItem("Abrir");
    salvar = new JMenuItem("Salvar");
    salir = new JMenuItem("Salir");
    reiniciar = new JMenuItem("Reiniciar");
    colores = new JMenuItem("Cambiar Color");

    JMenuBar barra = new JMenuBar();
    JMenu archivo = new JMenu("Archivo");
    JMenu color = new JMenu("Colores");
    archivo.add(nuevo);
    archivo.add(reiniciar);
    archivo.addSeparator();
    archivo.add(abrir);
    archivo.add(salvar);
    archivo.addSeparator();
    archivo.add(salir);
    color.add(colores);



    barra.add(archivo);
    barra.add(color);
    return barra;
}
```

Luego, se debe de implementar un oyente para esta nueva accion..

2. Implementen los elementos necesarios para reiniciar

El nuevo oyente dentro de prepareControllerMenu:

```java
private void prepareControllerMenu(){
    salir.addActionListener(new ActionListener(){

        public void actionPerformed(ActionEvent e){
            exitAction();
        }
    });

    salvar.addActionListener(new ActionListener(){

        public void actionPerformed(ActionEvent e){
            saveAction();
        }
    });

    abrir.addActionListener(new ActionListener(){

        public void actionPerformed(ActionEvent e){
            openAction();
        }
    });
    nuevo.addActionListener( new ActionListener(){
        public void actionPerformed(ActionEvent e){
            startGameAction();
        }
    });
    reiniciar.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent e){
            restartGameAction();
        }
```

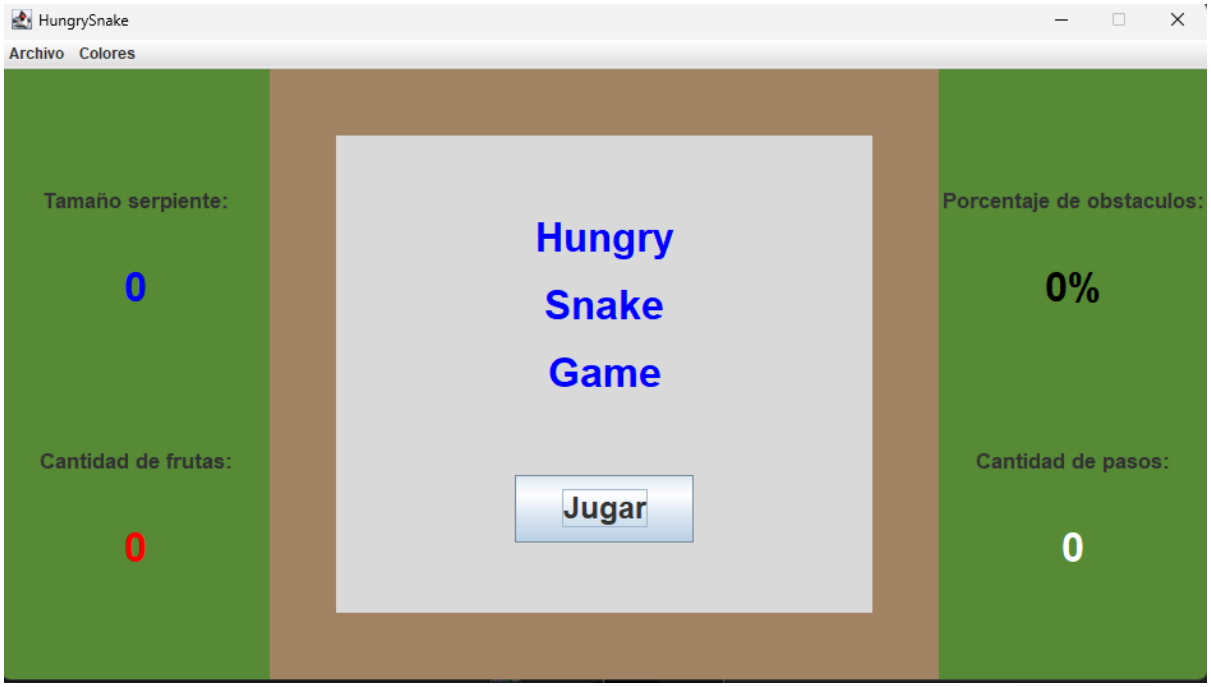
Luego se implementa el nuevo metodo restartGameAction();

```java
private void restartGameAction(){
    try {
        if(lastDirection != '\u0000'){
            hungrySnake = new HungrySnake(rows, columns, fruitsNumber, obstaclesNumber);
            lastDirection = 'E';
            snakeColor = colorBeforeDeath;
            length.setForeground(snakeColor);
            ended = false;
            stepsNumber = 0;
            game.setFocusable(true);
            game.requestFocusInWindow();
            game.removeAll();
            prepareGame();
            refresh();
        }else{
            JOptionPane.showMessageDialog(this, "Ha ocurrido un error: No ha se ha iniciado la partida" , "Error",JOptionPane.ERROR_MESSAGE );
        }

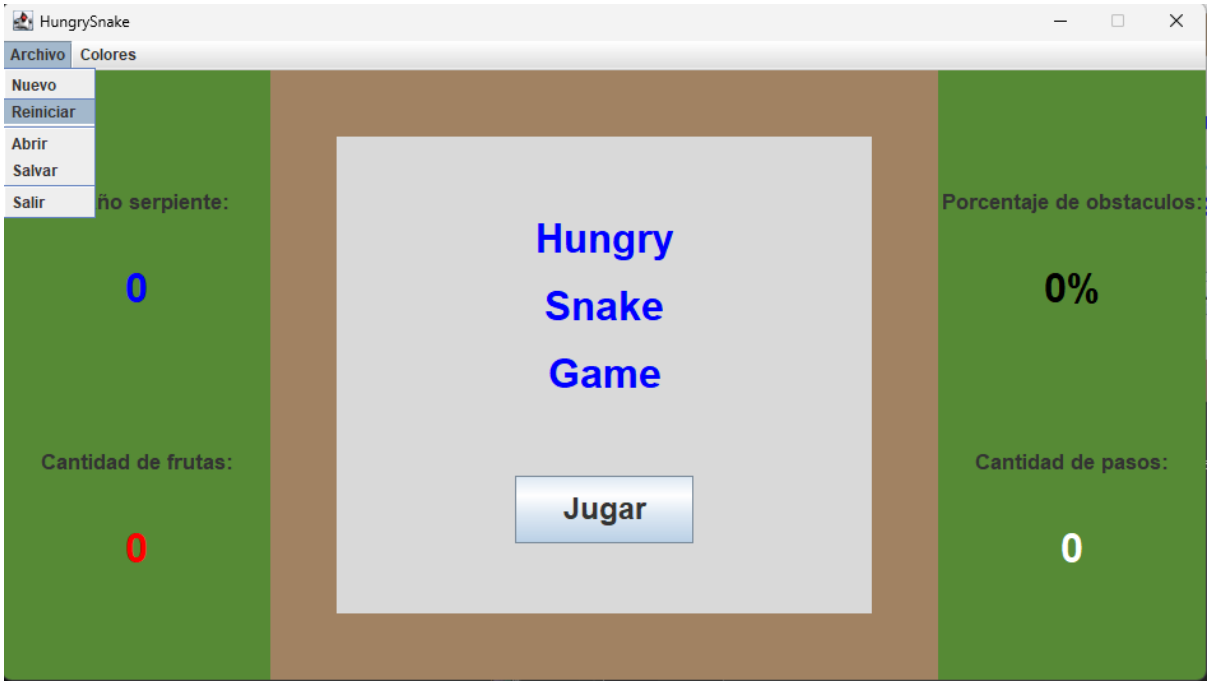
    } catch (HungrySnakeException e) {
        JOptionPane.showMessageDialog(this, "Ha ocurrido un error: " + e.getMessage(), "Error",JOptionPane.ERROR_MESSAGE );
    }
}
```

3. Ejecuten el caso de uso y capture las pantallas más significativas.

Se inicia el juego:



Si se intenta darle en reiniciar antes de empezar una partida:

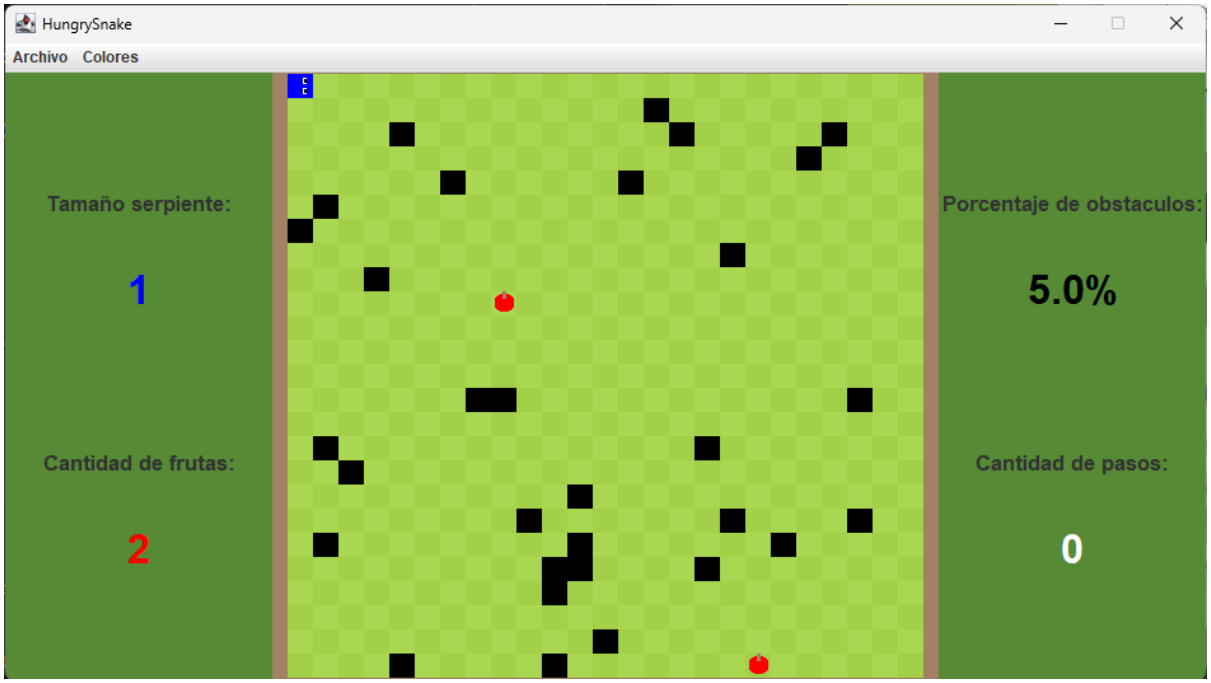




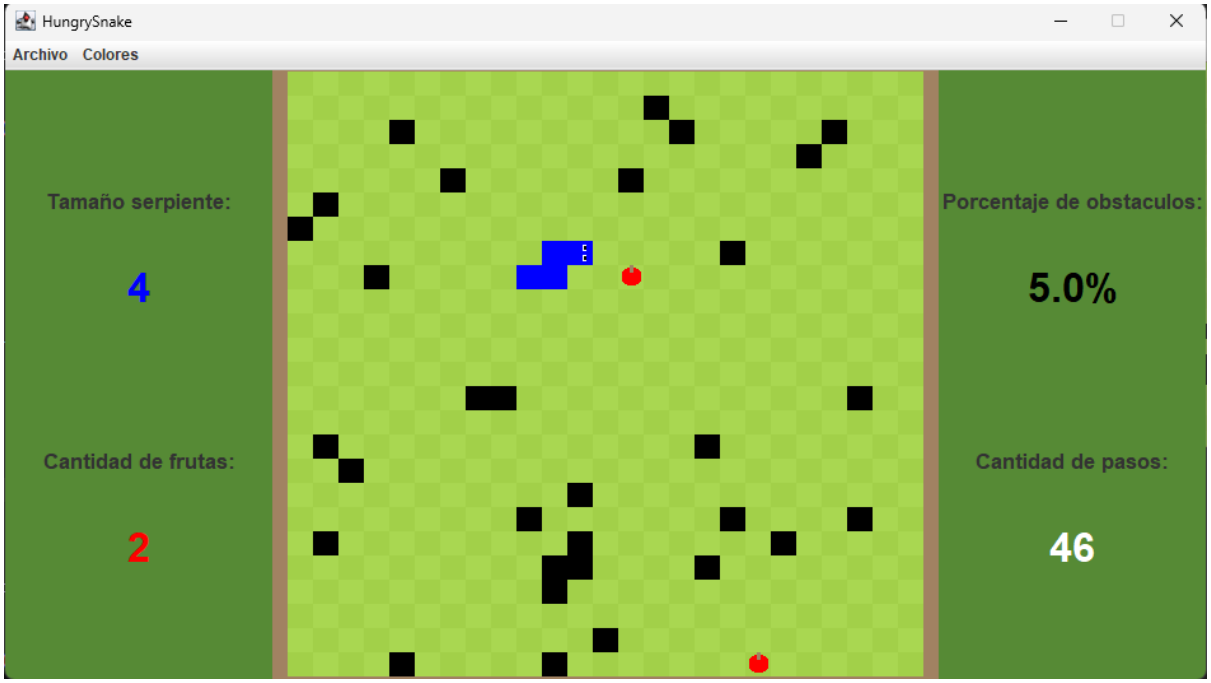
Luego al darle en jugar:



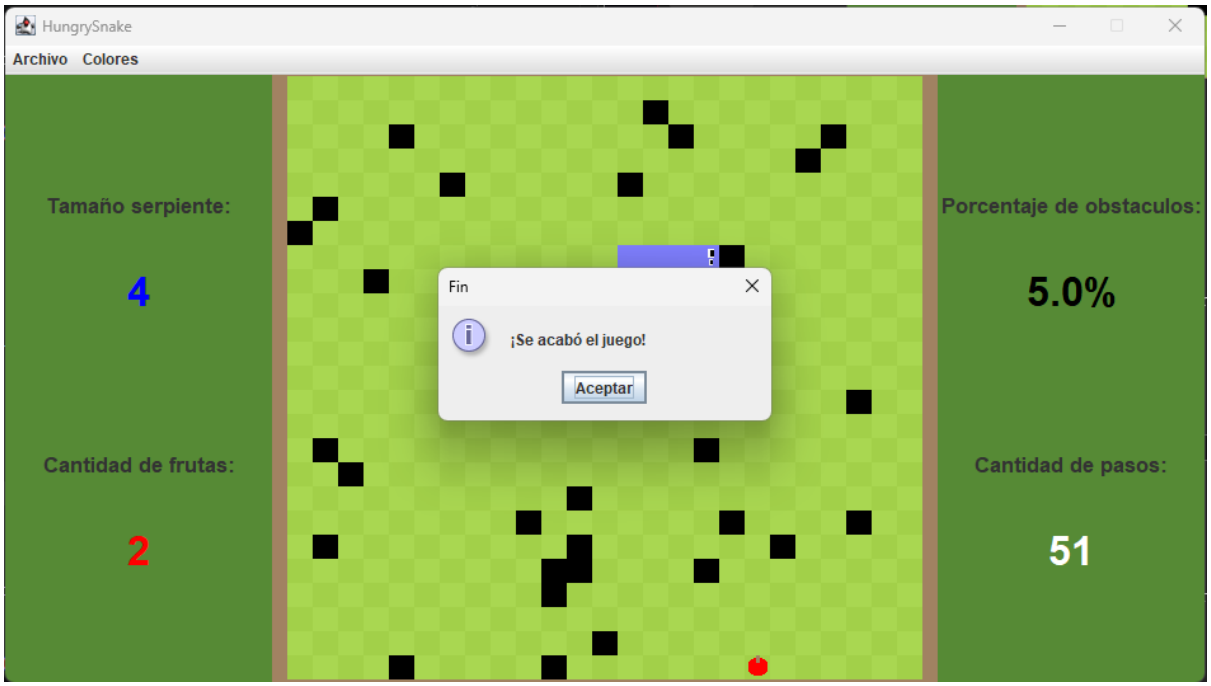




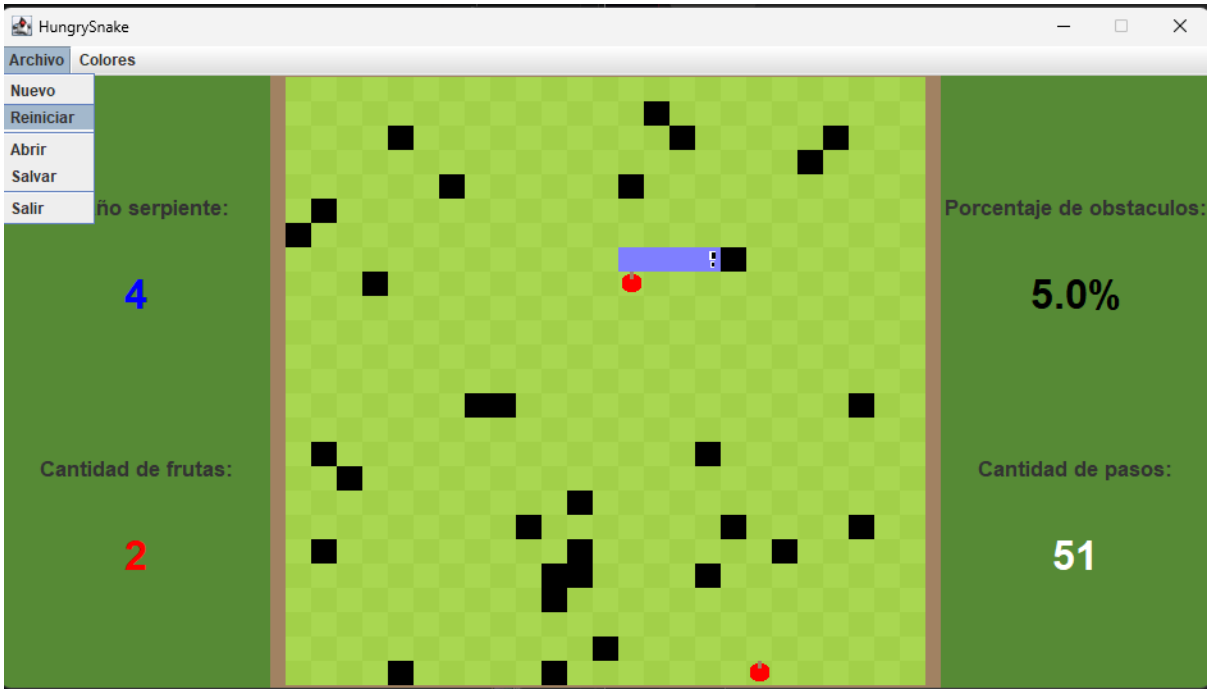
-Ahora si se empieza a jugar:

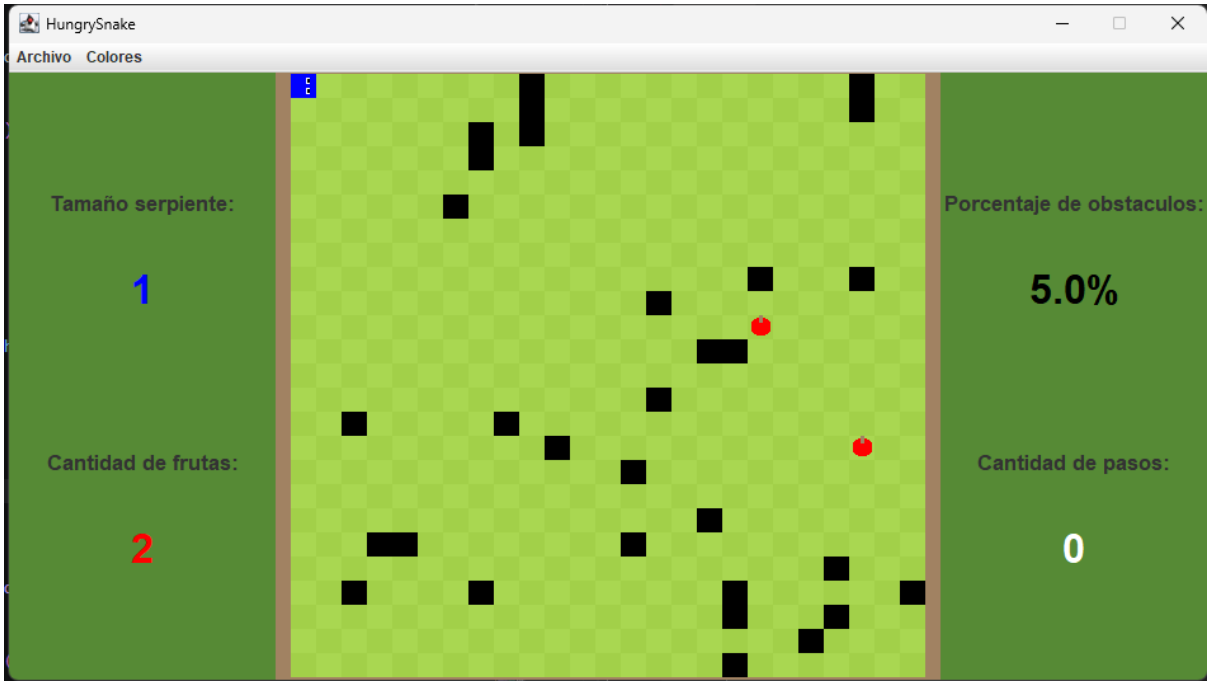


Y se estrella:



Luego se le da en reiniciar, se deben de guardar los valores que ingreso el usuario, el cual fueron 2 para cantidad de frutas y 5 para porcentaje de obstaculos:





B Ciclo 8: Cambiar el tamaño

El objetivo es implementar este caso de uso.

1. Expliquen los elementos a usar para implementar este caso de uso.
   Los elementos que se usaron fueron dos atributos que ya estaban implementados antes:
   Tanto el rows como el columns:

```
//Game attributes
private int rows = 25;
private int columns = 25;
```

Por otro lado, solo es necesario preguntarlos.
Además la idea para evitar deformaciones en los personajes es que se muestre todo el tablero, como si fuera completo, pero la parte no usada se pone de color cafe, como si fuera tierra. O sea, si el tablero es 3x10. Se dibuja como 10x10 pero los cuadrados que no se deberían usar se pintan de color cafe.
De resto realmente ya el metodo que se encarga de pintar todo, ya lo maneja. Sin mucha dificultad.

2. Implementen los elementos necesarios para cambiar el tamaño del juego

En un inicio es necesario preguntar las filas y columnas que va a tener el tablero:

```
String newRows = JOptionPane.showInputDialog(this, "Dame la cantidad de filas del mapa: ");
if(newRows != null){
    try{
        rows = Integer.parseInt(newRows);
    }
    catch(NumberFormatException e){
        JOptionPane.showMessageDialog(this, "El texto no es un número válido."+ "\n"+"Se pondrá por defecto 25", "No es un número válido", JOptionPane.ERROR_MESSAGE);
    }
}
else{
    rows = 25;
}
String newColumns = JOptionPane.showInputDialog(this, "Dame la cantidad de columnas del mapa: ");
if(newColumns != null){
    try{
        columns = Integer.parseInt(newColumns);
    }
    catch(NumberFormatException e){
        JOptionPane.showMessageDialog(this, "El texto no es un número válido."+ "\n"+"Se pondrá por defecto 25", "No es un número válido", JOptionPane.ERROR_MESSAGE);
    }
}
else{
    columns = 25;
}
```

Además, aunque no es obligatorio, decidimos añadir un mensaje para mostrarle al usuario qué debe hacer para jugar:

```
JOptionPane.showMessageDialog(this, "Mueve la serpiente con las flechas del teclado, intenta comer la mayor cantidad de frutas en el menor número de pasos sin estrellarte.", "Indicacior
```

Además es necesario cambiar la función que crea el tablero para que ponga el piso de color café en dado caso que exceda los limites de tablero que definió el usuario.

```java
private void prepareGame(){
    int maxSize = Math.max(rows, columns);
    game.setLayout(new GridLayout(maxSize, maxSize));
    board = new JPanel[maxSize][maxSize];


    for(int i = 0; i < maxSize; i++){
        for(int j = 0; j < maxSize; j++){
            board[i][j] = new JPanel();
            if(i>=rows || j >=columns){
                board[i][j].setBackground(new Color(161, 130,98));
            }
            else if((i+j)%2 == 0){
                board[i][j].setBackground(firstFloorColor);
            }
            else{
                board[i][j].setBackground(secondFloorColor);
            }
            board[i][j].setLayout(new BorderLayout());
            game.add(board[i][j]);
        }
    }
}
```

Para que ahora cambie la cuadricula y la ponga de tamaño máximo.

Ejecuten el caso de uso y capture las pantallas más significativas.

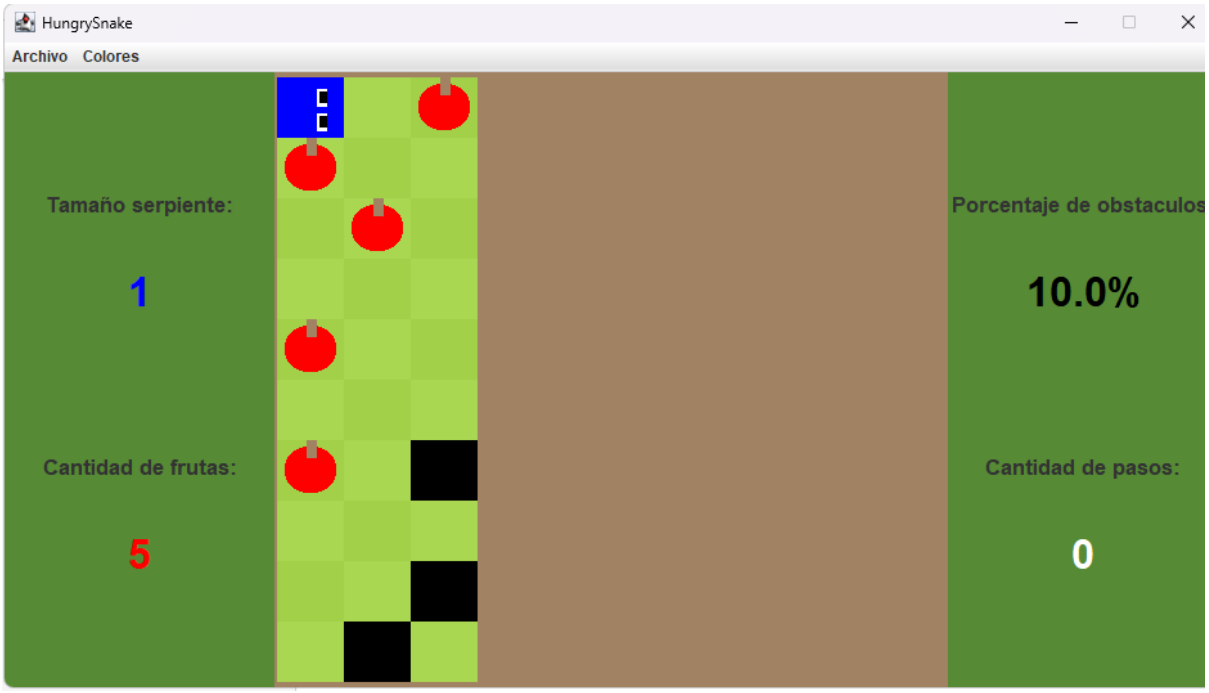


Si quiero poner 10 filas:



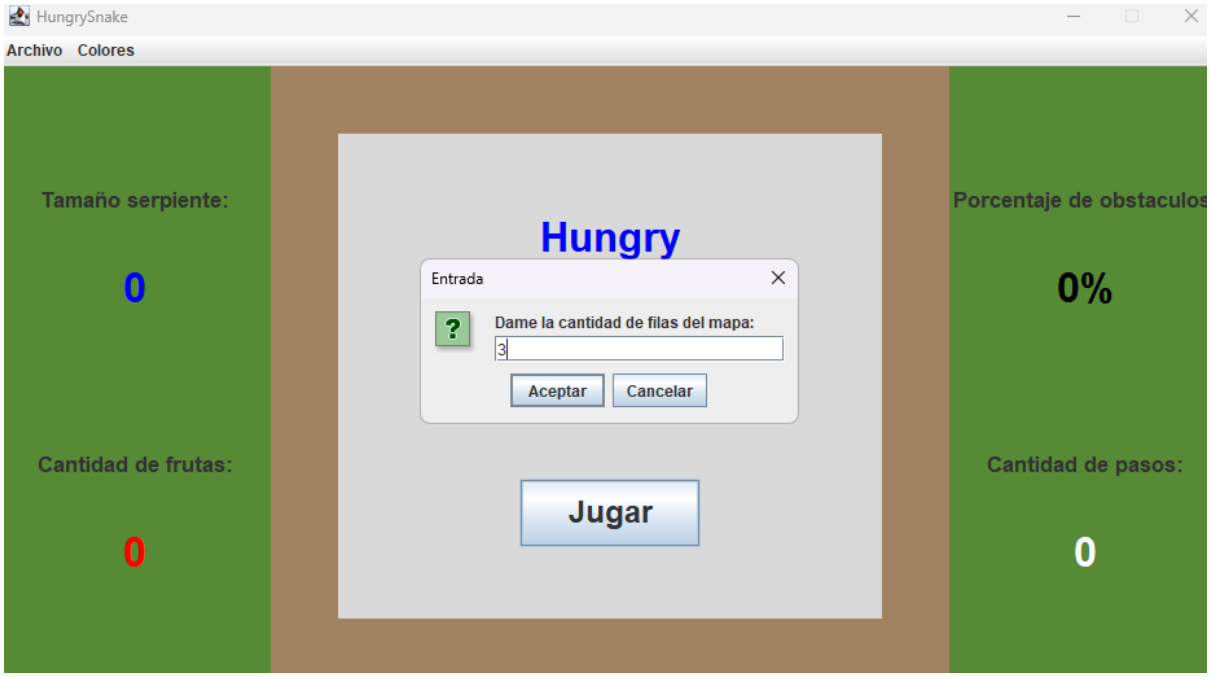

HungrySnake
Archivo Colores
Tamaño serpiente:
0
Cantidad de frutas:
0
Hungry
Entrada
Dame la cantidad de columnas del mapa:
3
Aceptar
Cancelar
Jugar
Porcentaje de obstaculos
0%
Cantidad de pasos:
0


HungrySnake
Archivo Colores
Tamaño serpiente:
0
Cantidad de frutas:
0
Hungry
Indicaciones
Mueve la serpiente con las flechas del teclado, intenta comer la mayor cantidad de frutas en el menor número de pasos sin estrellarte.
Aceptar
Jugar
Porcentaje de obstaculos:
Cantidad de pasos:
0

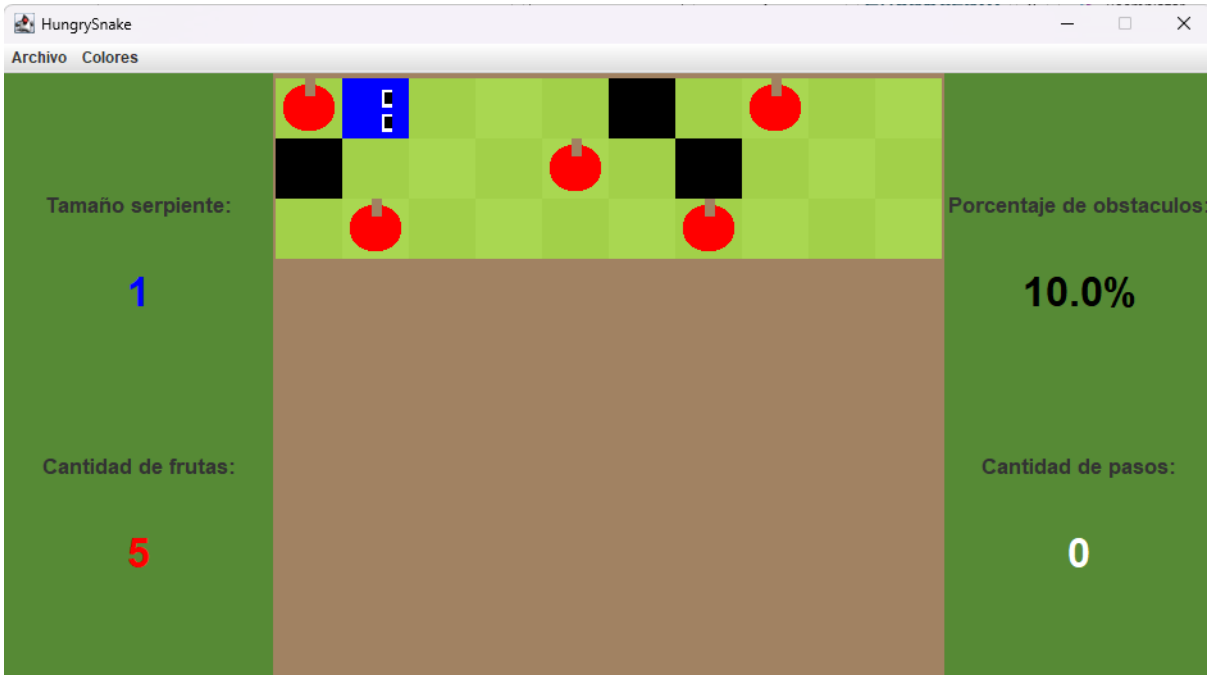

HungrySnake
Archivo Colores
Tamaño serpiente:
1
Cantidad de frutas:
5
Porcentaje de obstaculos
10.0%
Cantidad de pasos:
0

O podría ser:

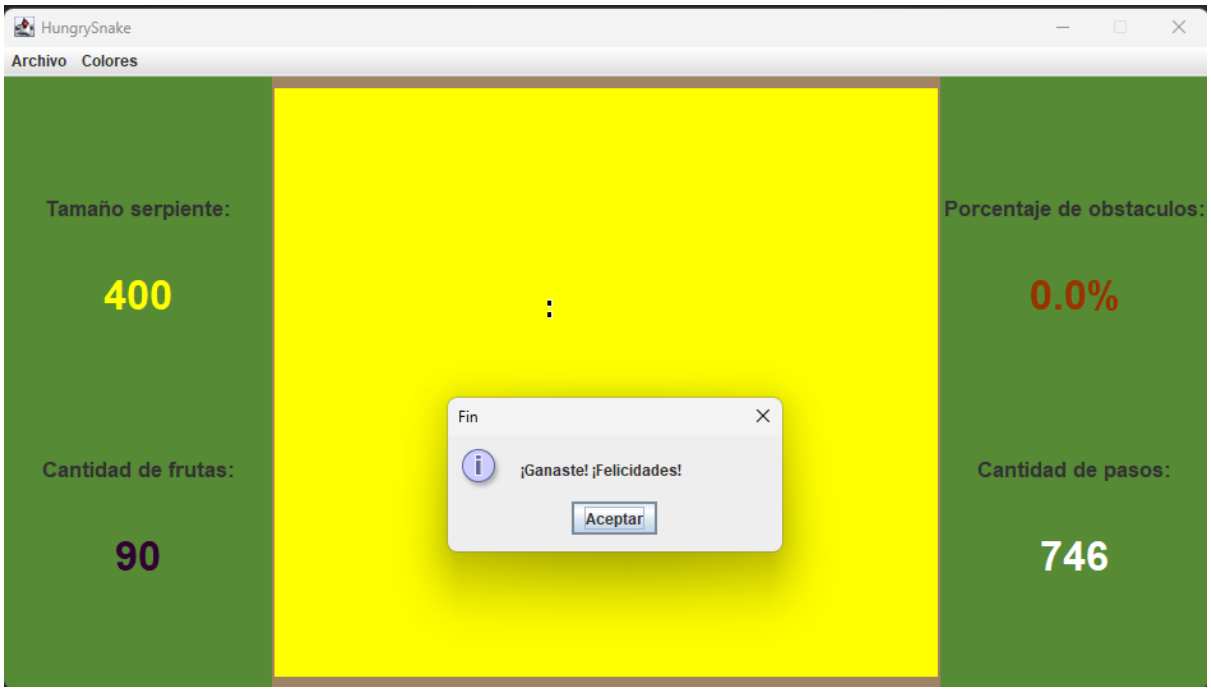

HungrySnake
Archivo Colores
Tamaño serpiente:
0
Cantidad de frutas:
0
Hungry
Snake
Game
Jugar
Porcentaje de obstaculos:
0%
Cantidad de pasos:
0


HungrySnake
Archivo Colores
Tamaño serpiente:
0
Cantidad de frutas:
0
Hungry
Entrada
Dame la cantidad de filas del mapa:
3
Aceptar
Cancelar
Jugar
Porcentaje de obstaculos
0%
Cantidad de pasos:
0


HungrySnake
Archivo Colores
Tamaño serpiente:
0
Cantidad de frutas:
0
Hungry
Entrada
Dame la cantidad de columnas del mapa:
10
Aceptar
Cancelar
Jugar
Porcentaje de obstaculos:
0%
Cantidad de pasos:
0



En el caso en que se gana:



Por último fue necesario modificar una prueba y añadir una prueba más para el tema del tamaño:

```java
/**
 * Boards shouldn't be bigger than 50x50, includes you shouldn't put more than 50 rows or more than 50 columns
 */
@Test
public void shouldNotMakeMapsSoBig(){
    try{
        HungrySnake game = new HungrySnake(row: 51,column: 51,fruits: 0,obstacles: 0);
        fail(message: "Didn't throw an exception");
    }
    catch(HungrySnakeException e){
        assertEquals(e.getMessage(), HungrySnakeException.BOARD_TOO_BIG);
    }
}
```

```java
/**
 * Should win if there's no more space for fruits
 */
@Test
public void shouldWinIfThereIsNoMoreSpaceForFruits(){
    try{
        HungrySnake game = new HungrySnake(row: 2,column: 1,fruits: 1,obstacles: 0);
        char[][] board = game.board();
        int snakePosY = 0;
        int fruitPosY = 0;
        for(int i = 0; i<2; i++){
            if(board[i][0] == 'F'){
                fruitPosY = i;
            }
            else if(board[i][0] == 'H'){
                snakePosY = i;
            }
        }
        if(snakePosY<fruitPosY){
            for(int i =0; i < fruitPosY-snakePosY; i++){
                game.move(direction: 'S');
            }
        }
        else{
            for(int i =0; i < snakePosY-fruitPosY; i++){
                game.move(direction: 'N');
            }
        }
        fail(message: "Didn't throw an exception");
    }
    catch(HungrySnakeException e){
        assertEquals(HungrySnakeException.WIN, e.getMessage());
    }
}
```

Al ejecutar los tests:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 12\Garzon-Mendivelso\HungrySnake>java -cp "bin;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.runner.
JUnitCore test.HungrySnakeTest
JUnit version 4.13.2
..............
Time: 0,027

OK (14 tests)


C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 12\Garzon-Mendivelso\HungrySnake>
```

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)

- Por cada uno tuvimos 40 horas invertidas en el laboratorio

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

-El estado actual del laboratorio es finalizado, ya que pudimos coordinar y efectuar cada uno de los puntos asignados

3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?

- Pair programming, porque fuimos codificando al tiempo y corrigiendo errores que de pronto el otro no se fijaba.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

- el mayor logro fue que el bosquejo que pensabamos en el inicio es igual o mejor que el que se dejó al hacer HungrySnakeGUI

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

- El mayor problema fue el tema de cuadrar el tablero ajustado medidas para que se viera bien, se resolvió pensando y usando tiempo para ello

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

- Hicimos bien como equipo hacer las practicas XP durante el laboratorio, ya que nos ayudo a diseñar y mejorar nuestro codigo. Nos comprometemos a mejorar los tiempos de desarrollo.

7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Usamos más que todo referencias de oracle y el estandar que tiene la profesora Irma en las notas de clase, la más útil fue el estandar de codificación para entender bien qué era lo que tocaba hacer, o pues, saber como lo hacía la profesora Irma.

Referencias

A visual guide to layout managers. (n.d.). Oracle.com. Retrieved October 31, 2026, from https://docs.oracle.com/javase/tutorial/uiswing/layout/visual.html

How to write a key listener. (n.d.). Oracle.com. Retrieved October 31, 2026, from https://docs.oracle.com/javase/tutorial/uiswing/events/keylistener.html

How to write window listeners. (n.d.). Oracle.com. Retrieved October 31, 2026, from https://docs.oracle.com/javase/tutorial/uiswing/events/windowlistener.html

Lesson: Performing custom painting. (n.d.). Oracle.com. Retrieved October 31, 2026, from https://docs.oracle.com/javase/tutorial/uiswing/painting/index.html

Lesson: Using Swing components. (n.d.). Oracle.com. Retrieved October 31, 2026, from https://docs.oracle.com/javase/tutorial/uiswing/components/index.html

The list interface. (n.d.). Oracle.com. Retrieved October 31, 2026, from https://docs.oracle.com/javase/tutorial/collections/interfaces/list.html